

Masters Degree Project

Designing a Reference Architecture for Secure Multi-Agent Systems

Security, Auditability, and Governance for Tool-Using Enterprise Agentic Workflows



Author: Niko Kauz
Supervisors: Hemant Ghayvat
Semester: VT 2026
Course: 5DV50E
Subject: Computer Science

Abstract

Context: Large Language Models (LLMs) are increasingly used in agentic systems that retrieve information, use tools and participate in multi-step enterprise workflows. While these systems create opportunities for automating knowledge-intensive processes, they also introduce risks related to security, governance, traceability and auditability. These risks are especially relevant in regulated domains where agents interact with sensitive data, documents, databases, APIs and customer-facing processes.

Objective: This thesis investigates how a secure and compliant Reference Architecture (RA) for tool-using Multi-Agent Systems (MAS) in enterprise environments can be designed. The objective is to identify architectural layers, components, trust boundaries and control mechanisms that support the controlled integration of dynamic LLM-based agents with static enterprise systems.

Method: The study follows a design-oriented research approach. Literature on MAS, LLM-based agents, tool use, Retrieval-Augmented Generation (RAG), security, governance and auditability is reviewed. These findings are combined with implementation experience and an expert questionnaire with 25 participants to derive requirements and design a secure-by-design RA. The architecture is instantiated through a household insurance claim-processing Proof of Concept (PoC) and complemented by a separate auditing artifact for tamper-evident traceability.

Results: The proposed RA combines orchestration, agent roles, knowledge access, integration, governance, security controls, audit evidence, observability and Human-in-the-Loop mechanisms into a layered model. The expert questionnaire confirmed the importance of security, compliance, auditability, traceability, controlled tool access and human oversight for enterprise adoption. The PoC showed that selected parts of the RA can be implemented in a realistic insurance workflow under controlled conditions and that predefined claim cases can be processed and routed to valid workflow outcomes. The workflow behaved conservatively by often requesting additional information instead of producing final decisions. The auditing artifact demonstrated that manipulated workflow records can be detected through record-hash mismatches and changed Merkle roots.

Conclusion: The thesis shows that secure enterprise MAS require more than orchestration frameworks and prompt-level controls. Security, auditability and governance must be treated as architectural concerns from the beginning. The proposed RA, PoC and auditing artifact provide a reusable foundation for designing secure and traceable tool-using MAS. However, the results remain limited by the controlled PoC scope, synthetic data, simplified retrieval strategy, conservative routing behaviour and the separation of the audit artifact from the live workflow.

Keywords: Secure Multi-Agent Systems, Reference Architecture, Large Language Models, Tool-using Agents, Enterprise Architecture, Governance, Auditability, Traceability, Merkle Trees, Insurance Claim Processing

AI Use Disclosure

Artificial Intelligence tools were used in the preparation of this thesis for supportive purposes only. These included text structuring, brainstorming, linguistic refinement, and the rephrasing or translation of selected passages. The intellectual content of the thesis, including the research design, scientific analysis, results, interpretation, discussion, and conclusions, remains solely the work and responsibility of the author.

OpenAI's ChatGPT was used for the supportive purposes described above and as a language-support tool to improve grammar, spelling, punctuation, readability and the clarity of selected sections. Its use did not involve the generation of original academic arguments, scientific analyses, research findings, interpretations or conclusions, and it did not alter the intended meaning of the author's work. DeepL was used exclusively for translation support.

All scientific reasoning, methodological decisions, critical assessments, evaluations and interpretations were conducted independently by the author. Any suggestions generated by AI tools and AI-assisted revision were critically reviewed and used only where they were consistent with the research context and the author's own academic judgment.

The use of AI in this thesis was guided by ethical standards and principles of good scientific practice including critical source evaluation and author accountability. Full responsibility for the content of this thesis rests with the author.

Contents

List of Figures vii

List of Tables ix

List of Abbreviations x

1 Introduction 1
- 1.1 Background . . . 2
  - 1.1.1 LLM-based and tool-using Agents . . . 2
  - 1.1.2 Multi-Agent Systems and Agentic AI . . . 2
  - 1.1.3 Orchestration Frameworks . . . 3
  - 1.1.4 Agentic Workflows in Enterprise Environments . . . 3
  - 1.1.5 Communication in Agentic Systems . . . 3
  - 1.1.6 Guardrails . . . 4
  - 1.1.7 Reference Architectures . . . 5
  - 1.1.8 Insurance claim processing as Application Domain . . . 5
  - 1.1.9 Security, Auditability and Compliance . . . 5
- 1.2 Problem Formulation . . . 6
- 1.3 Motivation . . . 7
- 1.4 Objectives . . . 8
- 1.5 Research Questions . . . 8
- 1.6 Contributions of the Work . . . 8
- 1.7 Scope and Limitations . . . 9
- 1.8 Target Audience . . . 10
- 1.9 Thesis Structure . . . 10

2 Related Work 11
- 2.1 Review of Existing Research . . . 11
  - 2.1.1 Classical Multi-Agent Systems . . . 11
  - 2.1.2 LLM-based Agents, Tool-use and Retrieval . . . 12
  - 2.1.3 LLM-based Multi-Agent Systems and Orchestration Frameworks 12
  - 2.1.4 Reference Architectures for LLM-based Multi-Agent Systems . . 14
  - 2.1.5 Security Risks in tool-using and RAG-enabled Agents . . . 14
  - 2.1.6 Governance, Compliance and AI Risk Management . . . 16
  - 2.1.7 Auditability and tamper-evident Evidence . . . 16
- 2.2 Comparison of Approaches . . . 17
- 2.3 Distinction from Existing Agent and Automation Platforms . . . 18
- 2.4 Identified Gaps and Positioning of this Work . . . 21
- 2.5 Summary . . . 21

3 Methodology 23
- 3.1 Research Approach . . . 24
  - 3.1.1 Architecture-centric DSR . . . 24
  - 3.1.2 Proof-of-Concept and Auditability as Research Artifacts . . . 24
  - 3.1.3 Mixed-method Evaluation . . . 25
- 3.2 Data Collection . . . 25
  - 3.2.1 Literature Selection and Review Process . . . 25
  - 3.2.2 Expert Questionnaire Design . . . 26

3.2.3 Selection of Experts . . . 26
3.2.4 Insurance Documents and policy versions . . . 27
3.2.5 Creation of synthetic customer data . . . 27
3.2.6 Design of Claim Cases . . . 27
3.3 Data Analysis . . . 27
3.3.1 Literature Synthesis . . . 28
3.3.2 Qualitative Analysis of Questionnaire Responses . . . 28
3.3.3 Evaluation of the Reference Architecture . . . 28
3.3.4 Case-based Analysis of PoC Outputs . . . 29
3.3.5 Evaluation of the Auditing Artifact . . . 29
3.4 Reliability and Validity . . . 29
3.5 Ethical Considerations . . . 30
3.6 Summary . . . 31

4 Reference Architecture and Artifact Design 32
4.1 Design Goals and Requirements . . . 32
4.2 Reference Architecture for secure Multi-Agent Systems . . . 32
4.3 Claim-Processing PoC Design . . . 37
4.4 Auditing Artifact Design . . . 45
4.4.1 Motivation for a separate Artifact . . . 46
4.4.2 Merkle tree Evidence Logging . . . 46
4.4.3 Blockchain-style Root Chaining . . . 50
4.4.4 Verification Logic . . . 51
4.5 Design Decisions and Trade-offs . . . 52
4.6 Summary . . . 52

5 Results and Analysis 55
5.1 Results . . . 55
5.1.1 Expert questionnaire results . . . 55
5.1.2 Reference Architecture for secure Multi-Agent Systems . . . 63
5.1.3 Insurance claim-processing PoC . . . 64
5.1.4 Auditing artifact results . . . 68
5.2 Analysis . . . 77
5.2.1 Cross-result Patterns and Significance . . . 77
5.2.2 Interpretation of the PoC . . . 78
5.2.3 Interpretation of the Auditing Artifact . . . 78
5.2.4 Synthesis against the Research Questions . . . 79
5.3 Summary of Findings . . . 79

6 Discussion 81
6.1 Interpretation and Significance of the Findings . . . 81
6.2 Assessment against Objectives and Research Questions . . . 82
6.2.1 Assessment against the Objectives . . . 82
6.2.2 Assessment against the Research Questions . . . 83
6.3 Comparison with Related Work . . . 84
6.4 Implications for Secure Enterprise and Regulated-Domain Design . . . 85
6.5 Limitations . . . 86
6.6 Generalisability of the proposed Reference Architecture . . . 87
6.7 Threats to Validity . . . 88
6.8 Summary . . . 89

7 Conclusion and Future Work **90**
7.1 Conclusion 90
7.2 Summary of Contributions 91
7.3 Answer to the Research Problem 91
7.4 Future Work 92

References **94**

A Expert Questionnaire: Reference Architecture for Secure Multi-Agent Systems **A**

B Claim Cases **M**

C Claim PoC WebApp, CrewAI Screenshots and Knowledge Source Overview **Q**

D Prompts, Tasks and Guardrails definitions **Y**

E Auditing Artifact Files, Figures and Examples **AZ**

F GitHub Repositories **BG**

List of Figures

3.1 Methodology: architecture-centric DSR process followed in this thesis . . 23
4.2 Proposed RA for secure tool-using MAS 34
4.3 Simplified claim-processing workflow 38
4.4 Implemented workflow in CrewAI Studio used in the claim-processing PoC 39
4.5 Supabase data model used in the claim-processing PoC as an entity relationship diagram 41
4.6 Detailed routing logic of the claim-processing workflow 44
5.7 Participants' experience in software and enterprise architecture 56
5.8 Participant familiarity with GenAI, LLMs and MAS 56
5.9 Enterprise use cases for tool-using MAS 56
5.10 Adoption blockers for tool-using MAS 57
5.11 Most important quality attributes for secure enterprise MAS 57
5.12 ASR for secure tool-using MAS 58
5.13 Critical threats for secure tool-using MAS 58
5.14 Relevant trust boundaries in secure tool-using MAS 59
5.15 Expert classification of architectural layers for secure MAS 59
5.16 Expert classification of logical components for a secure MAS 60
5.17 Evidence artifacts required for auditability in tool-using MAS 61
5.18 Importance of end-to-end traceability for secure tool-using MAS 61
5.19 Information required for useful end-to-end traceability 61
5.20 Useful viewpoints for communicating a secure RA 62
5.21 Useful architecture artifacts for communicating a secure RA 62
5.22 RA for secure MAS with Auditability 63
5.23 Implemented PoC architecture mapped to the proposed RA 64
5.24 Evidence chain of the auditing artifact 70
5.25 Truncated Merkle tree view for the original audit log, showing levels 3–7 71
5.26 Blockchain-style audit commitment structure 72
5.27 Truncated Merkle tree view for the malicious audit log, showing levels 3–7 74
5.28 Propagation effect of a tampered log node 75
5.29 Result for audit verification 77
C.1 WebApp claim input options . . . . . R
C.2 Example of a longer predefined claim case used for repeatable PoC evaluation . . . . . R
C.3 WebApp submission confirmation and result view . . . . . S
C.4 Console output showing the generated request . . . . . S
C.5 Predefined examples for policy-version and claim-detail comparison . . . S
C.6 WebApp validation and customer lookup error states . . . . . T
C.7 WebApp error state for an unsuccessful backend or workflow request . . . T
C.8 CrewAI Agent attributes and tool configuration . . . . . U
C.9 Configurable Agent settings in CrewAI . . . . . U
C.10 Agent instruction and context in CrewAI V
C.11 Agent response as output in CrewAI V
C.12 CrewAI Task configuration V
C.13 CrewAI Guardrail success and failure examples . . . . . W
C.14 CrewAI Automation view of a workflow execution stop (Exception) after repeated Guardrail validation failures . . . . . W

C.15 CrewAI Automation execution graph showing average token usage and execution time for workflow X
C.16 CrewAI Automation view trace details used to inspect task execution, outputs and validation behaviour X
E.17 Full Merkle tree for the original audit log . . . . . BE
E.18 Full Merkle tree for the malicious audit log . . . . . BF

List of Tables

1.1 Objectives of the thesis 8
1.2 Supporting RQs of the thesis 9
2.3 Comparison of related research categories and their relevance to this thesis 19
4.4 Design requirements for the proposed RA 33
4.5 Logical layers of the proposed RA 36
4.6 Agent roles and validation mechanisms in the claim-processing PoC . . . 41
4.7 Supabase data model description used in the claim-processing PoC 42
4.8 Routing actions used in the claim-processing PoC 44
4.9 Main field groups of a JSONL audit record 47
4.10 Fields of the Merkle commitment 49
4.11 Fields of a blockchain-style audit block 51
4.12 Verification steps of the auditing artifact 51
4.13 Design decisions and trade-offs for the claim-processing PoC 53
5.14 Overview of claim-processing PoC test cases, observed outcomes and execution metrics 67
5.15 Aggregated execution metrics for the predefined claim cases 68
5.16 Generated Merkle commitment for the audit log 69
5.17 Verification result for original and malicious audit logs 76
C.1 Overview of selected prepared policy documents in the PoC knowledge source . . . . . Q

List of Abbreviations

A2A	Agent-to-Agent
ACL	Access Control List
AES	Advanced Encryption Standard
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Architecturally Significant Requirement
DB	Database
DSR	Design Science Research
GCM	Galois/Counter Mode
GenAI	Generative Artificial Intelligence
HFU	Hochschule Furtwangen University
HITL	Human-In-The-Loop
IAM	Identity and Access Management
IdP	Identity Provider
IEC	International Electrotechnical Commission
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JSONL	JSON Lines
LLM	Large Language Model
LNU	Linnaeus University
MAS	Multi-Agent Systems
MCP	Model Context Protocol
NIST	National Institute of Standards and Technology
OIDC	OpenID Connect
PDF	Portable Document Format
PDP	Policy Decision Point
PEP	Policy Enforcement Point
PoC	Proof-of-Concept
RA	Reference Architecture
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RMF	Risk Management Framework
RQ	Research Question
SHA	Secure Hash Algorithm

1 Introduction

The rapid development of Large Language Models (LLMs) has changed how software systems process information, interact with users and support complex decision-making tasks. While early applications mainly focused on text generation, summarisation and question answering, newer agentic systems extend these capabilities through autonomy, goal-directed reasoning, planning and tool use. Such systems can retrieve information, call Application Programming Interfaces (APIs) and interact with digital environments as part of a workflow [1, 2, 3]. This creates new opportunities for automating knowledge-intensive processes in enterprise environments.

A particularly relevant direction is the use of Multi-Agent Systems (MAS), where multiple specialised agents cooperate to solve a larger task. Responsibilities can be distributed across agents with different roles, such as information retrieval, analysis, validation and communication. This builds on classical MAS concepts such as autonomy, coordination and role specialisation [4]. Recent LLM-based multi-agent frameworks show how these concepts can be applied to agentic workflows [5, 6, 7, 8]. However, such systems also introduce challenges related to coordination, control, reliability, security and auditability. Recent work shows that agentic risks can span inputs, tools, memory, workflow state, inter-agent communication and execution controls, which means that they cannot be addressed by one isolated defence [9, 10, 11, 12].

These challenges are especially important in enterprise and regulated domains. When agentic systems interact with Databases (DBs), documents, APIs or other organisational systems, they become part of broader socio-technical processes. Their outputs may influence decisions, customer communication or operational workflows. Therefore, such systems must not only be useful and flexible, but also traceable, secure, governable and aligned with compliance requirements [13, 14, 15].

Insurance claim processing provides a suitable application domain for investigating these challenges. Claim handling requires the combination of structured customer data, contractual documents, policy versions, claim descriptions and evidence-based reasoning. Decisions must be explainable and traceable to relevant policy conditions. This makes the domain appropriate for studying how LLM-based MAS can support enterprise workflows while maintaining transparency and accountability.

This thesis addresses the design of a secure-by-design Reference Architecture (RA) for tool-using MAS. The proposed architecture is instantiated through a Proof-of-Concept (PoC) for household insurance claim processing and complemented by a separate auditing artifact for tamper-evident traceability. The aim is not to develop a production-ready claims system, but to investigate how architectural principles, workflow design, evidence handling and auditability mechanisms can be combined to support secure and compliant agentic systems.

The work is consistent with RA research, which emphasises architectural knowledge reuse, maintainability and standardisation, while also recognising risks such as learning effort, dependency and architectural complexity [16]. It is also aligned with grounded architecture research, where architectural knowledge is derived from observed systems, stakeholder concerns and architecture-significant requirements [17, 18]. Related work has begun to address LLM-based MAS, trusted agentic pipelines, enterprise orchestration and audit trails, but security, compliance and audit evidence remain central concerns for enterprise-oriented design [19, 20, 21, 22].

1.1 Background

This subsection introduces the concepts required to understand the research problem addressed in this thesis. The work is situated at the intersection of Generative Artificial Intelligence (GenAI), LLMs, tool-using agents, MAS, enterprise software architecture and secure system design. These areas increasingly overlap as modern GenAI applications move beyond isolated chatbot interfaces toward systems that retrieve information, call tools, coordinate multiple steps and interact with enterprise environments.

1.1.1 LLM-based and tool-using Agents

LLMs are a central technology behind many current GenAI systems. They can generate, summarise, transform, classify and reason over natural language and structured information. Agentic systems extend this interaction pattern by using the model as a reasoning and decision-support component inside a larger workflow. Instead of only producing text, an agent may retrieve documents, call tools, invoke APIs and continue execution based on intermediate results [1, 2, 9].

Tool use is therefore a key difference between ordinary chatbot-style systems and agentic systems. A tool-using agent may interact with DBs, document repositories, business APIs, workflow systems or tools such as calculators and mail services. This makes agents useful for enterprise tasks, but it also increases risk. Errors, prompt-injection attacks, manipulated tool descriptions or unsafe action sequences may lead to unauthorised access, data leakage or unsupported conclusions [23, 24, 25, 26, 27, 28]. For this reason, tool-using agents require architectural controls such as authentication, authorisation, validation, logging, policy enforcement and least-privilege tool access.

A central security concern for LLM-integrated applications is prompt injection. Prompt injection occurs when untrusted input, retrieved content or tool output contains instructions that cause a model or agent to ignore, override or conflict with trusted workflow instructions. In tool-using systems, this can lead not only to misleading text, but also to unsafe tool selection, unintended information disclosure or unauthorised actions. OWASP lists prompt injection as LLM01:2025 in its Top 10 for Large Language Model Applications [29], and recent work formalises prompt-injection attacks and evaluates possible defences [30, 31, 32]. In this thesis, user claim descriptions, retrieved policy passages, document text and tool outputs are therefore treated as data rather than instructions. This follows the structured-query principle that untrusted user-controlled or retrieved content should not override workflow instructions or agent permissions [33].

1.1.2 Multi-Agent Systems and Agentic AI

A MAS consists of multiple interacting agents that cooperate, coordinate or communicate in order to solve tasks. Classical MAS are based on autonomy, communication, coordination, role specialisation and distributed problem solving [4]. In the context of GenAI, these ideas are increasingly applied to LLM-based agents with roles such as retrieval, analysis, validation, monitoring or communication [5, 6, 7, 34, 8, 35].

Compared to a single-agent system, a MAS can improve modularity and make responsibilities more explicit. Specialised agents can be assigned narrow tasks and connected through structured handoffs, which supports separation of concerns and makes validation easier. At the same time, MAS introduce additional complexity because agents exchange

information, share context and coordinate actions. This makes the design of the overall system as important as the design of individual agents [10, 21].

In this thesis, MAS are therefore understood as enterprise systems rather than only collections of agents. Their design must define roles, permitted communication paths, tool access, evidence handling, validation points, logging and escalation to human review.

1.1.3 Orchestration Frameworks

As agentic systems become more complex, orchestration becomes necessary. Orchestration refers to the coordination of tasks, roles, sequencing, communication and control flow between agents and tools. Relevant patterns include sequential workflows, planner-executor structures, reviewer or judge patterns and routing mechanisms. Recent LLM-based MAS demonstrate similar patterns through role-based collaboration, conversation control and structured workflows [5, 7, 34, 35].

Frameworks such as CrewAI, LangChain, LangGraph, AutoGen and n8n support the implementation of agentic orchestration patterns by providing abstractions for agents, tasks, tools, Guardrails, workflow state, execution flows, tracing, evaluation, Human-In-The-Loop (HITL) mechanisms and multi-agent applications [36, 37, 38, 39, 40]. However, an orchestration framework is not equivalent to a complete enterprise architecture. It may help coordinate agents, but it does not automatically solve identity management, authorization, evidence generation, compliance, deployment, monitoring or tamper-evident auditability. These concerns must be addressed at the architectural level.

1.1.4 Agentic Workflows in Enterprise Environments

Enterprise environments usually contain existing DBs, document repositories, business applications, APIs, access-control systems, monitoring tools and compliance processes. These components are often stable and governed by organisational policies. In contrast, LLM-based agents are dynamic components that interpret context, reason over tasks and generate outputs probabilistically. Integrating both types of components is a central challenge for enterprise agentic workflows.

This challenge becomes more important when Retrieval-Augmented Generation (RAG) and tool use are introduced. RAG can improve factual grounding by retrieving external information and providing it to an LLM as context [3]. However, the system must determine which sources are trusted, which documents may be retrieved, which permissions apply and how retrieved content is validated. Retrieved content may also contain malicious or misleading instructions, creating risks such as indirect prompt injection and retrieval-layer attacks [41, 42].

For enterprise use, agentic workflows therefore require more than functional task execution. They need clear trust boundaries, scoped access to tools and data, traceable evidence handling, policy enforcement, monitoring and human review for ambiguous or high-risk cases. Without such architectural guidance, agentic workflows may remain difficult to secure, operate and justify in regulated environments.

1.1.5 Communication in Agentic Systems

Communication is a central concern in agentic systems because agents may need to exchange tasks, intermediate outputs, retrieved evidence, tool results or final artifacts. In

MAS, communication is not only a coordination mechanism, but also a security and auditability concern. The architecture must define which information may be shared, which agent is allowed to act on it and how the exchange is recorded [43].

Two interoperability concepts are especially relevant for this thesis: Model Context Protocol (MCP) and Agent-to-Agent (A2A). MCP is an open standard for connecting Artificial Intelligence (AI) applications and agents to external systems such as data sources, tools and workflows through a standardised client-server protocol [44, 45]. For enterprise agentic workflows, MCP can reduce custom integration effort, but it also introduces a trust boundary where authentication, authorization, tool mediation, logging and policy enforcement are required [46, 47].

Google's A2A is a complementary interoperability standard for communication between agents. While MCP focuses on connecting agents to tools, A2A focuses on agent-to-agent collaboration, capability discovery, task delegation and artifact exchange [48, 49]. This distinction is important for enterprise MAS: MCP defines a boundary between agents and tools, whereas A2A defines a boundary between agents. Both communication paths require governance because they may carry instructions, evidence, intermediate results or sensitive workflow state.

Recent security work shows that inter-agent protocols can introduce risks such as malicious capability claims, task manipulation, unsafe artifact exchange, collusion and topology-dependent attack paths [50, 51, 52, 53]. Therefore, this thesis treats both MCP and A2A as architectural trust boundaries rather than simple connector mechanisms.

1.1.6 Guardrails

Guardrails are mechanisms that keep AI systems within acceptable technical, ethical, legal and organisational boundaries [54, 55]. They can be implemented as technical and organisational controls, including content moderation, schema validation, adversarial detection, privacy restrictions, source controls, human-review gates, monitoring and incident-response mechanisms [14, 56]. In LLM-based agent systems, such controls are especially important because unsafe behaviour may arise not only from a single prompt, but from the interaction of user input, retrieved context, agent planning, tool calls, memory, inter-agent handoffs and final responses [9, 57, 27].

For this thesis, these mechanisms are treated as architectural controls rather than only prompt-level instructions. They are placed at several points in the workflow. Input and context checks screen user input and retrieved content before they are used by an agent. Retrieval controls constrain sources, require source metadata and support evidence grounding. Tool and execution controls restrict which tools may be called, validate parameters and help enforce safe action sequences [25, 28, 57]. Output checks validate generated responses for expected structure, policy consistency, evidence support, missing information and possible disclosure of sensitive data before the output is passed downstream or shown to a user [55, 56].

Output validation is particularly relevant for the insurance PoC because final responses may be customer-facing or may feed another workflow step. These checks assess not only whether a response is fluent, but also whether it is grounded in retrieved evidence, follows the expected format and avoids unsupported decisions. Traceability and logging support this validation by preserving intermediate handoffs and making it possible to inspect where an error entered the pipeline [58]. Long-context safety results further indicate that secure workflows should not rely only on model refusals, because refusal behaviour and

task performance can shift with context length, type and placement [59]. Therefore, in the proposed RA, guardrail mechanisms complement identity, authorization, tool mediation, audit logging and HITL escalation rather than replacing them.

1.1.7 Reference Architectures

Software architectures are commonly understood as central quality-bearing structures of software-intensive systems. A RA builds on this idea by providing reusable architectural guidance for a class of systems rather than a complete design for one specific implementation. It can be understood as an abstraction of software elements, their main responsibilities and interactions, capturing essential architectural knowledge from existing systems in a domain and serving as a guide for designing new systems or system variants in that domain [60, 61].

This reuse-oriented view is important for this thesis because the proposed RA is not derived only from abstract modelling. It is informed by literature, expert feedback, implementation experience and the insurance use case, which is consistent with empirically grounded architecture approaches [17, 18]. Empirical research on software RAs shows that such architectures can support reuse, reduced development effort, improved maintainability and standardisation, while also introducing risks such as learning effort, dependency on the architecture and complexity if the architecture is not managed appropriately [16].

For this thesis, a RA is suitable because tool-using MAS are not only individual agent workflows, but enterprise systems that must integrate agents, orchestration, tools, data sources, identity, authorization, governance, monitoring, audit evidence and human oversight. The proposed RA is therefore intended as reusable guidance. It shows architectural layers, trust boundaries and control points that can be adapted to different enterprise domains, while the insurance claim-processing PoC provides one concrete instantiation.

1.1.8 Insurance claim processing as Application Domain

Insurance claim processing is used as the application domain in this thesis because it combines structured data, contractual documents, policy versions, claim descriptions and evidence-based reasoning. A claim may be clearly covered, clearly excluded, dependent on a specific policy version or ambiguous enough to require additional information. This makes the domain useful for testing whether an agentic workflow can retrieve relevant policy information, reason over a case, identify missing information and produce a traceable outcome.

The domain also highlights the need for explainability and evidence grounding. A claim decision should not only state whether a case is covered or not covered, but also show which policy passages were considered. This domain provides a realistic setting in which architectural concepts for secure, traceable and evidence-based MAS can be instantiated and evaluated.

1.1.9 Security, Auditability and Compliance

Security, auditability and compliance are central requirements for enterprise MAS. Security protects systems, data, tools and workflows against unauthorised access, misuse, manipulation and leakage [9]. Auditability refers to the ability to reconstruct and verify

what happened in a system. Compliance refers to alignment with legal, regulatory, organisational and policy requirements. Traceability connects these concerns by making system actions, decisions and evidence understandable across the workflow [13, 14, 15].

MAS introduce new attack surfaces due to complex workflows, orchestration, inter-agent communication, shared context and tool usage. Relevant risks include prompt injection, indirect prompt injection, malicious or colluding agents, hallucinations, unsafe tool calls, confused-deputy behaviour and sequence-level tool abuse [10, 62, 63, 27, 28]. These risks require architecture-level defence mechanisms rather than only prompt-level mitigation.

Auditability requires more than ordinary application logging. For a workflow to be auditable, it should be possible to reconstruct relevant events, including inputs, retrieved evidence, tool calls, intermediate decisions, final outputs, timestamps, actor identities and authorization context. Audit-trail work further emphasizes that accountable AI systems should preserve lifecycle and workflow evidence, including data use, model or agent configuration, approvals, monitoring signals, incidents and compliance-relevant decisions [22].

Tamper-evident mechanisms, such as Merkle trees and blockchain-style hash chaining, can support auditability by making changes to recorded evidence detectable [64, 65, 66, 67]. Cryptographic hash functions are central to this mechanism because they map audit records to fixed-length digests. Their security relies on properties such as collision resistance. In this thesis, hashes are therefore used for integrity verification and tamper evidence, not for confidentiality [68].

1.2 Problem Formulation

The increasing use of GenAI in enterprise environments creates new architectural challenges. Advanced systems increasingly combine LLM-based agents with tools, retrieval mechanisms, DBs, document repositories and workflow automation [1, 2, 5]. Once agents can access data sources or invoke tools, their behaviour is no longer limited to generating text. They may retrieve sensitive information, influence business processes and produce outputs that affect human decision-making.

The problem investigated in this thesis is how tool-using MAS can be designed so that they are secure, traceable, auditable and practically usable in enterprise contexts. This includes the challenge of combining static enterprise components, such as DBs, documents, APIs and existing business systems, with dynamic LLM-based agents that reason over tasks and interact with tools.

This problem is especially significant in regulated or high-risk domains, such as insurance. Enterprises require more than functional GenAI prototypes: they need systems that are explainable, governable and accountable. A claim-processing system, for example, should not only generate a decision recommendation, but also show which customer data, policy version, document passages and reasoning steps contributed to the result.

Existing orchestration frameworks support the implementation of agent roles, tasks and workflows [5, 7, 34]. However, they do not by themselves provide a complete model for identity, authorization, policy enforcement, evidence generation, tamper-evident logging, compliance or human oversight. Existing work also often discusses orchestration, tool use, retrieval or security controls separately, but provides limited guidance on how

these concerns can be combined into a reusable and implementable enterprise architecture [19, 20, 23, 24].

The central problem can therefore be formulated as follows: enterprises lack a reusable RA for GenAI MAS that integrates security, traceability, auditability and compliance by design. Existing agent frameworks can support orchestration, and existing audit mechanisms can support evidence integrity, but there is limited guidance on how these concerns should be combined into a coherent enterprise architecture for tool-using agents. This gap is reinforced by agent-security work showing that deployment paradigms, tool interfaces and chained attacks can change the security posture of LLM-based systems [69], and by audit-trail work emphasizing that accountable AI systems require structured evidence rather than unstructured logs [22].

This thesis addresses this problem by developing a secure-by-design RA for tool-using MAS. The architecture is instantiated through a PoC for household insurance claim processing and complemented by a separate auditing artifact based on Merkle tree logging and blockchain-style root chaining. The goal is not to build a production-ready claims system, but to investigate how architectural principles, workflow design, evidence handling and auditability mechanisms can be combined to support secure and compliant agentic systems.

1.3 Motivation

The motivation for this thesis is scientific, practical and societal. From a research perspective, GenAI systems are moving from isolated chatbot applications toward tool-using agents and MAS. This shift creates new architectural patterns, but also raises unresolved questions about control, security, auditability and governance. While existing research and frameworks increasingly describe agent collaboration, orchestration, tool use and retrieval, there is still a need for reusable architectural guidance that combines these capabilities with secure-by-design principles and enterprise-scale orchestration models [5, 6, 7, 19, 21].

From an industrial perspective, enterprises need practical guidance for adopting agentic AI beyond small prototypes. Scaling GenAI systems into real workflows is limited not only by technical feasibility, but also by governance, auditability, observability, reliability and compliance concerns. AI Practitioners need architectures that explain how agents can safely interact with enterprise data sources, documents, APIs and human decision points.

This is especially relevant for domains such as insurance, finance and healthcare, where automated or semi-automated decisions can affect customers and require accountability. In an insurance claim process, for example, a system should not only produce a decision recommendation, but also show which customer data, policy documents, policy versions and evidence passages were used.

The societal motivation is connected to responsible use of AI. As GenAI systems become more integrated into organisational processes, users and affected individuals need confidence that such systems are controlled, transparent and accountable. Secure and auditable architectures can help ensure that agentic systems support human decision-making rather than replacing responsibility with opaque automation [13, 14, 15].

1.4 Objectives

The overall objective of this thesis is to design and instantiate a secure-by-design RA for tool-using MAS in enterprise environments. The work combines conceptual architecture design, technical artifact development and expert-informed evaluation. The objectives in Table 1.1 are formulated to be specific and assessable.

Table 1.1: Objectives of the thesis

O1	Analyse existing research on MAS, LLM-based agents, tool use, security, auditability and interoperability in order to establish the conceptual foundation for the thesis.
O2	Derive requirements for a secure-by-design MAS RA based on literature, implementation experience and expert input.
O3	Design and propose a reusable RA for secure and compliant tool-using MAS in enterprise environments.
O4	Implement and instantiate the proposed RA in a PoC for an insurance claim process using structured customer data, policy data, document-based evidence and agent-based workflow roles.
O5	Implement a separate auditing artifact based on cryptographic logging, Merkle tree evidence structures and blockchain-style root chaining to explore tamper-evident traceability for agent workflows.
O6	Conduct and evaluate an expert questionnaire to identify relevant architectural priorities, security requirements, auditability needs and practical concerns for secure GenAI and MAS development.

The expected outcome is a structured and justified RA that identifies the main layers, components, trust boundaries and evidence mechanisms required for secure tool-using MAS. The PoC and auditing artifact instantiate selected parts of this architecture and support the evaluation of feasibility, traceability and auditability.

1.5 Research Questions

Based on the problem formulation and objectives, this thesis is guided by one main Research Question (RQ) and four supporting RQs. The main RQ is formulated as follows:

> **RQ:** How can a secure and compliant RA for tool-using MAS in enterprise environments be designed?

To answer this main RQ, the supporting RQs in Table 1.2 are investigated.

1.6 Contributions of the Work

This thesis contributes to the design and evaluation of secure MAS in enterprise environments. Its main contribution is the combination of a secure-by-design RA, an insurance claim-processing PoC, a separate auditability artifact and expert-informed requirements.

The main contributions are:

Table 1.2: Supporting RQs of the thesis

RQ1	What architectural requirements are necessary for a secure-by-design MAS in enterprise environments?
RQ2	How can static enterprise systems, such as DBs, documents, APIs and existing business systems, be integrated with dynamic LLM-based agents in a traceable and controlled way?
RQ3	How can tamper-evident evidence logging based on Merkle trees and blockchain-style root chaining support auditability in agentic workflows?
RQ4	How do experts assess the relevance of architectural, security, governance and auditability requirements for secure GenAI and MAS?

- **Conceptual contribution:** A secure-by-design RA for tool-using MAS, combining orchestration, tool use, retrieval, security controls, governance, auditability and human oversight.
- **Technical contribution:** Two implemented artifacts: an insurance claim-processing PoC and a separate auditability artifact using Merkle trees and blockchain-style root chaining.
- **Empirical contribution:** An expert questionnaire that identifies AI and architecture practitioner perspectives on security, auditability, governance, architectural layers, trust boundaries and adoption concerns.
- **Domain-specific contribution:** An application of the proposed architecture to household insurance claim processing as a regulated, document-intensive and evidence based workflow.

1.7 Scope and Limitations

This thesis focuses on the design of a secure-by-design RA for MAS in enterprise environments. The architecture is instantiated through a household insurance claim-processing PoC and complemented by a separate auditability artifact. The scope is intentionally limited in order to focus on the architectural problem rather than on building a production-ready insurance system.

The application domain is limited to household insurance claims. This domain was selected because it requires the combination of structured customer data, policy documents, claim descriptions and evidence-based reasoning. Other insurance domains, such as health insurance or commercial insurance, are not evaluated.

The implemented PoC uses a limited number of claim cases and policy documents under controlled conditions. The test cases cover relevant situations, such as covered, excluded, policy-version-dependent and ambiguous claims, but they do not represent the full complexity of real insurance operations. Synthetic customer data is used to avoid privacy risks and support reproducibility.

The thesis does not implement all security and compliance mechanisms required for production deployment. Concepts such as identity management, authorization, policy enforcement, human oversight and audit evidence are included in the RA, but only selected parts are implemented in the artifacts. The PoC also does not include a complete

enterprise knowledge base, full vector DB, advanced RAG platform or enterprise-scale document management system.

Finally, the auditability artifact is separate from the claim-processing PoC and demonstrates tamper-evident evidence logging in a simplified setting. The expert questionnaire provides expert-informed design priorities, but it is not statistically representative. The behaviour of LLM-based systems can also vary depending on model version, prompts, context and execution conditions. These limitations are discussed in more detail in later sections.

1.8 Target Audience

The thesis is intended for researchers, architects, engineers and practitioners interested in secure tool-using MAS in enterprise environments. For researchers, it provides a design-oriented contribution on agent orchestration, auditability and human oversight in LLM-based systems. For enterprise and software architects, it provides a reusable architectural perspective on integrating dynamic agents with static enterprise systems such as DBs, documents, APIs and workflow components.

For security engineers, security architects and governance teams, the thesis highlights identity, authorization, least privilege, tool access control, trust boundaries, audit evidence and tamper-evident logging as architectural concerns. It is especially relevant for regulated domains where automated or semi-automated decisions must remain explainable, accountable and verifiable.

1.9 Thesis Structure

The remainder of this thesis is structured as follows.

Section 2 – Related Work reviews research on classical MAS, LLM-based agents, orchestration frameworks, tool use, security, auditability and tamper-evident logging. It also compares relevant approaches and identifies the research gaps addressed by this work.

Section 3 – Methodology describes the research approach, the role of the PoC and auditing artifact as research artifacts, the expert questionnaire and the evaluation strategy.

Section 4 – Reference Architecture and Artifact Design presents the proposed RA, the insurance claim-processing PoC, the separate auditing artifact and the main design decisions. It describes the architecture and artifacts in greater technical depth.

Section 5 – Results and Analysis presents and analyses the results of the RA, the PoC, the auditing artifact and the expert questionnaire. It examines how the findings address the RQs.

Section 6 – Discussion interprets the findings in relation to the objectives, RQs, related work and practical implications. It also discusses limitations, generalisability and threats to validity.

Section 7 – Conclusion and Future Work summarises the main findings and contributions, answers the central research problem and outlines future work.

2 Related Work

This section reviews research relevant to the design of secure, traceable and auditable tool-using MAS. While Section 1 introduced the background and motivation of the thesis, this section positions the work within existing academic and technical research. The aim is to identify which concepts, architectures, governance approaches and security and auditability mechanisms already exist, and where gaps remain for secure enterprise MAS.

The review is organised thematically. It first discusses foundational work on classical MAS, LLM-based agents, tool use and RAG. It then reviews research on LLM-based multi-agent orchestration, agent specification and RAs. The section subsequently examines security, communication and governance work, including prompt injection, tool misuse, MCP, A2A, Guardrails and compliance-oriented risk management. Finally, it discusses auditability, tamper-evident logging, Merkle trees and blockchain-style evidence mechanisms.

This structure reflects the interdisciplinary nature of the thesis. The proposed RA is not based on a single research stream. It combines ideas from software architecture, MAS, GenAI, security engineering, enterprise governance and cryptographic auditability. This section therefore identifies the gap addressed by the thesis: the lack of reusable secure-by-design architectural guidance for tool-using enterprise MAS that combines orchestration, controlled tool access, evidence grounding, traceability, auditability and human oversight.

2.1 Review of Existing Research

Existing research relevant to this thesis can be understood as a set of overlapping research streams rather than as strictly separate domains. Classical MAS research provides the conceptual foundation for autonomy, coordination, communication and role specialisation. LLM-based agent research extends these ideas with natural-language reasoning, tool use and retrieval-based grounding. Work on LLM-based MAS, orchestration and agent specification is closest to the workflow-oriented part of this thesis, while RA and enterprise architecture research provide the basis for reusable architectural guidance. Security, communication and integration research identifies risks and controls for tool-using agents, MCP-based tool access and A2A interaction. Governance and Guardrail research contributes requirements for risk management, monitoring, accountability and human oversight. Finally, auditability research provides mechanisms for traceability, tamper-evident logging and verifiable evidence.

The following subsections review these streams and explain how they contribute to the design of the proposed RA. The categories are analytical and partly overlapping, because several sources address more than one concern. This overlap is intentional: secure enterprise MAS require the integration of orchestration, security, governance, communication control and auditability rather than isolated solutions for each concern.

2.1.1 Classical Multi-Agent Systems

Classical MAS research studies systems composed of multiple interacting agents that can perceive, decide, act and coordinate in pursuit of individual or shared goals. Concepts such as autonomy, social ability, reactivity, proactivity, coordination and communication are central to this tradition. Wooldridge and Jennings describe intelligent agents as systems situated in an environment and capable of autonomous action, while MAS extend this idea to settings where multiple agents interact and coordinate with each other [4].

This foundation is relevant because the thesis relies on role separation, coordination and distributed responsibility. The claim-processing PoC, for example, separates the workflow into roles such as Case Builder, Claims Examiner, Claims Judge, Customer Communicator and HITL Handler. This reflects a classical MAS idea: complex tasks can be decomposed into specialised responsibilities and coordinated through interaction.

However, classical MAS research does not directly address the risks introduced by LLM-based agents, such as probabilistic outputs, prompts, retrieved context, tool calls, prompt injection, auditability and enterprise compliance. It therefore provides the conceptual basis, but not a complete solution for secure enterprise GenAI workflows.

2.1.2 LLM-based Agents, Tool-use and Retrieval

LLM-based agents extend ordinary text-generation systems by combining language-model reasoning with actions, tools and contextual information. ReAct is an important early example because it interleaves reasoning traces and actions, allowing a model to reason about a task, invoke external sources or environments and continue based on the obtained information [1]. Toolformer further shows that language models can learn when and how to call external tools through APIs, such as calculators, search engines, translation systems and calendars, and compensate for limitations of the LLM itself [2]. These works are relevant because they show how LLMs can become part of executable workflows rather than only text generators.

RAG is another important foundation. Lewis et al. propose RAG as a method that combines parametric model knowledge with non-parametric external memory retrieved from a document index [3]. This is relevant for enterprise workflows because agents often need access to current, domain-specific or organisation-specific information that is not reliably stored in model parameters. In the claim-processing PoC, policy documents provide the external source material that grounds the workflow.

ACE extends the discussion from tool use to secure system architecture. It argues that LLM-integrated application systems need clearer separation between planning and execution, because malicious app descriptions or app outputs can affect workflow integrity and availability [70]. This concern is closely related to prompt injection, where untrusted input, retrieved content or tool output attempts to override trusted system instructions. OWASP lists Prompt Injection as LLM01:2025 in its Top 10 for Large Language Model Applications, and Liu et al. formalize and benchmark prompt-injection attacks and defences [29, 30]. For regulated enterprise workflows, it is therefore not enough to retrieve context or call tools. The system must also define trusted sources, access permissions, provenance, evidence preservation and auditability.

2.1.3 LLM-based Multi-Agent Systems and Orchestration Frameworks

Recent research applies LLM-based agents in multi-agent settings where several agents cooperate to solve a task. Surveys show that LLM-based MAS are used for complex task solving, simulation and evaluation of generative agents, while introducing challenges related to coordination, communication, planning, memory, safety and security [8, 35, 50, 10]. CAMEL demonstrates role-based cooperation through communicative agents, while AutoGen, MetaGPT and ChatDev show how agent roles, conversation control and structured workflows can support multi-step collaboration [6, 5, 7, 34].

Recent survey work also emphasizes that LLM-based MAS can be analysed through workflow, infrastructure and communication dimensions, including agent roles, interaction structures, communication protocols and exchanged content [35, 43]. This supports the thesis view that inter-agent communication is not only a coordination mechanism, but also an architectural trust boundary.

Adimulam et al. provide a closely related orchestration-oriented perspective by arguing that orchestrated MAS require specialised agents, coordination logic and standardised communication protocols [21]. Agents execute narrowly scoped responsibilities, while the orchestration layer determines task order, manages dependencies and aligns agent outputs into a coherent workflow. This is closely related to the proposed RA, which also treats role separation, orchestration and controlled communication as central architectural concerns.

Traceability work on role-specialised multi-agent pipelines further shows that sequential agent workflows can become difficult to debug when errors propagate silently between roles [58]. This supports the thesis decision to define explicit agent responsibilities, controlled handoffs, validation points and audit records.

AutoGen provides a framework for building applications with multiple conversable agents. Its agents can combine LLMs, human input and tools, and developers can define different conversation patterns for solving tasks [5]. This is closely related to the implementation logic of the thesis, where a workflow is decomposed into specialised agents and tasks. AutoGen is especially useful as a reference for multi-agent orchestration and human-tool-agent interaction.

MetaGPT takes a different approach by encoding standardised operating procedures into multi-agent collaboration. It assigns roles and structures outputs so that agents can perform parts of a software-engineering workflow more consistently [7]. ChatDev similarly uses specialised communicative agents for software development and structures collaboration across design, coding and testing phases [34]. These systems show that LLM-based MAS can benefit from workflow structure, role specialisation and controlled handoffs.

The Agent Design Pattern Catalogue identifies recurring patterns for foundation-model-based agents, including RAG, role-based cooperation, Guardrails, tool registries, agent registries and evaluation components [71].

AgentNet represents a different direction by proposing decentralized evolutionary coordination, where agents coordinate through dynamically evolving graph structures [72]. These approaches show alternative forms of agent collaboration, but this thesis deliberately focuses on enterprise-governed orchestration, where bounded routing, traceability, auditability and human review are more important than open-ended self-evolving coordination.

Overall, LLM-based multi-agent frameworks demonstrate the value of role-based collaboration and workflow decomposition. However, they mainly focus on task performance, collaboration or orchestration. They do not fully address enterprise requirements such as least-privilege authorization, compliance evidence, tamper-evident logging, HITL escalation and auditability. This thesis builds on the orchestration idea, but shifts the focus toward secure-by-design enterprise architecture.

2.1.4 Reference Architectures for LLM-based Multi-Agent Systems

RAs and architecture-oriented work are especially relevant because this thesis contributes a RA. RAs support architectural knowledge reuse and provide an abstract basis from which concrete software architectures can be derived [61, 16]. Empirically grounded and grounded-architecture research further emphasizes that architectural guidance should be derived from recurring systems, stakeholder concerns and architecture-significant requirements rather than from abstract design alone [17, 18].

SALLMA is one of the closest academic works because it proposes a software architecture for LLM-based MAS and frames the architecture of such systems as an underexplored research area [19]. It distinguishes between an Operational Layer for request-intent management, real-time task execution and dynamic agent orchestration, and a Knowledge Layer for managing workflow and agent configurations. This supports the thesis assumption that LLM-based MASs require architectural thinking rather than only prompt-level design. However, SALLMA focuses on general software architecture. Security, compliance, audit evidence and tamper-evident traceability are not its central research object.

Another closely related work is the orchestration framework for MAS by Adimulam et al. [21]. It proposes an enterprise-oriented architecture with specialised agents, orchestration, planning and policy management, state and knowledge management, quality operations, observability, MCP-based tool access and A2A-based agent communication. This is highly relevant to the thesis, but its focus is broader orchestration and enterprise adoption, while this thesis emphasizes secure-by-design RA design, insurance claim processing, evidence grounding and tamper-evident auditability.

The blockchain-monitored agentic AI architecture by Jan et al. is also closely related because it combines a LangChain-based multi-agent architecture with permissioned blockchain monitoring, policy enforcement and immutable auditability [20]. However, Jan et al. focus on blockchain-monitored perception–reasoning–action pipelines for autonomous agentic systems, while this thesis focuses on a reusable enterprise RA, expert-informed requirements, an insurance PoC and a separate Merkle tree auditing artifact.

Ge et al. provide a governance architecture for autonomous agent systems, including control, accountability and oversight concerns [73]. SAGA also contributes to governed inter-agent communication by combining agent registration, user-defined contact policies and cryptographic access-control tokens [74]. These works are relevant to the governance and communication aspects of the proposed RA, but they do not cover the full combination of enterprise architecture layers, document-grounded workflows, controlled tool access, HITL routing and tamper-evident auditability targeted in this thesis.

This thesis therefore differs from existing architecture-oriented work by making security, governance and auditability primary design drivers. The proposed RA is not only concerned with how agents coordinate, but also with how their actions are authorized, constrained, logged and verified.

2.1.5 Security Risks in tool-using and RAG-enabled Agents

Recent survey work on agentic AI security emphasizes that agents introduce risks beyond ordinary LLM applications because they combine autonomy, planning, memory, tool use and action execution [9]. The security problem in this thesis is therefore not limited to harmful text generation. Tool-using MAS can interact with enterprise systems, invoke actions, exchange information between agents and influence workflow outcomes.

Recent landscape studies show that agentic AI security risks emerge from the interaction of model behaviour, tools, memory, interfaces and workflow execution [11]. They also show that prompt-injection defences involve trade-offs between trustworthiness, utility and latency, and that no single defence is sufficient across all contexts [12]. Broader work on LLM risks and Guardrails similarly supports the need for layered controls because hallucination, privacy leakage, unsafe outputs and misuse cannot be addressed only through model prompting [75, 76].

Tool-using agents introduce specific risks because natural-language reasoning is connected to executable actions. AgentDojo evaluates prompt injection attacks and defences for agents that execute tools over untrusted data [23]. ToolHijacker shows that the tool-selection stage itself can be attacked through malicious tool documents, which reinforces the need to treat tool registries, tool metadata and tool descriptions as security-relevant inputs [26]. IPIGuard and Les Dissonances further show that indirect prompt injection and cross-tool information-flow risks can arise when agents combine multiple tools and data sources [77, 78].

Several works show that tool-use risk is not limited to isolated tool calls. AgentGuard focuses on unsafe tool orchestration and generates safety constraints for unsafe workflows [57]. It repurposes the agentic orchestrator as a safety evaluator to identify unsafe workflows, validate them through execution, generate safety constraints and validate whether those constraints are effective. This is relevant for the thesis because enterprise MAS may fail not only through a single unsafe model response, but also through a sequence of individually plausible tool calls that become unsafe in combination.

STAC shows that individually harmless tool calls can become harmful when chained across multiple turns, while TAMAS evaluates adversarial risks in LLM-based MAS, including prompt-level attacks, indirect prompt injection, collusion and contradiction [27, 63]. Related work on safe tool use adds a preventive perspective by deriving workflow-level safety constraints and enforcing information-flow or tool-sequence restrictions through capability-aware MCP declarations [79]. Together, these findings support sequence-aware logging, bounded workflows, tool mediation and policy enforcement.

This aligns with Guardrail engineering work, which treats Guardrails as explicit risk-reduction components that must be specified, integrated and validated, and with AGrail's agent-specific focus on adaptive Guardrails for tool-using agents [55, 56].

Prompt injection remains a central concern for LLM-based agents. Beurer-Kellner et al. propose design patterns for securing LLM-based agents against prompt injection and discuss trade-offs between security and utility, while formal security work argues that agent security depends on execution context, including authorized task, action alignment, source authorization and data isolation [24, 25]. These works are relevant because they treat agent security as a system-design problem rather than a prompt-engineering problem.

Architecture-level security work further shows that deployment choices such as Function Calling and MCP change the attack surface, while authenticated workflow approaches treat prompts, tools, data and context as control surfaces requiring identity and policy checks [69, 80].

StruQ provides a relevant defence principle by treating prompt injection as a consequence of mixing instructions and data in the same input. It proposes structured queries that separate trusted prompt instructions from user-controlled data and trains models to follow instructions only from the prompt part [33]. This is directly relevant for tool-using

MAS, where retrieved documents, user claims and tool outputs may contain instruction-like text that should not override workflow instructions. Model-level defences such as SecAlign and Meta SecAlign complement this principle, but they do not remove the need for architectural controls around tools, retrieval and inter-agent communication [31, 32].

Security risks also arise at the A2A level. A2ASecBench shows that A2A systems can be vulnerable to supply-chain manipulation and protocol-logic weaknesses, including malicious capability claims, task flooding, agent-side request forgery and unsafe artifact exchange [51]. GroupGuard and G-Safeguard add group-level and topology-guided perspectives on multi-agent threats, such as collusion and attack propagation [52, 53]. Connected GenAI systems can also create propagation risks, as shown by the Morris-II work on self-replicating adversarial prompts in RAG-enabled connected applications [81].

Overall, the security literature shows that tool-using and retrieval-enabled agents require controls at multiple points: input handling, retrieval, context construction, tool invocation, A2A and MCP communication, output handling and logging. These findings support the proposed RA, especially its governance layer, tool gateway, Guardrails, trust boundaries, least-privilege access and audit logging.

2.1.6 Governance, Compliance and AI Risk Management

Security risks are connected to broader governance and compliance concerns. Scientific literature on LLM-based agents identifies risks such as hallucination, weak controllability, prompt injection, unsafe tool use and unreliable outputs. In enterprise environments, these risks must be managed through organisational and architectural controls, not only through model-level improvements.

Risk Management Frameworks (RMFs) and standards provide normative anchors for this thesis. The National Institute of Standards and Technology (NIST) AI RMF provides a general framework for managing risks in systems that design, develop, deploy or use AI [13]. The NIST Generative AI Profile adapts this risk-management perspective to GenAI and highlights risks that are unique to GenAI [14]. International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC) 42001 define requirements for an AI management system and are relevant for organisational governance of AI systems [15].

Guardrail research frames such controls as technical, ethical and regulatory mechanisms that keep AI systems aligned with safety, fairness, privacy, transparency and accountability expectations [54, 55]. In agentic workflows, these controls should be connected to orchestration, tool access, routing and auditability rather than treated only as prompt-level filters [56, 57]. Together with broader AI risk-management and governance frameworks, this supports the thesis argument that secure MAS require policies, identity, authorization, documentation, monitoring, audit evidence and HITL escalation [13, 14, 15, 22]. The proposed RA translates these concerns into architectural elements.

Observability is also part of governance for agentic systems. Design principles for LLM observability show that traces, monitoring signals and runtime inspection are needed to understand and control LLM-based systems during operation [82].

2.1.7 Auditability and tamper-evident Evidence

Auditability is central to this thesis because enterprise MAS should be able to explain and reconstruct workflow behaviour after execution. Secure audit-log research addresses

the problem of preserving evidence even when systems may be compromised [65]. For agentic workflows, auditability requires more than ordinary application logging: relevant events include inputs, retrieved evidence, tool calls, intermediate decisions, final outputs, timestamps, actor identities, authorization context and Guardrail outcomes.

Traceability work further supports this requirement by emphasizing the need to connect generated outputs with the intermediate steps, sources and system actions that produced them [58]. Tamper-evident mechanisms, such as Merkle trees and hash chaining, can support this goal by making changes to recorded evidence detectable [64, 65, 66, 67].

Merkle trees provide a cryptographic mechanism for compactly committing to many records. Merkle's work on hash-based signatures introduced the idea of using tree structures where a root hash can represent many underlying values [64]. In the context of this thesis, the same principle is used to aggregate workflow record hashes into a Merkle root. If one record changes, the recomputed root also changes.

Recent work on Merkle trees in blockchain systems further highlights that collision probability and hash-function assumptions are part of the security argument for Merkle-based commitments, not only implementation details [68]. Certificate Transparency is a practical example of Merkle-tree-based transparent logging. It uses append-only logs so that certificates can be publicly audited and suspicious certificate issuance can be detected [66]. Although Certificate Transparency is not about MAS, it demonstrates the broader value of verifiable log structures for accountability.

Sutton and Samavi apply blockchain-based tamper-proof logging to privacy auditing for AI systems [67]. Their work is relevant because it connects audit logs, non-repudiation and blockchain-style integrity proofs in an AI-related setting. However, it is not specific to tool-using LLM-based agents or MAS workflows. Recent AI audit-trail work extends this perspective by emphasizing structured lifecycle evidence, while blockchain-enabled audit-trail work motivates compact cryptographic commitments instead of storing large evidence artifacts directly on-chain [22, 83].

The auditing artifact in this thesis builds on these ideas. It does not introduce a new cryptographic primitive. Instead, it applies record hashing, Merkle tree commitments and blockchain-style root chaining to structured workflow events from a MAS. This positions auditability as part of the architecture rather than as ordinary application logging.

2.2 Comparison of Approaches

To compare the related work, the reviewed literature is organised into eight overlapping thematic categories: classical MAS foundations; LLM-based agents, tool use and RAG; LLM-based MAS, orchestration and agent specification; RAs and enterprise architecture practice; security risks and defences for tool-using agents; communication and integration security for MCP and A2A; governance, Guardrails and risk management; and auditability, traceability and tamper-evident evidence. These categories provide important building blocks for this thesis, but none fully addresses the complete problem of designing a reusable secure-by-design RA for tool-using enterprise MAS with auditability and compliance as primary concerns.

Classical MAS research provides the conceptual foundation for autonomy, communication, coordination and role specialisation, but does not address LLM-specific risks such as prompt injection, probabilistic outputs, RAG, tool invocation or enterprise compliance. LLM-based agent frameworks such as AutoGen, CAMEL, MetaGPT and ChatDev move

closer to the thesis topic by demonstrating role-based collaboration, tool use and structured workflows. However, their main focus remains task execution and orchestration rather than secure enterprise architecture.

Architecture-oriented work, especially SALLMA and the orchestration architecture by Adimulam et al., is closest to this thesis because it shows that LLM-based MAS require architectural structuring. However, these works do not fully cover the specific combination targeted here: a reusable secure-by-design enterprise RA, expert-informed requirements, document-grounded insurance claim processing, task-level Guardrails and a separate Merkle tree auditing artifact.

Security, governance and auditability research provide the strongest justification for the controls in the proposed RA. Prompt-injection, tool-use, RAG-attack and A2A security research motivate trust boundaries, least privilege, context separation, tool mediation, Guardrails and audit logging. Governance frameworks provide risk-management and accountability principles, while auditability research contributes mechanisms such as secure audit logs, Merkle trees, hash chaining and blockchain-style evidence commitments. However, these streams usually address risks, controls or integrity mechanisms separately rather than integrating them into one reusable enterprise MAS architecture.

Table 2.3 summarises these categories and their relation to this thesis. It shows that existing work provides important foundations for workflow and role design, threat-informed controls, governance principles and tamper-evident auditability, but that these concerns are often treated separately. This thesis is positioned between these areas by integrating MAS and LLM-based agent research, security literature, governance frameworks and Merkle-tree-based auditability mechanisms into one secure-by-design RA, supported by an insurance claim-processing PoC, expert-informed requirements and a separate auditing artifact.

2.3 Distinction from Existing Agent and Automation Platforms

Commercial and low-code agent platforms such as Claude Cowork, Claude-based agent tooling and n8n can already automate tasks, invoke tools and connect to external systems. Claude Cowork supports delegated knowledge-work tasks with access to selected files, connectors and applications, while Claude's API and Agent SDK support tool use, MCP-based access and programmable agents [84, 85, 86, 87]. n8n similarly supports AI agents, multi-agent workflows, application integrations, Guardrails, monitoring and MCP connectivity [88, 89, 90, 91].

The contribution of this thesis is not that the implemented claim-processing workflow can only be built with the selected tools. Rather, the thesis addresses how tool-using MAS should be structured so that security, traceability, auditability, governance and human oversight are considered by design. Existing platforms may serve as implementation technologies, but they do not by themselves define a reusable enterprise RA, identify required trust boundaries, specify audit evidence records or demonstrate tamper-evident traceability for regulated workflows.

The distinction is therefore one of abstraction and research objective. Platforms such as Claude Cowork and n8n support the implementation of agentic workflows, while this thesis focuses on architecture-level guidance for structuring, controlling, tracing and auditing such workflows in regulated enterprise environments. The insurance PoC and the separate auditing artifact are therefore not intended to compete with these platforms, but to instantiate and examine selected parts of the proposed RA.

Table 2.3: Comparison of related research categories and their relevance to this thesis

Research category	**Main contribution**	**Limitations for this thesis**	**Relevance to this thesis**
Classical MAS foundations [4]	Define autonomy, coordination, communication, role specialisation and distributed problem solving.	Do not address LLM-specific risks, probabilistic outputs, RAG, tool misuse or enterprise compliance.	Provide the conceptual foundation for role-based agents, coordination and distributed responsibility in the proposed RA.
LLM-based agents, tool use and RAG [1, 2, 3, 70]	Show how LLMs can combine reasoning with external tools, APIs, documents and retrieval-based grounding in external knowledge sources for model outputs.	Focus mainly on model-tool or model-retrieval interaction, not on enterprise authorization, governance, auditability or tamper-evident traceability.	Motivate controlled tool access, evidence grounding, prompt-data separation and the knowledge layer of the claim-processing PoC.
LLM-based MAS, orchestration and agent specification [5, 6, 7, 34, 71, 21, 92]	Enable role-based collaboration, structured workflows, agent roles, task decomposition, handoffs, prompt specification and evaluation mechanisms.	Primarily focus on orchestration, collaboration or specification; security, compliance and audit evidence are usually not central.	Support the thesis design of specialised agents, explicit task definitions, bounded workflows, Guardrails and controlled handoffs.
RAs and enterprise architecture practice [61, 16, 60, 19, 20, 93]	Provide reusable architectural knowledge, software-architecture guidance, architecture-oriented views of LLM-based MAS and trusted agentic pipelines.	Existing work does not fully combine secure-by-design enterprise RA, insurance claim processing, document grounding, HITL routing and tamper-evident auditing.	Position the thesis as an architecture contribution and justify a reusable, layer-based RA rather than a single implementation-specific design.
Security risks and defences for tool-using agents [23, 24, 33, 9, 57, 11, 12, 27, 63, 28, 79]	Analyse prompt injection, indirect prompt injection, unsafe tool execution, tool-dependency attacks, sequence-level risks, privilege escalation and information-flow constraints.	Often focus on attacks, benchmarks or specific defences rather than a complete enterprise MAS RA.	Justify trust boundaries, least privilege, scoped tool access, structured context handling, sequence-aware logging, tool mediation and validation points.

Continued on next page

Table 2.3: Comparison of related research categories and their relevance to this thesis (continued)

Research category	**Main contribution**	**Limitations for this thesis**	**Relevance to this thesis**
Communication and integration security for MCP and A2A [44, 45, 48, 49, 46, 47, 51, 52, 53, 74]	Describe standardised agent-to-tool and A2A communication, and analyse protocol-level, gateway-level and group-level security risks.	Focus on specific integration or communication concerns rather than a full secure enterprise RA.	Support treating MCP and A2A as explicit trust boundaries that require authentication, authorization, capability validation, logging and governance.
Governance, Guardrails and risk management [13, 14, 15, 54, 55, 56, 82]	Provide guidance for risk management, responsible AI, Guardrails, monitoring, observability and organisational accountability.	Generally broad and not specific to tool-using enterprise MAS or document-grounded claim-processing workflows.	Support the governance, operations, Guardrail, monitoring and HITL elements of the proposed RA.
Auditability, traceability and tamper-evident evidence [64, 65, 66, 67, 22, 83, 68, 58]	Provide mechanisms and concepts for secure audit logs, traceability, Merkle tree commitments, hash chaining, blockchain-style audit trails and evidence verification.	Usually not specific to LLM-based MAS or enterprise agent workflows.	Provide the technical foundation for the auditing artifact and for treating audit evidence as an architectural output.

2.4 Identified Gaps and Positioning of this Work

The reviewed literature shows that important foundations already exist for tool-using and LLM-based MAS. Classical MAS research provides concepts such as autonomy, coordination, communication and role specialisation. LLM-based agent frameworks demonstrate role-based collaboration, tool use and structured workflows. Security research identifies risks such as prompt injection, tool misuse, RAG poisoning and cross-agent propagation, while governance and auditability research provide principles and mechanisms for accountability and tamper-evident evidence.

However, these research streams are often treated separately. Existing agent frameworks mainly focus on collaboration, task decomposition and orchestration, but they do not provide a complete enterprise architecture for security, compliance, auditability and controlled tool access. Architecture-oriented work such as SALLMA and blockchain-monitored agentic AI architectures moves closer to this thesis, but does not fully address the specific combination targeted here: a reusable secure-by-design enterprise RA, expert-informed architectural requirements, a document-grounded insurance claim-processing PoC, task-level Guardrails and a separate Merkle tree auditing artifact.

A central gap therefore concerns the translation of security and auditability research into architectural design. Prompt-injection, tool-use and RAG-attack literature shows why controls are needed, but often focuses on attacks, benchmarks or isolated defences rather than on where controls should be placed across architectural layers. Similarly, Merkle trees, secure audit logs and blockchain-based logging provide foundations for tamper-evident evidence, but they do not define how agent events, tool invocations, Guardrail outcomes and workflow decisions should be structured and embedded into a secure MAS architecture.

A further gap concerns enterprise integration. Many LLM-agent systems are evaluated in software-engineering tasks, benchmark environments or generic collaboration scenarios. Fewer works address how dynamic LLM-based agents can be integrated with static enterprise systems such as DBs, policy documents, Identity Providers (IdPs), external tools and human review processes. This is especially important in regulated and document-intensive domains where decisions must be traceable to authoritative evidence.

This thesis is positioned at the intersection of these gaps. It does not propose a new LLM model, cryptographic primitive or general-purpose orchestration framework. Instead, it contributes a secure-by-design RA that integrates ideas from MAS, LLM-agent orchestration, security engineering, governance and tamper-evident auditability. The architecture is instantiated through an insurance claim-processing PoC, complemented by a separate auditing artifact and grounded through an expert questionnaire.

The positioning of this work can therefore be summarised as follows: existing research provides important components, but there is still a lack of reusable architectural guidance that combines secure tool use, document grounding, enterprise integration, audit evidence, Guardrails and HITL into one coherent MAS design. This thesis addresses that gap by treating security, traceability and auditability as primary architectural concerns rather than as later add-ons.

2.5 Summary

This section reviewed research relevant to secure, traceable and auditable tool-using MAS. Classical MAS research provides the conceptual foundation for autonomy, coordination,

communication and role specialisation. LLM-based agent research extends these ideas through tool use, retrieval, reasoning-action loops and multi-agent orchestration. Frameworks and systems such as ReAct, Toolformer, AutoGen, CAMEL, MetaGPT and ChatDev show that LLMs can be used in structured agentic workflows, but they mainly focus on task execution, collaboration and orchestration rather than enterprise security, compliance and auditability.

Architecture-oriented work such as SALLMA, the orchestration architecture by Adimulam et al. and blockchain-monitored agentic AI architectures is closest to the contribution of this thesis. These works show that LLM-based MAS require architectural structuring, orchestration, monitoring and governance. However, they do not fully address the combination targeted here: a reusable secure-by-design enterprise RA, expert-informed requirements, document-grounded insurance claim processing, task-level Guardrails, controlled tool access, HITL routing and tamper-evident auditability.

The reviewed security literature shows that tool-using and retrieval-augmented agents introduce risks such as prompt injection, indirect prompt injection, unsafe tool execution, RAG poisoning, cross-tool information flow, sequence-level tool abuse and cross-agent propagation. These risks justify architectural controls such as trust boundaries, least-privilege access, authorization, tool mediation, prompt-data separation, Guardrails, sequence-aware logging and controlled A2A communication. Governance and RMFs further support the need for documentation, monitoring, accountability and human oversight, but they do not provide a concrete MAS-specific enterprise architecture.

The auditability literature provides the technical foundation for the separate auditing artifact. Secure audit logs, Merkle trees, hash chaining, Certificate Transparency and blockchain-style audit trails show how recorded evidence can be protected against unnoticed manipulation. However, these mechanisms are rarely connected directly to LLM-based agent workflows. This thesis therefore connects tamper-evident evidence mechanisms with structured workflow events from a tool-using MAS.

Overall, the related work shows that existing research provides important building blocks, but these building blocks are often treated separately. This thesis addresses the identified gap by proposing a secure-by-design RA that integrates agent orchestration, controlled tool use, document grounding, security controls, governance, audit evidence and human oversight into one architectural model. The following section describes the methodology used to design, instantiate and evaluate this architecture.

3 Methodology

This section describes the methodological approach used to investigate the research problem formulated in Section 1. This thesis addresses a design-oriented problem: how a secure and compliant RA for tool-using MAS can be designed, instantiated and evaluated in an enterprise context. Therefore, the methodology combines architecture-centric Design Science Research (DSR), artifact development, case-based evaluation and expert-informed analysis.

The approach is suitable because the thesis does not only analyse an existing system, but designs and evaluates artifacts that address a practical and conceptual problem. In line with DSR, knowledge is generated through the construction and evaluation of artifacts [94, 95]. The main artifact is the proposed RA. It is supported by two implementation artifacts: an insurance claim-processing PoC and a separate auditability artifact.

The research process follows a structured build–evaluate logic. It begins with the formulation of the research problem: the design of secure and compliant tool-using MAS. Relevant literature, technical sources, expert input and insurance-domain material are then collected and reviewed to identify concepts, risks, patterns, gaps, requirements and design constraints. These inputs are synthesised into architecture-significant requirements related to security, auditability, governance and feasibility.

Based on these requirements, the proposed RA is designed by defining layers, components, trust boundaries and control points. The architecture is then instantiated through artifact development, consisting of the insurance claim-processing PoC and the separate auditability artifact. Finally, the artifacts and expert-questionnaire results are evaluated through case-based, audit-based and expert-informed analysis in relation to the RQs. The process also includes a feedback loop from evaluation to architecture design, reflecting the refinement of design decisions during the thesis.

Figure 3.1 summarises the methodology followed in this thesis. It shows how literature and technical sources, expert input, domain material, the PoC and the auditability artifact contribute to the design, evaluation and resulting research contribution of the proposed RA. The following subsections describe the research approach, data collection, data analysis, reliability and validity considerations, as well as ethical considerations.

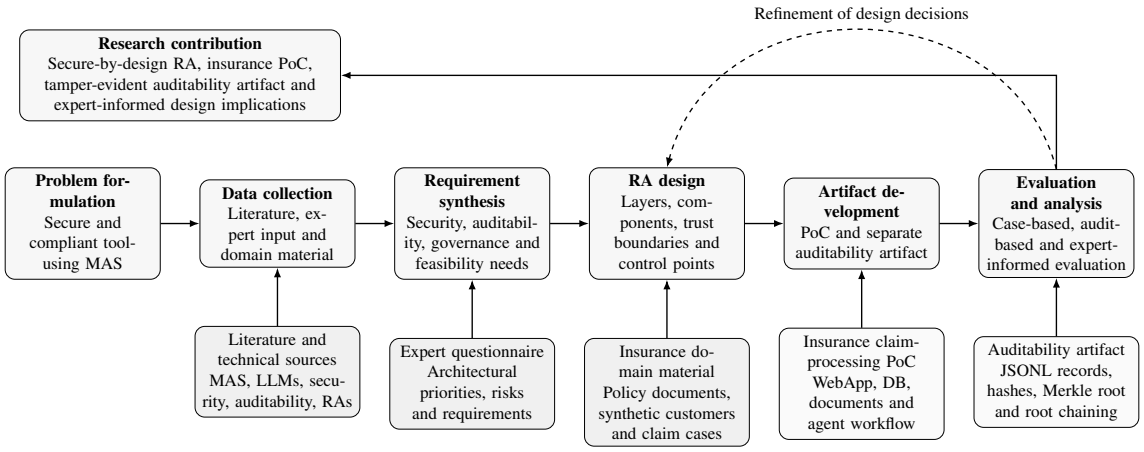


Figure 3.1: Methodology: architecture-centric DSR process followed in this thesis

3.1 Research Approach

This thesis follows a design-oriented research approach based on DSR [94, 95]. DSR is appropriate when the objective is to create and evaluate an artifact that addresses a relevant problem. In this thesis, the central artifact is a secure-by-design RA for tool-using MAS in enterprise environments.

The research problem is both practical and conceptual. The practical problem concerns how enterprises can adopt tool-using MAS in a secure, traceable and auditable way. The conceptual problem concerns how such systems should be structured at the architectural level. A purely theoretical approach would not be sufficient, because it would not show whether the proposed architecture can be instantiated. A purely experimental approach would also be insufficient, because the main contribution is not an algorithmic performance improvement, but an architectural model supported by implementation artifacts and expert-informed evaluation.

The RA therefore represents the main design artifact. The insurance claim-processing PoC and the separate auditing artifact instantiate selected architectural concerns and provide practical insight into feasibility, workflow design, evidence handling and traceability. The expert questionnaire complements the artifact evaluation by grounding architectural requirements in practitioner perspectives.

3.1.1 Architecture-centric DSR

The thesis applies DSR in an architecture-centric way. The main artifact is not a single software component, but a RA that describes how secure MAS can be structured. The architecture focuses on layers, components, responsibilities, trust boundaries, control points, evidence mechanisms and governance concerns.

This perspective is necessary because security and auditability in tool-using MAS cannot be solved only through prompt engineering or by improving individual agents. They require architectural decisions about identity, authorization, retrieval, tool access, logging, evidence generation, human oversight and governance. The architecture-centric approach therefore allows these concerns to be treated as design elements of the overall system.

3.1.2 Proof-of-Concept and Auditability as Research Artifacts

In addition to the RA, this thesis includes two implementation artifacts. The first artifact is an insurance claim-processing PoC. It instantiates selected parts of the architecture in a realistic but controlled enterprise-oriented workflow. The domain was chosen because claim handling combines structured customer data, policy metadata, policy documents, claim descriptions, exclusions, policy versions and evidence-based reasoning.

The second artifact is a separate auditability artifact. It demonstrates how workflow events can be represented as structured records, hashed, used as Merkle tree leaves, committed into a Merkle root and linked over time through blockchain-style root chaining. The artifact is separated from the PoC to keep the implementation feasible and to evaluate tamper-evident traceability independently from the business workflow.

Together, the two artifacts support the evaluation of selected parts of the RA. The PoC examines practical workflow instantiation, while the auditing artifact examines evidence integrity and tamper detection.

3.1.3 Mixed-method Evaluation

The evaluation follows a mixed-methods approach. It combines technical artifact evaluation with expert-informed qualitative and semi-quantitative analysis. The PoC is evaluated through predefined claim cases, including covered claims, excluded claims, policy-version-dependent claims and ambiguous claims. The evaluation examines whether the workflow can retrieve relevant customer and policy information, use document evidence, generate traceable outputs and route cases according to the available information.

The auditing artifact is evaluated by verifying whether manipulated workflow records can be detected through record-hash mismatches and changed Merkle roots. The goal is not to evaluate cryptographic performance, but to examine how tamper-evident evidence can be represented and verified in a simplified agentic workflow.

The expert questionnaire provides practitioner input on quality attributes, architectural layers, components, trust boundaries, audit evidence requirements and trade-offs. Since the final RA was not presented to the experts as a complete artifact, the questionnaire results are interpreted as expert-informed design priorities rather than as direct validation of the final architecture.

Together, these evaluation activities support the assessment of the thesis artifacts. The literature review grounds the design in existing research, the PoC demonstrates feasibility in a realistic use case, the auditing artifact explores tamper-evident traceability and the questionnaire provides expert-informed requirements and priorities.

3.2 Data Collection

This subsection describes the material collected for this thesis. Since the thesis follows a DSR approach, the collected material includes academic and technical literature, expert questionnaire responses, insurance policy documents, synthetic customer data, predefined claim cases and implementation-related data from the PoC and auditing artifact. Together, these sources support the derivation of requirements, the design of the RA and the evaluation of the artifacts.

The data collection process was designed to support traceability and reproducibility. The literature review provides the theoretical and technical foundation. The expert questionnaire provides practitioner input on architectural priorities and practical concerns. The insurance documents, synthetic data and claim cases provide the domain-specific basis for the claim-processing PoC. The implementation artifacts provide technical evidence for evaluating feasibility and auditability.

3.2.1 Literature Selection and Review Process

The literature review was used to establish the conceptual, technical and architectural foundation of this thesis. Relevant sources were collected from academic DBs, conference proceedings, arXiv archive, journals, technical standards and selected technical documentation. The review focused on MAS, LLM-based agents, tool use, orchestration frameworks, agentic security, auditability, evidence logging, Merkle trees and enterprise architecture.

The search was guided by keywords and keyword combinations such as *multi-agent systems*, *LLM agents*, *agentic AI*, *tool-using agents*, *agent orchestration*, *secure MAS*,

prompt injection, *auditability*, *tamper-evident logging*, *Merkle trees*, *reference architecture* and *enterprise GenAI*. The goal was not to conduct a systematic literature review, but to identify and synthesize relevant concepts, patterns, risks and architectural requirements.

The selection prioritized peer-reviewed research when available. However, because tool-using LLM agents and enterprise GenAI architectures are rapidly evolving, selected newer and technical sources, framework documentation, standards and industry reports were also considered. These sources were used to complement the academic literature and to capture recent developments in practice.

The collected literature was grouped thematically into classical MAS, LLM-based MAS orchestration, tool use and workflow control, security and trustworthiness, auditability and logging, Merkle tree mechanisms and enterprise architecture. This grouping informed the related work section and supported the derivation of architectural requirements in Section 4.

3.2.2 Expert Questionnaire Design

An expert questionnaire was designed to collect practical input on secure tool-using MAS. Its purpose was to identify relevant architectural priorities, quality attributes, security concerns, auditability needs, adoption blockers and design trade-offs. The questionnaire was not intended to statistically validate the final RA. Instead, it was used as expert-informed requirements elicitation. The full questionnaire is provided in Appendix A.

The questionnaire covered several thematic areas: enterprise use cases for GenAI and MAS, adoption blockers, quality attributes, Architecturally Significant Requirements (ASRs), threats, trust boundaries, architectural layers, logical components, evidence requirements, traceability needs and the expected reusability of a RA. The questions were designed to connect directly to the RQs and to the design of the proposed architecture.

Both closed and open question formats were used. Closed questions allowed participants to select or rank relevant options, such as important quality attributes, threats or architectural components. Open questions allowed participants to add explanations, missing aspects and practical experiences. This combination made it possible to collect both structured and qualitative input.

The questionnaire results were recorded digitally and later analysed through descriptive summaries and thematic interpretation. Counts and percentages were used to identify dominant patterns, while open-text answers were used to extract recurring themes, practical concerns and design implications.

3.2.3 Selection of Experts

Participants were selected from the company NTT DATA Deutschland SE based on their relevance to the thesis topic. The target group included practitioners with experience in software architecture, enterprise architecture, cloud architecture, cybersecurity, GenAI, LLM-based systems, data engineering, software development and technical leadership.

A total of 25 experts participated in the questionnaire. The group included participants with different levels of experience in software and enterprise architecture, GenAI and MAS. This made the sample suitable for collecting expert-informed requirements and practical design priorities, especially from an enterprise architecture and security perspective.

The selection was not intended to create a statistically representative sample of all MAS practitioners. Instead, the goal was to collect relevant input from practitioners familiar with enterprise system design, AI-enabled solutions, security concerns and operational constraints. This limitation is considered in the validity discussion.

3.2.4 Insurance Documents and policy versions

Insurance policy documents were collected to provide the domain knowledge source for the claim-processing PoC. The documents consist of real household insurance terms and conditions from different insurers, products and policy years. They were selected because they contain contractual rules, exclusions, coverage conditions and version differences relevant to claim decisions. The prepared policy documents, their source links and their use in the predefined evaluation cases are listed in Table C.1 in Appendix C. The documents are publicly accessible and were made available online through insurer websites or public policy-document sources.

Using different policy versions supports the evaluation of traceability and evidence grounding. It allows the PoC to test whether the workflow can link claim reasoning to the applicable contractual source. The documents were prepared into a controlled text-based knowledge source for the PoC. The preparation process is described in Section 4.

3.2.5 Creation of synthetic customer data

Synthetic customer data were created to support the insurance claim-processing PoC. Synthetic data was used to avoid privacy risks and to make the evaluation reproducible. No real customer records or real claim submissions were used.

The synthetic data links customers, policies and policy documents so that a submitted `customer_id` can be resolved to an active policy and the corresponding document metadata. This supports traceability from claim input to customer record, policy version and evidence source. The detailed data model is described in the PoC design Subsection 4.3.

3.2.6 Design of Claim Cases

The predefined claim cases were designed to evaluate different aspects of the claim-processing PoC. They include covered claims, excluded claims, policy-version-dependent claims and ambiguous or incomplete claims. This variation allows the workflow to be evaluated not only for straightforward decisions, but also for cases where missing information, exclusions or policy versions affect the outcome.

The cases are not intended to represent the full diversity of real insurance operations. Instead, they provide a controlled evaluation set for assessing whether the proposed architecture and PoC workflow can support traceable claim handling under bounded conditions. The complete claim cases are listed in Appendix B.

3.3 Data Analysis

The data analysis combines thematic synthesis, descriptive summaries, requirements mapping and case-based evaluation. Since the thesis follows a DSR approach, the goal is to derive architectural insights, assess artifact feasibility and interpret expert-informed requirements.

The analysis is structured around five activities. First, the literature is synthesized into architectural themes and design requirements. Second, the questionnaire responses are analysed through descriptive summaries and qualitative thematic interpretation. Third, the proposed RA is evaluated against the derived requirements and RQs. Fourth, the insurance claim-processing PoC is analysed through predefined claim cases. Fifth, the auditing artifact is evaluated by examining whether its logging, Merkle tree construction, root chaining and verification logic support tamper-evident traceability.

LLMs are used within the agentic workflow, but the analysis is architectural and functional rather than model-performance-oriented. The analysis focuses on architectural fit, traceability, evidence grounding, auditability and feasibility.

3.3.1 Literature Synthesis

The literature was analysed through thematic synthesis. Relevant sources were grouped according to the main topics of the thesis, including MAS, LLM-based agents, orchestration frameworks, tool use, security, auditability, Merkle trees and enterprise architecture. Within each theme, key concepts, patterns, limitations and recurring challenges were identified.

The purpose of this synthesis was to derive the conceptual foundation and requirements for the proposed RA. Literature on MAS and orchestration informed the role-based and workflow-oriented parts of the architecture. Literature on security and trustworthy AI informed requirements such as least privilege, controlled tool access, trust boundaries and human oversight. Literature on auditability and tamper-evident logging informed the evidence and auditability aspects of the architecture.

The synthesis was conducted as a focused conceptual review to identify architectural requirements and design principles relevant to secure tool-using MAS. The resulting themes are used to position the thesis and justify the proposed architecture.

3.3.2 Qualitative Analysis of Questionnaire Responses

The questionnaire responses were analysed through descriptive summaries and qualitative thematic interpretation. Closed questions were summarised with counts and percentages to identify dominant patterns, priorities and areas of agreement among participants. These results were used to describe the participant profile, enterprise use cases, adoption blockers, quality attributes, architectural layers, components, trust boundaries and auditability requirements. Open-text answers were reviewed and grouped into recurring themes. Similar answers were clustered to identify repeated concerns and practical recommendations.

The questionnaire results were not interpreted as statistically representative because of the limited sample size and the varying level of MAS-specific expertise among participants. Instead, they were treated as expert-informed design input. Their main role was to support the derivation of architectural priorities and to compare the proposed RA with practical concerns from industry.

3.3.3 Evaluation of the Reference Architecture

The proposed RA was evaluated through requirements mapping and architectural reasoning. Requirements derived from literature, domain analysis, implementation experience and expert input were compared with the proposed architectural layers and components. The goal was to assess whether the architecture covers the main concerns for secure MAS.

The evaluation considered whether the RA addresses security, auditability, governance, controlled tool use, traceability, evidence generation and human oversight. It also examined whether the architecture separates static enterprise components from dynamic agent components and defines control points for interactions between users, agents, tools, documents and external systems.

The RA was not evaluated as a finished production architecture. Instead, it was evaluated as a reusable architectural blueprint. The analysis therefore focuses on completeness, coherence, feasibility and alignment with the RQs. The insurance claim-processing PoC and the auditing artifact provide practical evidence for selected parts of the architecture, while the questionnaire results provide expert-informed support for its design priorities.

3.3.4 Case-based Analysis of PoC Outputs

The insurance claim-processing PoC was analysed through predefined claim cases. Each case was executed through the workflow and assessed according to whether the correct customer and policy data were retrieved, whether the relevant policy document was selected, whether the output was grounded in suitable evidence, whether the decision path was traceable and whether missing information or uncertainty was handled appropriately.

The cases included covered, excluded, policy-version-dependent, ambiguous and incomplete scenarios. Covered cases were used to assess whether the workflow could identify applicable coverage. Excluded cases tested whether exclusions were recognized and explained. Policy-version-dependent cases examined whether the workflow could use the applicable policy version. Ambiguous or incomplete cases tested whether the workflow could identify missing information and support escalation or HITL fallback.

The case-based analysis was qualitative and functional. It did not aim to produce statistically generalisable accuracy values. Instead, it examined whether the PoC could demonstrate structured workflow execution, evidence-based decision support, traceability and controlled routing under realistic but bounded conditions.

3.3.5 Evaluation of the Auditing Artifact

The auditing artifact was evaluated by analysing whether it can represent and verify an evidence chain for a simplified agentic workflow. The analysis focused on structured event logging, conversion of events into Merkle tree leaves, construction of Merkle roots, blockchain-style root chaining and verification of chain integrity.

The main evaluation criterion was whether manipulation of logged evidence could be detected through hash recomputation or chain verification. The artifact was therefore analysed in terms of integrity, traceability and verifiability. The evaluation also considered whether the artifact can conceptually support the auditability layer of the proposed RA.

The auditing artifact is intentionally simplified. It does not represent a complete production-grade audit infrastructure and it is not fully integrated into the claim-processing PoC. Its purpose is to demonstrate how tamper-evident evidence structures can be applied to agent workflow logs and how such mechanisms could strengthen auditability in future enterprise MAS.

3.4 Reliability and Validity

This subsection discusses how the credibility of the research design was supported. Since the thesis follows a DSR approach, reliability and validity are considered in relation to

the proposed RA, the insurance claim-processing PoC, the auditing artifact and the expert questionnaire.

Reliability concerns whether a similar research process could produce comparable results under similar conditions. In this thesis, reliability is supported through a structured research process, documented design decisions, predefined claim cases, explicit workflow roles, prepared document sources and reproducible audit files. The auditing artifact is particularly reproducible because it uses deterministic hash calculations, Merkle tree construction and blockchain-style root chaining.

Complete reproducibility is limited by the use of LLM-based components. Model outputs may vary depending on model version, prompt wording, context length, execution environment and framework behaviour. To reduce this limitation, the PoC uses controlled inputs, documented prompts, defined task structures and predefined evaluation cases. The results should therefore be interpreted as evidence from a controlled implementation snapshot rather than as permanent benchmark results.

Validity concerns whether the conclusions are supported by the selected methods, artifacts and data sources. Construct validity is supported by linking broad concepts such as security, traceability and auditability to concrete architectural mechanisms. Security is examined through identity, authorization, least privilege, controlled tool access and enforcement points. Auditability is examined through structured logs, record hashes, Merkle tree roots and hash chaining. Practical feasibility is examined through the implemented PoC and auditing artifact.

Internal validity is supported by predefined scenarios and explicit analysis criteria. The claim cases cover covered claims, excluded claims, policy-version-dependent claims and ambiguous or incomplete claims. The expert questionnaire adds practitioner perspectives on architectural priorities, adoption blockers, trust boundaries and auditability needs.

External validity is limited because the PoC focuses on household insurance claims, synthetic customer data and a specific implementation setup. The expert questionnaire is also not statistically representative. Therefore, the findings are not interpreted as proof of production readiness or general effectiveness across all enterprise domains. Instead, they provide design-oriented evidence for the feasibility and relevance of the proposed architectural concepts.

Overall, reliability and validity are strengthened by triangulating multiple sources of evidence: literature, implementation experience, predefined case evaluation, deterministic audit verification and expert input. The remaining limitations are discussed in detail in Section 6.

3.5 Ethical Considerations

This thesis involves ethical considerations related to expert participation, synthetic customer data, publicly accessible insurance policy documents and the use of LLM-based decision support.

The expert questionnaire involved human participants. Participants were informed about the purpose of the questionnaire and the intended use of their responses. Participation was voluntary, and results are reported only in aggregated form. The questionnaire did not require confidential organisational information, and individual participants are not identified in the thesis.

The insurance claim-processing PoC uses synthetic customer data instead of real customer records. This was important because insurance claims may involve sensitive personal, financial and household information. The fictional data was designed to be realistic enough for the PoC, but it does not represent actual individuals.

Real insurance policy documents are used as domain knowledge sources. These documents are publicly accessible and do not contain customer-specific information. They are used only for research and technical evaluation. The PoC is therefore positioned as an architectural demonstrator and decision-support workflow, not as a production-ready claims system, legally binding insurance tool or replacement for professional claims assessment.

The use of LLM-based agents in decision-support workflows also raises ethical concerns because generated outputs may be incomplete, inconsistent or overly confident. For this reason, the thesis emphasizes evidence grounding, traceability, auditability and HITL fallback. The proposed RA treats human oversight and governance as essential mechanisms for responsible use of agentic systems, especially in regulated domains.

3.6 Summary

This section described the methodology used to design and evaluate the proposed RA. This thesis follows an architecture-centric DSR approach in which the RA is the main artifact, supported by an insurance claim-processing PoC, a separate auditing artifact and an expert questionnaire.

Data collection included literature, expert responses, insurance policy documents, synthetic customer data, predefined claim cases and implementation-related data from the artifacts. The analysis combined literature synthesis, requirements mapping, case-based PoC evaluation, audit verification and qualitative interpretation of expert input.

The methodology supports the development and assessment of a secure, traceable and auditable MAS architecture, while acknowledging limitations related to the bounded PoC scope, synthetic data, expert sample size and variability of LLM-based workflows.

4 Reference Architecture and Artifact Design

This chapter presents the proposed RA and the design of the two supporting artifacts in more technical depth. It moves from the methodological foundation in Chapter 3 to the main design contribution of the thesis: a secure-by-design architecture for tool-using MAS and artifact designs that instantiate selected architectural concerns.

The proposed RA addresses the central problem of this thesis: enterprises lack reusable architectural guidance for secure, traceable and compliant tool-using MAS. The architecture connects dynamic LLM-based agents with static enterprise components such as DBs, documents, APIs and workflow systems. Security, governance, auditability and human oversight are therefore treated as architectural design concerns rather than optional add-ons.

Two artifacts are used to examine selected parts of the architecture. The first artifact is an insurance claim-processing PoC for household insurance claims. It demonstrates how customer data, policy metadata, document-based evidence and role-based agents can be combined in a controlled enterprise workflow. The second artifact is a separate auditing artifact. It shows how structured workflow events can be represented as evidence records and protected through record hashing, Merkle tree commitments and blockchain-style root chaining.

The section first introduces the design goals and requirements. It then presents the proposed RA, the insurance claim-processing PoC, the auditing artifact and the main design decisions and trade-offs.

4.1 Design Goals and Requirements

The proposed RA is guided by requirements derived from the literature review, the insurance claim-processing use case, implementation experience and expert questionnaire results. Since the architecture is intended as reusable guidance, it should not be limited to the insurance PoC, but should support enterprise settings in which LLM-based agents interact with tools, documents, structured data and human decision points [61, 16, 60].

Table 4.4 summarises the main requirement categories and their architectural implications. These requirements define the design direction for the chapter. The RA shows how agents, tools, data sources, workflow logic, human review and audit mechanisms interact. The insurance claim-processing PoC instantiates selected parts of this architecture, while the auditing artifact focuses on tamper-evident traceability.

4.2 Reference Architecture for secure Multi-Agent Systems

This subsection presents the proposed RA in Figure 4.2. The architecture is designed as a layered model that separates dynamic agent behaviour from static enterprise components, security controls, governance mechanisms, integration interfaces and auditability structures. Its purpose is to provide reusable guidance for designing GenAI-based MAS that interact with tools and enterprise data sources while remaining traceable, governable and auditable.

The architecture is based on the assumption that enterprise MAS should not consist only of agents and tools. Agents must be embedded into a wider enterprise architecture

Table 4.4: Design requirements for the proposed RA

Requirement category	Main requirement	Architectural implication
Security	Users, agents, tools and services must be identifiable and access must follow least privilege. Tool invocation and data access should be governed by system-level authorization rather than by prompt instructions alone.	The RA requires explicit identities, scoped permissions and authorization checks [25, 70, 80, 96, 28]. Tool access should be mediated through controlled gateways or MCP-based boundaries with validation, logging and policy enforcement [46, 69]. Context isolation and sequence-aware validation reduce prompt-injection, tool-dependency, cross-tool risks and broader multi-agent attack paths [57, 77, 78, 10].
Auditability	The system should make it possible to reconstruct how an output was produced, which evidence was used, which agents or tools were involved and which controls were applied.	The RA requires structured evidence records, correlation identifiers, timestamps, tool invocation records, retrieved evidence, agent outputs, routing decisions, audit logs and observability signals [22, 82, 58].
Evidence integrity	Audit evidence should be protected against unnoticed manipulation. If a workflow record is changed after execution, the manipulation should be detectable.	The auditing artifact explores this through record hashes, Merkle tree commitments and blockchain-style root chaining. This uses compact cryptographic commitments and depends on collision-resistant hash functions [83, 68].
Compliance and governance	Agentic workflows should operate within organisational, legal and policy boundaries. High-risk or ambiguous cases should support human review.	The RA requires policies, governance mechanisms, approval gates, HITL escalation, accountability and documented architecture views [55, 56].
Prompt and context separation	Trusted workflow instructions should be separated from untrusted user input, retrieved documents and tool outputs. User-provided or retrieved text should not redefine tasks, override policies or change permissions.	The RA requires structured context assembly, guarded prompt templates, separation between instructions and data, and validation of retrieved or tool-provided content. This requirement is inspired by StruQ's structured-query principle [33, 31, 32].

that includes orchestration, identity, authorization, data access, policy control, observability, audit evidence and human oversight. This is especially important when agents interact with sensitive data, business documents, internal APIs or external systems.

The proposed RA is intentionally platform- and framework-independent. It is not tied to CrewAI, Claude Cowork, n8n, LangChain, AutoGen or any specific implementation environment. Such tools may instantiate parts of the architecture, but the contribution of this thesis lies in defining reusable layers, trust boundaries, control points and audit mechanisms for secure enterprise implementations.

The security literature reviewed in Section 2 shows that tool-using MAS expose attack surfaces across user input, retrieved documents, tool use, inter-agent communication and workflow state [9, 10]. Therefore, the proposed RA follows a defence-in-depth approach. It combines scoped identity and authorization, least-privilege tool access, mediated integration, controlled agent communication, prompt–data separation, evidence provenance, task-level Guardrails, HITL escalation, observability and tamper-evident audit logging. Identity and least-privilege controls are grounded in formal agent-security and

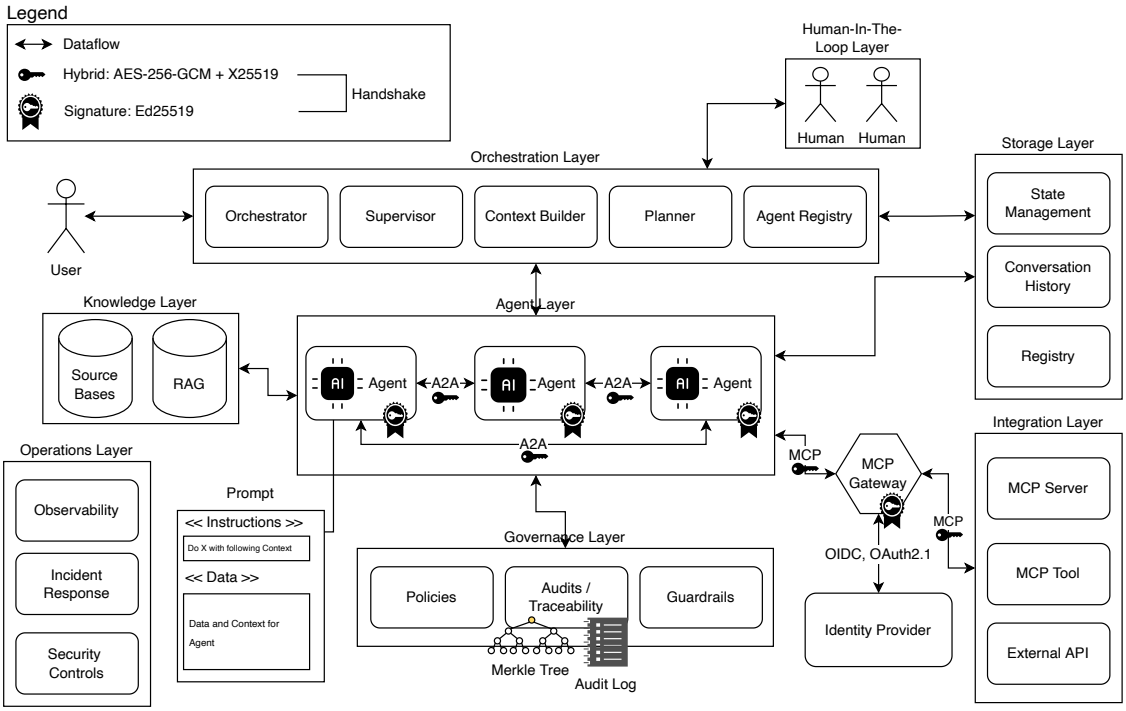


Figure 4.2: Proposed RA for secure tool-using MAS

privilege-control work [25, 70], mediated tool and agent communication are informed by MCP and A2A security studies [46, 51], and task-level validation addresses unsafe tool-orchestration risks [57]. The goal is not to eliminate all residual risk, but to distribute enforcement, validation and evidence mechanisms across the architecture so that failures remain constrained, observable and traceable.

Several design principles shape the RA. Tool use should be sequence-aware because harmful effects can emerge across multiple tool calls rather than from a single isolated action [27]. Agent actions should follow least privilege so that agents cannot access data, invoke tools or delegate tasks beyond the current workflow need [28]. Verifiably safe tool-use work further motivates capability-aware tool declarations, information-flow constraints and tool-sequence restrictions for MCP-based environments [79]. Traceability work on role-specialised pipelines supports explicit roles, handoffs and intermediate records for identifying where errors originate and propagate [58]. Long-context safety results further support the use of validation and Guardrails instead of relying only on model refusals [59].

The design is also informed by enterprise-oriented orchestration and architecture work for MAS, including Microsoft's Multi-agent RA [97] and orchestration-oriented work by Adimulam et al. [21] that emphasizes specialised agents, planning and policy management, execution control, state and knowledge management, quality operations, observability, tool access and agent communication. The proposed RA adapts these ideas toward the more specific focus of this thesis: security, traceability, auditability and regulated enterprise workflows.

The proposed RA is grounded in the literature review, the insurance claim-processing use case, implementation experience and expert questionnaire results. This is consistent with RA and grounded-architecture research, which emphasizes reusable architectural guidance based on evidence, stakeholder concerns and ASRs [61, 16, 17, 18].

Table 4.5 summarises the logical layers of the proposed RA. The layer structure separates flexible agent behaviour from deterministic enterprise control structures. This separation is important because LLM-based agents generate outputs dynamically, while enterprise environments require stable enforcement points, access control, traceability and operational governance [35].

The proposed architecture favours controlled workflow execution over unrestricted interleaved plan-and-execute loops. In interleaved agent architectures, untrusted tool outputs may influence later planning decisions. ACE addresses this risk by separating abstract planning, concrete plan instantiation and isolated execution [70]. The proposed RA follows the same general security principle at the enterprise-architecture level.

A central design distinction is the separation between static enterprise components and dynamic agent components. Static components include DBs, document repositories, enterprise systems, IdPs, monitoring tools, storage services and integration interfaces. Dynamic components are the LLM-based agents that interpret context, reason over tasks and generate intermediate or final outputs. The RA connects both sides through orchestration, controlled retrieval, mediated integration and governance mechanisms.

In the insurance claim-processing PoC, this distinction is visible in the interaction between the WebApp, Supabase, policy documents, OneDrive tools and the CrewAI workflow. Static components provide customer data, policy metadata and document evidence, while dynamic agents prepare the case, examine the claim, validate the result and prepare communication or follow-up actions. The orchestration logic defines the execution sequence, and task-level Guardrails validate whether an agent output is suitable for the next task.

Identity, authorization and evidence handling are treated as core architectural controls. Agents should not share one broad permission set. Instead, permissions should be scoped to agent role, task context and requested action. Similarly, the knowledge and retrieval layer is separated from the agent layer so that access control, filtering, evidence selection and logging occur before information is passed to an agent. This separation is important for auditability and explainability. When a document passage is used, the system should preserve the source document, document version, metadata and relevant passage location. In the insurance domain, this ensures that a claim recommendation can be traced back to the applicable policy version.

Trusted instructions are separated from untrusted data. They are defined by the workflow, task descriptions, Guardrails and policies. Untrusted data includes user-submitted claims, retrieved policy passages, tool outputs and intermediate workflow content. This follows the structured-query principle of StruQ and supports the design choice that retrieved or user-provided text should be treated as evidence or context, not as a source of new instructions [31, 32, 33].

Finally, audit evidence and human oversight are treated as explicit architectural outputs. Relevant audit evidence includes workflow identifiers, timestamps, actor identity, tool invocations, retrieved evidence, authorization decisions, intermediate outputs, Guardrail outcomes, routing decisions and final outputs. The separate auditing artifact extends this idea by showing how structured workflow records can be hashed, aggregated into a Merkle tree and linked through blockchain-style root chaining. The HITL layer complements this by ensuring that ambiguous, incomplete, high-risk or regulatory cases can be escalated to human review and included in the audit trail.

Table 4.5: Logical layers of the proposed RA

Layer	Main responsibility	Security and audit relevance
Orchestration Layer	Coordinates execution through orchestrator, supervisor, planner, context builder and agent registry. Controls task order, routing, context preparation and agent invocation.	Adds deterministic workflow structure around dynamic agent behaviour. Records which agent is called, with which context and at which step. Planning, execution and quality operations are orchestration responsibilities [5, 35, 21, 97].
Agent Layer	Contains specialised agents for retrieval, examination, validation or communication. Agents interact through controlled inter-agent communication (A2A) and mediated tool interfaces (MCP).	Supports modularity and role separation. Agents operate within assigned roles, permissions and workflow context. Inter-agent communication is a trust boundary for context, artifacts, instructions and workflow state [48, 49, 10, 74, 92, 58].
Knowledge Layer	Provides access to documents, metadata, knowledge bases and RAG-style retrieval.	Grounds outputs in organisational knowledge. Requires source traceability, access control, document-version awareness and evidence preservation [3, 41, 42, 33].
Integration Layer	Connects agents to tools, external APIs, MCP servers, MCP tools and enterprise systems through controlled interfaces and the MCP Gateway.	Prevents unrestricted tool execution through authentication, authorization, parameter validation, logging and policy enforcement. Tool integration is an explicit trust boundary requiring governance and auditability [44, 45, 97, 47, 21, 79, 69].
Identity Provider (IdP)	Provides identities for users, agents, services, tools and reviewers. Supports scoped permissions through OpenID Connect (OIDC), OAuth, Policy Decision Point (PDP) and Policy Enforcement Point (PEP).	Enables least privilege, accountable tool use and reconstruction of who initiated or approved an action. Scoped identity supports trusted communication across users, agents, services and tools [96, 74, 80].
Governance Layer	Defines policies, routing rules, approval requirements, compliance constraints and Guardrails.	Turns organisational and compliance rules into enforceable controls. Reduces reliance on prompts by validating outputs before downstream use. Failed validation leads to retries or traceable workflow termination [55, 56, 59, 13, 14, 15].
Logging and Traceability	Forms part of the Governance Layer. Records requests, agent calls, tool invocations, retrieved evidence, authorization decisions, routing outcomes, Guardrail results and final outputs.	Provides structured audit evidence for reconstruction. Traceability supports inspection of how outputs were produced [58]. Record hashes, Merkle roots and hash chaining strengthen evidence integrity [22, 83, 68, 64, 65].
Storage Layer	Stores workflow state, conversation history, registry information, metadata and persistent records needed during execution.	Requires governance to avoid uncontrolled retention, cross-agent context leakage and poisoning, and unauthorised access to sensitive state [35, 97, 82, 15].
Operations Layer	Supports monitoring, observability, debugging, incident response, operational metrics and security monitoring.	Helps detect abnormal behaviour, execution failures, Guardrail failures and policy violations. Observability and quality operations support reliable, auditable MAS execution [82, 21, 98, 22].
HITL Layer	Provides human review, escalation, approval and override for ambiguous, incomplete, low-confidence or high-risk outputs.	Preserves human accountability in regulated workflows. Human decisions should be returned to the workflow and included in the audit trail [13, 14, 15, 73, 10].

4.3 Claim-Processing PoC Design

The claim-processing PoC instantiates selected parts of the proposed RA in a concrete enterprise-oriented use case. Its purpose is to demonstrate feasibility and how a tool-using MAS can support a household insurance claim workflow by combining user input, structured customer data, policy metadata, document-based evidence, specialised agent roles, routing logic and controlled customer communication.

The PoC is a simplified but realistic demonstrator. It does not replace a production claims system and does not provide legally binding insurance decisions. Instead, it is used to examine whether the proposed RA can be instantiated in a workflow that requires evidence-based reasoning, traceability, routing decisions and escalation. The complete German claim cases are listed in Appendix B. Screenshots of the WebApp and CrewAI configuration interface are included in Appendix C. The full agent prompts, task descriptions, expected outputs and Guardrail definitions are documented in Appendix D.

The implementation consists of a Flask WebApp, a Supabase DB, prepared insurance policy documents stored in OneDrive and a CrewAI workflow. In CrewAI, agents and tasks are configured into a sequential workflow: agents define the role-based capabilities, while tasks define the concrete steps, expected outcomes, response formats and Guardrails. The WebApp retrieves the relevant customer, policy and document metadata from the DB, constructs a request payload and sends it to the CrewAI workflow through a REST API. The workflow then prepares the case, examines the claim, validates the result and routes the case to customer communication, customer information request, examiner revision or human review.

The PoC maps to the proposed RA as a selective instantiation. The WebApp represents the user-facing interface, Supabase and OneDrive represent static enterprise and knowledge components, and CrewAI provides the orchestration and agent runtime. Routing rules, confidence thresholds and task-level Guardrails instantiate selected governance mechanisms. Other RA elements, such as production-grade Identity and Access Management (IAM), a scalable RAG platform, operations management and live tamper-evident audit integration, remain simplified or outside the PoC scope. This is intentional: the RA is used as reusable guidance, not as a requirement that every implementation must include every layer in full.

Use case and Workflow. The selected use case is a household insurance claim process. A customer submits a claim description together with a `customer_id`. The system retrieves the customer record, identifies the active policy, locates the corresponding policy document and prepares a structured case file for the agent workflow. The workflow evaluates whether the reported damage is covered, excluded, dependent on missing information or unsuitable for automated continuation.

Figure 4.3 shows the simplified workflow used in the PoC. It abstracts from the revision limits and focuses on the main routing outcomes after judicial review.

The process starts in the WebApp. The user enters a `customer_id` and a claim description, either manually or by selecting a predefined test case. For self-written claims, the rich text input is converted into plain text while preserving relevant structure such as paragraphs and lists. Before submission, the user confirms the request. If no active policy is found for the customer, the application displays an error and the CrewAI workflow is not started.

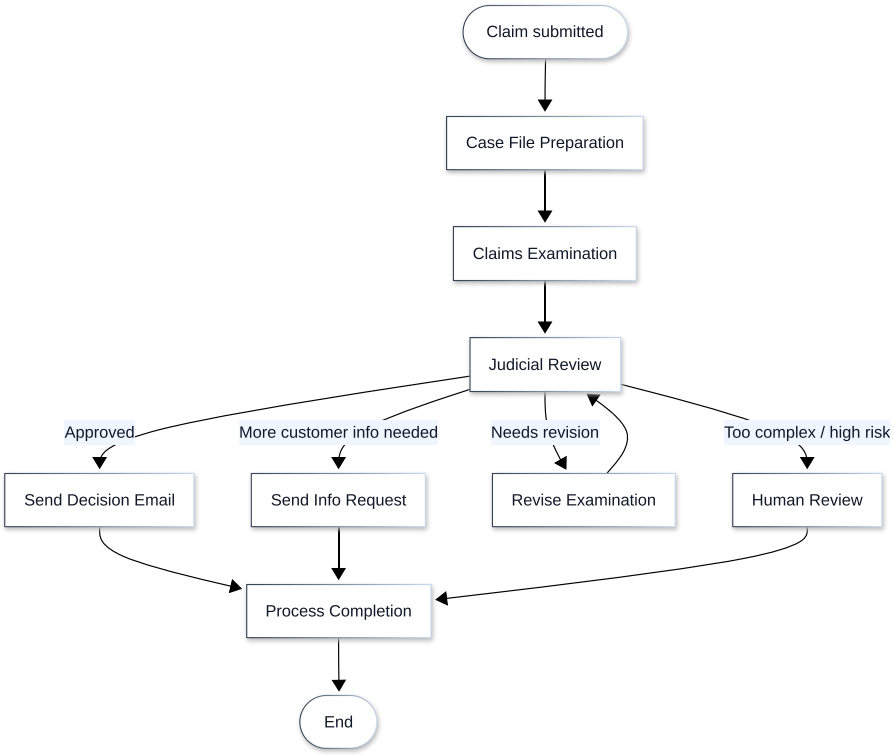


Figure 4.3: Simplified claim-processing workflow

If an active policy is found, the backend builds the request payload. The payload includes a generated `case_id`, the claim description, customer information, policy metadata, document metadata, the OneDrive file path and the hash of the relevant policy document. The `case_id` is generated as a hash-based identifier from the customer identifier and claim content so that submitted cases can be referenced without manually assigned case numbers.

The implemented workflow in CrewAI Studio is shown in Figure 4.4. It follows a controlled sequence rather than an unrestricted agent conversation. It builds the case file, performs the claims examination, validates the result through judicial review and routes the case to one of the defined next actions. Revision paths and terminal actions make the process bounded and traceable.

CrewAI Agent and Task Configuration. CrewAI represents the workflow through configured agents and tasks. Each agent is defined by a role name and descriptive attributes, including a goal and backstory, while the underlying language model can be selected individually for each agent. In the configured environment, available model identifiers included OpenAI models such as `gpt-4.1` [99], `gpt-4o-mini` [100] and `gpt-5.4` [101], as well as Google Gemini models such as `gemini-2.5-flash` [102], `gemini-3.1-pro-preview` [103] and `gemini-3.5-flash` [104]. For this PoC, `gpt-4.1` was used with a low temperature setting of `0.2` to encourage more factual and stable responses. Other configuration parameters, including maximum tokens, maximum iterations, maximum requests per minute and maximum execution time, were left at their default values.

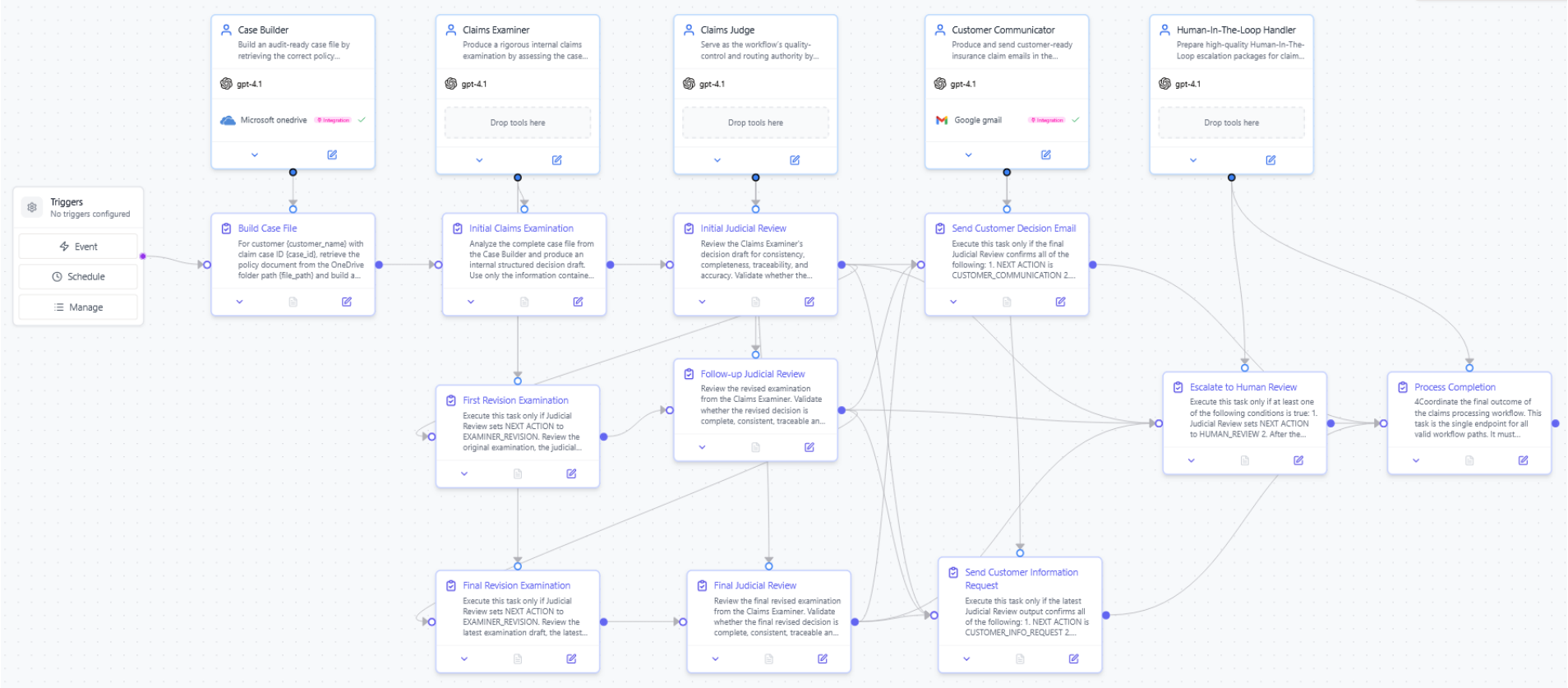


Figure 4.4: Implemented workflow in CrewAI Studio used in the claim-processing PoC

Agents can also be equipped with tools. Examples include OneDrive, Google Mail, internet search or custom tools. Tool usage requires configuration for a specific agent, authentication through an IdP and selection of the functions the agent is allowed to execute. In the PoC, the OneDrive integration was used to allow the Case Builder to list files by path and download the relevant prepared policy document. Google Mail is used for controlled communication by sending final answers or information requests to customers, but only if explicitly authorized and constrained.

A task has a name, description, expected outcome and, where required, task-level Guardrails. This aligns with specification-oriented work on LLM-based MAS, where agents, prompts, actions, data and evaluation functions are made explicit so that system behaviour can be executed and assessed more systematically [92]. After an agent generates a task output, the corresponding Guardrail checks whether the output satisfies the required rules and constraints. If the output passes, it becomes valid input for the next task. If it fails, the Guardrail returns a failure reason and the agent retries. After the configured retry limit is reached, the workflow stops with a traceable error.

The prompts follow an instruction-data separation at workflow level. Agents receive a task prompt and task context. The prompt contains trusted instructions, such as the task description and expected outcome. The context contains case-specific data, such as the customer claim, customer data, retrieved policy passages and previous task outputs. These data elements are treated as context and evidence, not as instructions that may override the task definition. This does not implement the full StruQ defence, because the PoC uses existing LLM and CrewAI capabilities rather than a structured-instruction-tuned model. However, it applies the same design principle at workflow level by separating trusted task instructions from untrusted or case-specific data [33].

Agent Roles and Responsibilities. The workflow is structured around specialised roles. Each role has a defined purpose and output responsibility. This role-specialised design is motivated by traceability work on sequential multi-agent pipelines, where explicit role boundaries and intermediate handoffs support debugging, accountability and error localization [58]. Table 4.6 summarises the roles and validation mechanisms used in the PoC.

Data Model and Knowledge Source. The PoC uses Supabase to store synthetic customer data, policy relationships and document metadata. The DB consists of three main tables: `customers`, `policies` and `documents`. The `policies` table links customers to their active policies and connects each policy to the corresponding policy document. Figure 4.5 complements Table 4.7 by showing the relationships between the three Supabase tables. One customer can have multiple policies, while each policy references one prepared policy document.

The query logic starts from the submitted `customer_id`. The system retrieves the customer record, identifies active policies, joins the policy record with document metadata and selects the latest policy by ordering records by `start_date`. If no active policy is found, the request is not sent to CrewAI. This supports traceability because every submitted claim can be linked to a customer, active policy and specific policy document.

Table 4.6: Agent roles and validation mechanisms in the claim-processing PoC

Role	Purpose	Architectural relevance
Case Builder	Prepares an audit-ready case file from the claim input, customer data, policy metadata and relevant policy evidence. It retrieves `document.txt` from OneDrive and extracts relevant policy passages verbatim.	Bridges static enterprise data and the dynamic agent workflow. Separates evidence preparation from claims assessment.
Claims Examiner	Analyses the claim against the available policy evidence and determines whether coverage appears confirmed, excluded, limited or dependent on missing information. Incorporates feedback from Judge in revision examinations.	Performs evidence-based reasoning while remaining grounded in the case file and extracted policy text.
Claims Judge	Judicially reviews the examination output for completeness, consistency, evidence support and reproducibility. It assigns a confidence score and selects the next workflow action. Provides feedback for revision.	Acts as quality-control and routing authority. Supports validation, revision, customer information requests and escalation.
Customer Communicator	Prepares customer-facing communication based only on the approved communication basis or information request basis. Sends the composed email to the customer through Google Mail.	Separates internal reasoning from external communication and reduces the risk of exposing unsupported or internal workflow details.
Human-In-The-Loop Handler	Prepares escalation packages for human review when automated processing cannot safely continue and produces the final process summary for completed workflow paths. It summarises case context, examination history, judicial concerns, routing reasons and recommended reviewer actions.	Provides a controlled handoff for ambiguous, high-risk or unresolved cases. Preserves accountability and keeps escalation decisions and final workflow outcomes traceable and audit-ready. It also finishes the process by producing a summary with final information of the workflow and chronological order of actions taken.
Guardrail Agent and task-level Guardrails	Validate task outputs against structural, evidence and routing constraints. Failed outputs are rejected and retried until the retry limit is reached.	Provide deterministic checks around probabilistic outputs and make validation failures traceable through explicit failure reasons.

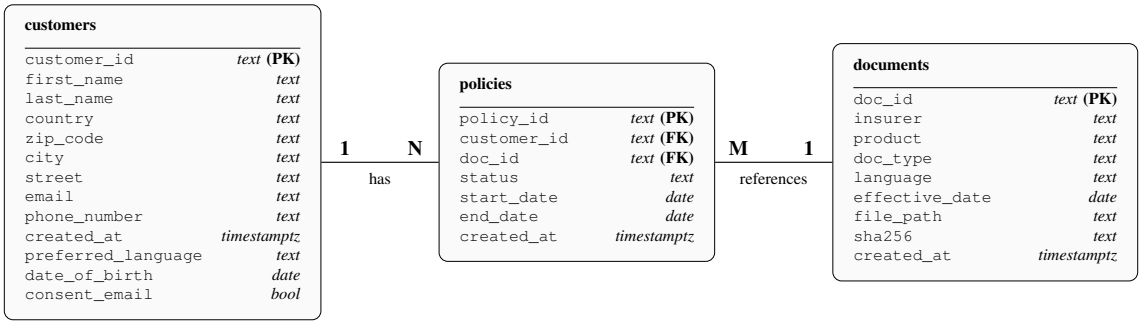


Figure 4.5: Supabase data model used in the claim-processing PoC as an entity relationship diagram

Table 4.7: Supabase data model description used in the claim-processing PoC

Table	Relevant fields	Purpose in the PoC
customers	customer_id, name fields, address fields, email, phone number, preferred language, date of birth, email consent.	Stores synthetic customer master data and supports identification, communication and language-specific behaviour.
policies	policy_id, customer_id, doc_id, status, start date, end date, creation timestamp.	Links customers to active policies and connects a policy to the relevant document record. Many-to-Many relationship (N-M) between documents and customers in data model.
documents	doc_id, insurer, product, document type, language, effective date, file path, SHA-256 hash, creation timestamp.	Stores metadata for prepared policy documents and links DB records to the OneDrive knowledge source.

The policy documents are prepared from original Portable Document Format (PDF) files into document-specific folders. For each source document, the preparation pipeline creates `document.txt`, `pages.json`, `chunks.json` and `meta.json`. The file `document.txt` contains extracted policy text with page markers, `pages.json` stores text per page, `chunks.json` stores overlapping chunks and `meta.json` stores extraction metadata, hash values and processing statistics.

Although chunks are generated, the PoC primarily uses `document.txt`. This simplification avoids multiple PDF parsing at runtime and keeps the workflow easier to inspect and reproduce. In a production setting, chunk-based retrieval, hybrid search or a full RAG platform would be more appropriate to reduce token usage, improve relevance ranking and support larger document repositories.

Retrieval and Evidence Handling. Retrieval and evidence handling are divided between the WebApp, Supabase, OneDrive and the Case Builder. The WebApp retrieves structured customer, policy and document metadata from Supabase. The Case Builder uses the provided `file_path` to locate the prepared policy document in OneDrive, confirm that `document.txt` exists, download it and extract relevant policy passages.

The Case Builder has a deliberately narrow role. It retrieves and prepares evidence, but it must not determine coverage, interpret policy meaning or provide recommendations. Instead, it extracts relevant clauses, definitions, exclusions, conditions and surrounding paragraphs exactly as written in the policy document. This separates evidence assembly from claim assessment and reduces the risk that unsupported interpretation enters the workflow during retrieval.

The expected output is an audit-ready case file containing meta and customer data, the original claim description and source-faithful policy passages. This supports traceability because downstream agents receive a structured case file that links the claim, customer, policy, document and evidence passages.

WebApp and REST Workflow Execution. The WebApp provides the user-facing entry point for the PoC. It is implemented as a lightweight Flask application and acts as a controlled test harness for self-written and predefined claim cases. It combines user input, DB lookup, payload construction, request execution and response visualisation.

After a valid submission, the backend generates the `case_id`, constructs the CrewAI request payload and sends it as a REST API POST request to the configured `CREWAI_URL`. Configuration values such as `CREWAI_URL`, `API_KEY` and `SUPABASE_URL/KEY` are stored as environment variables. Secrets are omitted or masked in the thesis documentation.

A simplified version of the request payload is shown below.

```
{
  "inputs": {
    "case_id": "996a4582d30a1475aed94c2b12c5d2f3",
    "claim_case": "...",
    "customer_name": "Luca Becker",
    "email": "luca.becker@gmx.de",
    "preferred_language": "de",
    "insurer": "R+V",
    "product": "Hausrat-Classic S",
    "file_path": "Insurance_Knowledge_Base/R+V/...",
    "sha256": "0a63844***4cece29fdc"
  }
}
```

After submission, the WebApp displays the generated `claim_id`, the POST request body, the status code and the response returned by the CrewAI endpoint. For a successful request, the endpoint returns a `kickoff_id`, which identifies the started workflow execution. This result view supports debugging and traceability because it makes the exact workflow input visible during evaluation.

Guardrails and Routing Logic. The PoC uses routing logic and task-level Guardrails as related but distinct controls. Routing logic determines which workflow path should be taken. Guardrails validate whether a task output is acceptable before it is passed downstream. Since LLM-based agents can produce variable outputs, the workflow does not rely only on prompt instructions. Selected tasks therefore define expected output structures, validation rules and failure conditions. The detailed Guardrail definitions are provided in Appendix D.

Figure 4.6 shows the detailed routing logic implemented in the PoC. In contrast to the simplified workflow in Figure 4.3, it includes the first and second revision loops and the final judicial review. The diagram therefore represents the bounded revision structure used to avoid unrestricted agent loops.

The routing logic is mainly controlled by the judicial review tasks. After the Claims Examiner produces an examination draft, the Claims Judge reviews it for evidence support, completeness, reproducibility and internal consistency. The Judge assigns a confidence score from 0 to 100 and selects the next workflow action. This score is PoC-specific and is not a statistical probability. It is used as a structured workflow signal for routing.

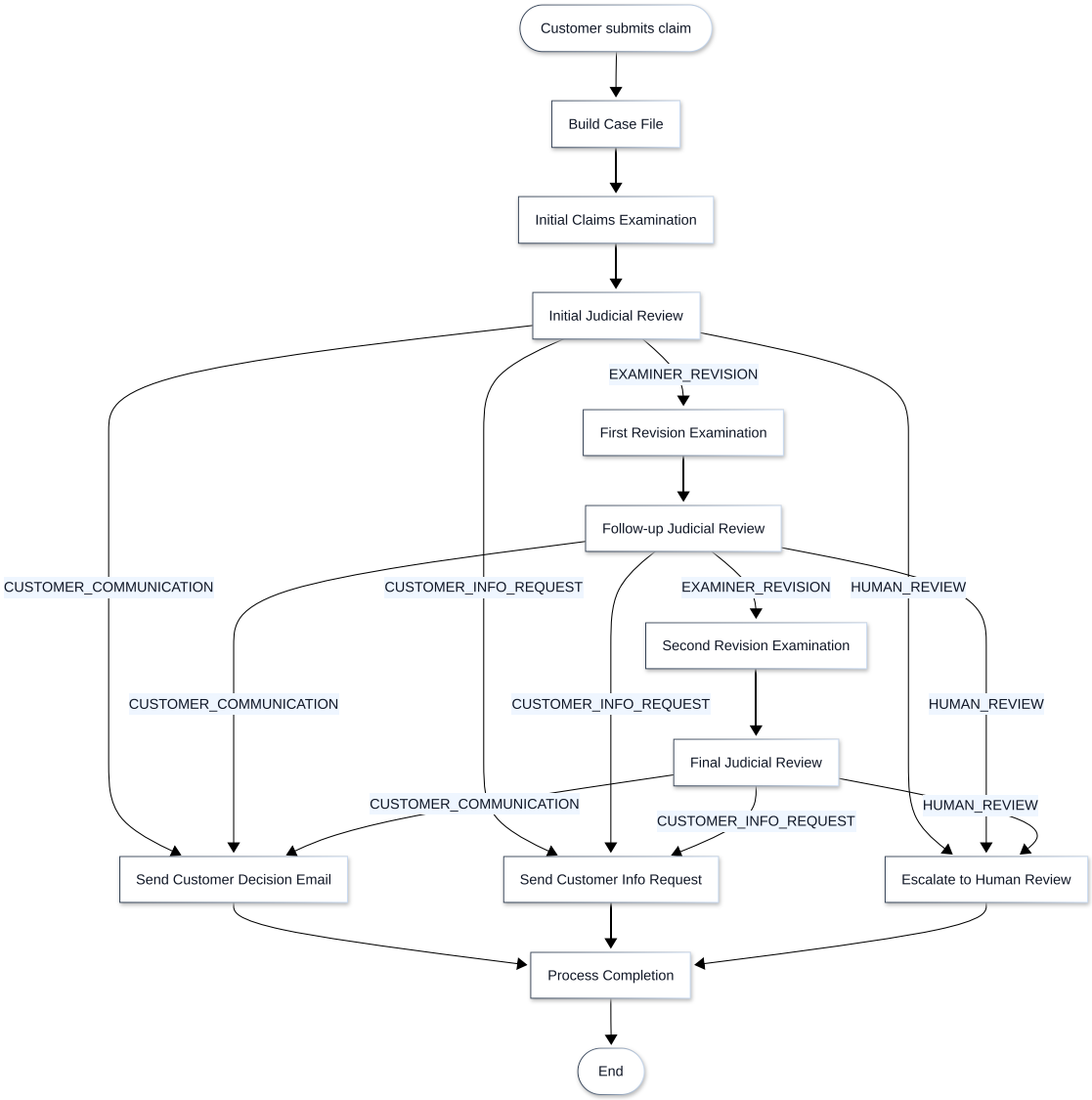


Figure 4.6: Detailed routing logic of the claim-processing workflow

Table 4.8: Routing actions used in the claim-processing PoC

Routing action	**Meaning**
`CUSTOMER_COMMUNICATION`	The case is sufficiently supported and ready for customer-facing communication through email (Confidence > 95). It is either covered or not covered.
`CUSTOMER_INFO_REQUEST`	Specific customer-providable information or documents are missing. Request is sent to customer through email.
`EXAMINER_REVISION`	The examination contains specific and fixable issues that can be addressed using the existing case file (Confidence $\leq$ 95).
`HUMAN_REVIEW`	The case is ambiguous, complex, high-risk, fraud-related, regulatory or unsuitable for automated continuation. Escalated to HITL Handler

The routing rules are intentionally conservative. If the confidence score is greater than 95 and the case is ready for final communication, `CUSTOMER_COMMUNICATION` may be selected by the Judge. If specific customer-providable information is missing, the

Judge may select `CUSTOMER_INFO_REQUEST`. If the confidence score is 95 or below and the issues are specific and fixable using the existing case file, the Judge may select `EXAMINER_REVISION`. If the case is complex, ambiguous, high-risk, fraud-related or regulatory, the Judge may select `HUMAN_REVIEW`. Fraud or regulatory concerns can require human review regardless of the confidence score.

The selected next action is then validated by a Guardrail. The Guardrail checks whether the judicial review output contains a valid action from the allowed action set and whether the selected action is consistent with the stated confidence, evidence situation and severity of the case. Only a Guardrail-approved output is accepted as valid input for the next workflow step.

The workflow supports bounded revision cycles. After the initial examination, the Judge can route the case back to the Examiner for revision. The revised examination is reviewed again, and a second revision is possible before the final judicial review. The final judicial review selects a terminal path and cannot route back to another examination step. This prevents infinite revision loops and keeps the process bounded.

Guardrails therefore act as output validators for individual tasks. For the Case Builder, they check that required metadata is present and that policy text is copied verbatim from `document.txt`. For the Claims Examiner, they check that required analytical sections are present and supported by the case file. For judicial review, they check that the confidence score is valid, that the selected next action belongs to the allowed action set and that the routing decision is consistent with the evidence situation and severity rules. Customer-facing tasks are guarded so that communication is generated only from approved handoff material, without containing sensitive data. The HITL Handler Guardrails control the outputs, by emphasizing human readability and formatting such as chronological order.

If a Guardrail check fails, the workflow does not continue with the invalid output. Instead, the Guardrail returns a failure reason and the responsible agent retries the task. If the output still fails after three attempts, the workflow ends with an error. This behaviour supports traceability because failed outputs and Guardrail reasons can be inspected afterwards. Guardrails therefore do not replace authorization, human review or domain validation, but they provide deterministic validation points around probabilistic agent outputs.

Overall, the routing logic and Guardrails operationalize several principles of the proposed RA. Routing defines the possible workflow paths, while Guardrails decide whether generated task outputs are valid enough to move along those paths. Together, they support structured evidence handling, bounded revision, traceable failures, governance and controlled continuation of the workflow.

4.4 Auditing Artifact Design

The second thesis artifact focuses on auditability and tamper-evident traceability for agentic workflows. While the insurance claim-processing PoC demonstrates the business workflow, the auditing artifact demonstrates how workflow events can be represented as structured evidence and protected against unnoticed manipulation.

The artifact is intentionally separated from the main claim-processing PoC. In a production system, audit events would be generated dynamically during workflow execution. Each relevant task, agent action, tool invocation, Guardrail result, routing decision and workflow transition would create a structured audit event. In this thesis, these events are

represented as pre-exported JSON Lines (JSONL) records based on an observed CrewAI workflow execution.

The artifact demonstrates the following evidence chain:

`workflow event` → `JSONL audit record` → `record hash` → `Merkle leaf` → `Merkle root` → `audit block` → `blockchain-style chain`

The complete raw JavaScript Object Notation (JSON) examples, detailed field lists, full Merkle tree figures and verification report are provided in Appendix E. The following subsections describe the design rationale, Merkle tree evidence logging, blockchain-style root chaining and verification logic.

4.4.1 Motivation for a separate Artifact

Auditability is a central quality attribute of the proposed RA. A secure tool-using MAS should not only execute a workflow, but also produce evidence that allows the workflow to be reconstructed and reviewed afterwards. Relevant evidence includes which agents were involved, which tasks were executed, which tools were used, which Guardrails were applied, which routing decisions were made and which outputs were produced.

Full integration of cryptographic audit mechanisms into the CrewAI workflow was outside the implementation scope. Although custom tool integration was possible in principle, additional tools introduced practical issues related to module resolution, dependencies and the runtime infrastructure of the provided CrewAI instance. A fully custom Python implementation would have allowed deeper audit integration, but implementing orchestration, tool use, routing, document access, communication and audit logging from scratch would have shifted the thesis away from its architectural focus.

Separating the artifact therefore improves clarity. The claim-processing PoC focuses on workflow execution, evidence-based claim handling and routing. The auditing artifact focuses on evidence integrity, tamper detection and verifiable traceability. The concept remains transferable because the JSONL records represent the type of structured events that a production MAS could generate during runtime.

The artifact focuses on cryptographic integrity, not confidentiality. It uses Secure Hash Algorithm (SHA)-256 hashes to make changes detectable, but it does not encrypt audit records. This keeps the records inspectable for verification and debugging, while having separate integrity evidence. In a production setting, encryption with Advanced Encryption Standard (AES) in Galois/Counter Mode (GCM) could protect sensitive audit content at rest or in transit, while digital signatures, such as Ed25519 signatures, could prove which system component created or approved an audit commitment.

4.4.2 Merkle tree Evidence Logging

The auditing artifact uses structured JSONL records as input for Merkle tree evidence logging. Each line represents one audit record from a workflow execution event of the PoC. The records contain workflow context, step information, agent and task metadata, inputs, outputs, Guardrail status, tool usage and timing information. A SHA-256 hash is calculated for each record and stored as `record_hash`. These hashes become the leaves of the Merkle tree. A simplified audit record is shown next. The full record structure is included in Appendix E.

```
{
  "schema_version": "1.0",
  "record_id": "113139c2-930a-4cb8-9a49-351a097931c2",
  "record_hash": "592ca7af***81b2f8d5d",
  "record_hash_algorithm": "sha256",
  "workflow_id": "wf-001",
  "step_id": 1,
  "event_type": "build_case_file",
  "parent_record_id": "a6d0f4a5-3d4a-4d46-966b-49ba8ca6fe12",
  "crew_name": "InsuranceClaims***RevisionSystemCrew",
  "agent_name": "Case Builder",
  "task_name": "Build Case File",
  "task_status": "completed",
  "input_messages": [{"inputs": "..."}],
  "output": "...",
  "guardrail_status": null,
  "started_at": "2026-05-07T00:54:05.313Z",
  "ended_at": "2026-05-07T00:54:29.481Z",
  "tool_invocation": null
}
```


Table 4.9 summarises the main field groups. The fields are designed to support workflow reconstruction and later integrity verification.

Table 4.9: Main field groups of a JSONL audit record

Field group	**Attributes**	**Purpose**
Record identity	`schema_version`, `record_id`	Identifies the record and schema version.
Record integrity	`record_hash`, `record_hash_algorithm`	Stores the hash used as the Merkle leaf. The hash is computed over all record fields except the hash fields themselves.
Workflow context	`workflow_id`, `step_id`, `event_type`, `parent_record_id`	Links the record to a workflow execution and preserves event order and parent-child relationships.
Crew, agent and task metadata	`crew_id`, `crew_name`, `agent_name`, `agent_id`, `agent_version`, `task_name`, `task_id`, `task_version`, `task_status`	Shows which crew, agent and task were involved in the recorded event.
Inputs and outputs	`input_messages`, `output`	Stores the relevant workflow inputs, context and generated outputs needed for later reconstruction.
Guardrails and tools	`guardrail_version`, `guardrail_status`, `guardrail_failure_reason`, `guardrail_retry_count`, `tool_invocation`	Captures Guardrail information such as validation results, failure reasons, retries and tool usage.
Timing	`started_at`, `ended_at`	Records when the event started and ended.

The implementation loads the audit log line by line, parses each JSONL record, extracts the `record_hash` and validates that it has the expected SHA-256 length of 64 hexadecimal characters. The hash is stored together with the line number, step identifier and record identifier.

The leaves are ordered according to the JSONL line order. This is important because the event sequence is part of the evidence structure. If records are reordered, the Merkle root changes. The leaf ordering strategy is therefore recorded as `jsonl_line_order`.

The Merkle tree is built bottom-up. Adjacent record hashes are paired, concatenated as bytes and hashed with SHA-256. The resulting hash becomes the parent node. This process is repeated until one hash remains: the Merkle root. If a level contains an uneven number of nodes, the last node is duplicated. The artifact records this strategy as `duplicate_last_leaf`.

From a security perspective, this construction depends on collision-resistant hashing: a manipulated record should not feasibly produce the same record hash or Merkle root. Recent work on Merkle trees in blockchain systems highlights that collision probability, digest length and hash-function assumptions are part of the security argument for Merkle commitments [68]. The artifact therefore uses SHA-256 hashes as integrity commitments for audit records.

The generated Merkle commitment is written to a JSON file. It contains the workflow identifier, source audit log, hash algorithm, leaf count, tree depth, Merkle root, creation timestamp, ordering strategy, odd-leaf strategy and the list of leaves used to construct the tree.


```
{
  "workflow_id": "wf-001",
  "source_file": "audit_logs/audit_log_hashed.jsonl",
  "hash_algorithm": "sha256",
  "leaf_count": 78,
  "tree_depth": 8,
  "merkle_root": "3d0f30145b***4724d6d0c416",
  "created_at": "2026-05-12T04:28:38.542218Z",
  "leaf_order": "jsonl_line_order",
  "odd_leaf_strategy": "duplicate_last_leaf",
  "leaves": [
    {
      "line_number": 1,
      "step_id": 0,
      "record_id": "a6d0f4a5-3d4a-4d46-966b-49ba8ca6fe12",
      "record_hash": "5c037aa6e3b***92b39dfe0d18ce"
    }
  ]
}
```


Table 4.10 summarises the commitment fields.

Table 4.10: Fields of the Merkle commitment

Field	Purpose
workflow_id	Identifies the workflow execution represented by the commitment.
source_file	Stores the JSONL audit log used as input.
hash_algorithm	Defines the hash algorithm used for leaves and parent nodes.
leaf_count and tree_depth	Record the number of audit records and the number of Merkle tree levels.
merkle_root	Stores the root hash that compactly commits to all included audit records.
created_at	Records when the commitment was generated.
leaf_order	Defines the ordering rule for leaves. In this artifact, JSONL line order is used.
odd_leaf_strategy	Defines how uneven tree levels are handled. In this artifact, the last leaf is duplicated.
leaves	Lists included records with line number, step identifier, record identifier and record hash.

This design provides a compact integrity commitment for the complete audit log. If an audit record is modified, removed, inserted or reordered and the root is recomputed, the Merkle root changes. Compared with a simple comparison of complete audit logs, the Merkle tree structure supports scalable and space-efficient verification: when intermediate nodes or Merkle proofs are preserved, a verifier can check the affected record and its hash path to the root instead of comparing every record. The Merkle tree does not prevent manipulation by itself, but it makes manipulation detectable if the original commitment is preserved or anchored in a later structure.

To demonstrate tamper detection, the artifact also includes a deliberately manipulated JSONL file. This malicious file is created as a copy of the original audit log, but selected fields are changed after the original record hashes and Merkle commitment have already been generated. For example, one record changes the record_id field to MALICIOUSLY_CHANGED_RECORD_ID, while other records modify fields such as task_status or guardrail_status. The stored record_hash still corresponds to the original record, while the record content has been changed.

```
{
  "record_id": "MALICIOUSLY_CHANGED_RECORD_ID",
  "record_hash": "5c037aa6e3b***92b39dfe0d18ce",
  "workflow_id": "wf-001",
  "step_id": 0,
  "event_type": "crew_kickoff_completed",
  "output": "...",
  "started_at": "2026-05-07T00:53:57.120Z",
  "ended_at": "2026-05-07T00:56:24.855Z"
}
```

The manipulated file shows the practical value of the commitment. A normal log file can be edited without obvious indication, but a hashed and committed audit log ex-

poses such changes during verification. This supports tamper-evident traceability in the proposed RA.

4.4.3 Blockchain-style Root Chaining

After the Merkle root has been generated, the artifact commits it into a blockchain-style hash chain. The Merkle tree provides an integrity commitment for one audit log, while the chain links multiple commitments into an ordered sequence.

Each block stores the Merkle root, workflow metadata and the hash of the previous block. The block itself is hashed using a canonical JSON representation that excludes the `block_hash` field. Any modification to the block content changes the block hash. Since each following block stores the previous block hash, manipulation of an earlier block breaks the chain.

In the implementation, the script `create_audit_block.py` loads the generated Merkle commitment, reads the latest previous block from the blockchain JSONL file, creates a new block, computes its block hash and appends it to the blockchain file. If no previous block exists, the new block becomes block number 0 and `previous_block_hash` is set to `null`. Otherwise, the block number is incremented and the previous block hash is copied from the latest existing block. Table 4.11 summarises the block fields.


```
{
  "block_schema_version": "1.0",
  "block_number": 0,
  "workflow_id": "wf-001",
  "created_at": "2026-05-11T15:06:31.936135Z",
  "previous_block_hash": null,
  "merkle_root": "3d0f30145b***4724d6d0c416",
  "merkle_hash_algorithm": "sha256",
  "leaf_count": 78,
  "tree_depth": 8,
  "leaf_order": "jsonl_line_order",
  "odd_leaf_strategy": "duplicate_last_leaf",
  "source_commitment_file": "audit_merkle_commitment.json",
  "source_jsonl_file": "audit_log_hashed.jsonl",
  "block_hash": "02cd0aef3***b4b80cbe1a5"
}
```


Through the `previous_block_hash` field the previous block is referenced by the next block. If block 0 is modified, its recomputed block hash changes. As a result, block 1 no longer points to the correct previous block and verification fails. In the thesis artifact, several blocks are appended to demonstrate the chaining mechanism. The same Merkle root may appear in multiple blocks because the artifact repeatedly commits the same audit log during testing. In a real system, each completed workflow execution or audit interval could produce a new Merkle root and each new root could be appended as a new block.

This mechanism is blockchain-style, but it is not a full blockchain network. It does not include decentralization, consensus, mining, smart contracts or distributed replication. Instead, it uses the core integrity principle that each block contains a cryptographic reference to its predecessor. This is sufficient for the thesis artifact because the goal is to demonstrate tamper-evident audit commitments, not to implement a decentralized ledger.

Table 4.11: Fields of a blockchain-style audit block

Field	Purpose
`block_schema_version`	Defines the schema version of the audit block.
`block_number`	Defines the position of the block in the chain.
`workflow_id` and `created_at`	Identify the workflow and timestamp of the commitment.
`previous_block_hash`	Links the block to its predecessor. The first block stores `null`.
`merkle_root`	Stores the Merkle root representing the committed audit log.
`merkle_hash_algorithm`, `leaf_count`, `tree_depth`, `leaf_order`, `odd_leaf_strategy`	Preserve the Merkle tree construction metadata.
`source_commitment_file` and `source_jsonl_file`	Link the block to the generated commitment and source audit log.
`block_hash`	Stores the hash of the block content and acts as the block integrity identifier.

4.4.4 Verification Logic

The verification logic connects the audit records, Merkle commitment and blockchain-style chain into one evidence chain. Its purpose is to verify that the audit log still matches the original commitment and that it remains anchored in the block chain. The script `verify_audit_chain.py` performs three checks, summarised in Table 4.12.

Table 4.12: Verification steps of the auditing artifact

Verification step	Check	Failure indication
Record hash verification	Recomputes each record hash from its canonical JSON representation. Derived fields such as `record_hash`, `record_hash_algorithm`, `merkle_leaf_hash` and `merkle_proof` are excluded.	A mismatch indicates that the record content was changed after hashing.
Merkle commitment verification	Rebuilds the Merkle root from the verified record hashes and compares it with the stored `merkle_root`. It also checks the expected `leaf_count`.	A root mismatch indicates that the audit log no longer matches the original commitment.
Blockchain-style chain verification	Checks block numbers, previous-block references, recomputed block hashes and whether the latest block commits the expected Merkle root.	A mismatch indicates that one or more blocks or block links were modified.

To test tamper detection, the artifact includes a manipulated copy of the audit log in which selected fields were changed after the original record hashes and Merkle commitment had already been generated. During verification, the changed records produce different recomputed hashes. These differences propagate to the Merkle root and cause the manipulated file to fail verification against the original commitment.

In the context of the proposed RA, this verification logic illustrates how audit and

traceability could support accountability in tool-using MAS. In a production system, logs would be generated during execution and commitments could be created automatically after workflow completion or at defined intervals. The thesis artifact uses exported records for feasibility reasons, but the verification principle is transferable to real systems.

4.5 Design Decisions and Trade-offs

The design of the proposed RA, the claim-processing PoC and the auditing artifact required several deliberate trade-offs. These decisions were shaped by the thesis scope, available implementation resources, tool limitations and the aim of producing an explainable architecture rather than a production-ready enterprise system.

Table 4.13 summarises the most important design decisions. The common trade-off across these decisions is that the artifacts should be realistic enough to demonstrate the proposed RA, but simple enough to remain inspectable, reproducible and feasible within the thesis scope. Therefore, the design prioritizes architectural clarity, traceability and security reasoning over production-scale completeness.

4.6 Summary

This chapter presented the design of the proposed RA and the two artifacts used to instantiate selected architectural concerns. The RA provides a layered, platform-independent blueprint for secure tool-using MAS. It separates dynamic LLM-based agents from static enterprise components and defines architectural layers for orchestration, agents, knowledge access, integration, identity, governance, logging, storage, operations and HITL oversight.

The insurance claim-processing PoC demonstrates how selected parts of the RA can be instantiated in a controlled enterprise workflow. It combines a Flask WebApp, a Supabase DB, prepared policy documents in OneDrive and a CrewAI workflow with specialised agents, routing logic and task-level Guardrails. The PoC shows how customer data, policy metadata, document evidence and agent-generated outputs can be connected in a traceable workflow, while also making clear which parts of the full RA remain simplified or conceptual.

The auditing artifact complements the PoC by focusing on tamper-evident traceability. It shows how structured workflow events can be represented as JSONL audit records, hashed, aggregated into a Merkle tree and linked through blockchain-style root chaining. The artifact does not provide a full production audit platform, but demonstrates how audit evidence from agentic workflows can be protected against unnoticed manipulation.

The design decisions and trade-offs show that the thesis prioritizes architectural clarity, traceability and feasibility over production completeness. The resulting artifacts are therefore intended as controlled partial instantiations of the proposed RA. The following chapter evaluates the resulting RA, the claim-processing PoC, the auditing artifact and the expert questionnaire results.

Table 4.13: Design decisions and trade-offs for the claim-processing PoC

Decision	Reason	Consequence
No full vector DB or advanced RAG platform	A full vector DB could support semantic search, scalable retrieval, ranking and efficient handling of larger document collections. However, implementing and evaluating such infrastructure would have shifted the focus away from the central research problem: the design of a secure-by-design RA for tool-using MAS. The selected policy documents are limited in size, so a lightweight approach was sufficient for the controlled PoC.	The PoC remains simpler, easier to inspect and easier to explain. The relationship between document source, extracted evidence and workflow output stays transparent. However, this approach does not scale well to large enterprise repositories, many policy versions or complex retrieval scenarios. A production system should use chunk-based retrieval, hybrid search or a full RAG platform.
Prepared knowledge source instead of raw PDF processing at runtime	The prepared knowledge source contains extracted policy text, page information, chunks and metadata for each policy document. This avoids repeated PDF parsing during workflow execution and creates a more stable input for the agent workflow with reduced token input. The `document.txt` file allows the Case Builder to retrieve and inspect policy content through the OneDrive tool.	The approach is sufficient for the PoC because the evaluation uses a controlled set of cases and documents. It supports traceability from claim decision to policy document and reduces implementation complexity. However, reading complete documents into the workflow increases token usage and would not be ideal for larger environments. The generated chunks indicate how a future retrieval extension could be implemented.
Use a centralized DB instead of Solid Pods for the PoC	Solid Pods were examined as an architectural option because they are relevant for data provenance, user sovereignty, identity-based access and fine-grained consent. They are especially attractive in cross-organisation scenarios, such as healthcare ecosystems, where a data subject controls access to personal data and grants permissions to different actors or systems [105, 106]. However, the claim-processing PoC is insurer-centric. The authoritative customer master data, policy metadata and policy documents are owned and maintained by the insurance organisation.	Supabase was selected because it better fits the authority model of the PoC and provides a simple way to model customers, policies and document metadata. Solid Pods would have added platform complexity, including Solid server setup, WebID/Solid-OIDC, Access Control List (ACL) management, consent handling and user-facing permission workflows. Solid remains relevant for future scenarios where customers own the data and grant scoped access to agents, insurers or third parties.

Continued on next page

Table 4.13: Design decisions and trade-offs for the claim-processing PoC (continued)

Decision	Reason	Consequence
Separate the auditing artifact from the main PoC and focus on cryptographic integrity instead of a full cryptographic security layer	Auditability is central to the RA, but live integration of cryptographic audit mechanisms into CrewAI would have required additional custom tooling and deeper runtime control. The separate artifact therefore focuses on integrity verification through SHA-256 record hashes, Merkle roots and blockchain-style hash chaining [83, 68]. The artifact does not implement confidentiality mechanisms such as AES-GCM encryption or authenticity mechanisms such as Ed25519 signatures, because the goal was to demonstrate tamper evidence rather than a complete cryptographic audit platform.	The PoC can focus on claim processing, while the auditing artifact demonstrates tamper detection and verifiable traceability. The trade-off is that audit commitments are not generated live during workflow execution. The artifact does not implement confidentiality or authenticity mechanisms such as AES-GCM encryption or Ed25519 signatures. A production system should combine integrity, encryption, secure transport, key management and digital signatures for stronger authenticity.
Use CrewAI for the PoC, but keep the RA framework-independent	CrewAI was useful for implementing agents, tasks, tools, Guardrails and sequential workflow execution. It also provided platform-level functionality such as tool integration, event traces, conversation history and Guardrail execution. However, an orchestration framework is not equivalent to a complete enterprise architecture.	The PoC demonstrates feasibility with an available framework, while the proposed RA remains reusable beyond CrewAI. The architecture describes layers, components, trust boundaries, evidence flows and governance mechanisms that could also be implemented with another framework or a custom orchestration layer.
Controlled workflow orchestration instead of decentralized self-evolving coordination	Recent work such as AgentNet explores decentralized and dynamically evolving coordination for LLM-based MAS [72]. Such approaches can improve adaptability, but make workflow reconstruction, deterministic routing and governance more difficult. The insurance use case requires bounded execution, traceable task order and clear review points.	The PoC uses a controlled CrewAI workflow with defined tasks, routing actions and revision limits. This improves traceability and auditability, but does not evaluate decentralized or self-evolving coordination. Future work could examine whether such MAS can still satisfy enterprise governance and audit requirements.
Prioritize architectural clarity and feasibility over production completeness	The thesis is a master thesis rather than a full enterprise implementation project. A production system would require enterprise IAM, fine-grained authorization, production monitoring, access-controlled retrieval, secure deployment, incident response, live audit integration and legal validation. Implementing all of these would exceed the thesis scope.	The artifacts demonstrate selected architectural concepts rather than operational maturity. The PoC should therefore not be interpreted as a production-ready secure MAS. Instead, it serves as a controlled demonstrator for evaluating design principles, feasibility and future work.

5 Results and Analysis

This section presents the results of the thesis and analyses them in relation to the RQs. The results cover four main elements: the expert questionnaire, the proposed RA, the insurance claim-processing PoC and the separate auditing artifact. Together, these elements show how secure, traceable and compliant tool-using MAS can be designed and instantiated in an enterprise-oriented context.

The section first presents the results descriptively. It summarises the expert-informed design priorities, the resulting RA, the implemented PoC and the auditing artifact. The aim is to describe what was produced and observed without detailed interpretation.

The second part analyses these results in relation to the RQs. It discusses how the proposed architecture addresses secure-by-design requirements, how the PoC and auditing artifact support feasibility and traceability, and where limitations remain.

5.1 Results

The results are presented in the order of their role in the research process. First, the expert questionnaire is summarised as input for identifying architectural priorities and practical concerns. Second, the proposed RA is presented as the central design result. Third, the claim-processing PoC and the auditing artifact are summarised as technical results that instantiate and examine selected parts of the architecture.

This distinction is important for interpreting the results. The questionnaire provides expert-informed requirements and design priorities. The RA represents the main architectural contribution. The PoC and auditing artifact demonstrate feasibility under controlled conditions, but they are not intended to prove production readiness.

5.1.1 Expert questionnaire results

The expert questionnaire collected practitioner perspectives on secure tool-using MAS. Since no complete RA was presented to the participants, the results are interpreted as expert-informed requirements and design priorities rather than as direct validation of the final architecture.

A total of 25 experts participated. The respondents represented roles in software architecture, cloud architecture, cybersecurity, enterprise architecture, GenAI architecture, software development, data engineering, technical leadership and AI-enabled solution design. The sample therefore provides relevant practical input, but it is not statistically representative.

Participant Profile The participant profile is shown in Figures 5.7 and 5.8. The sample was strongly weighted toward experienced practitioners, with many respondents reporting substantial experience in software or enterprise architecture. Familiarity with GenAI and LLMs was generally high, while familiarity with MAS and tool-using agents was more mixed. Overall, the group is suitable for expert-informed requirements elicitation. However, because MAS-specific expertise varied, the findings should be understood as practical design guidance rather than as specialist validation of the final RA.

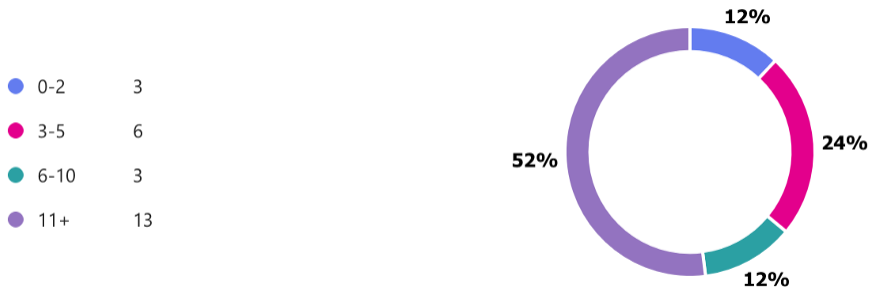
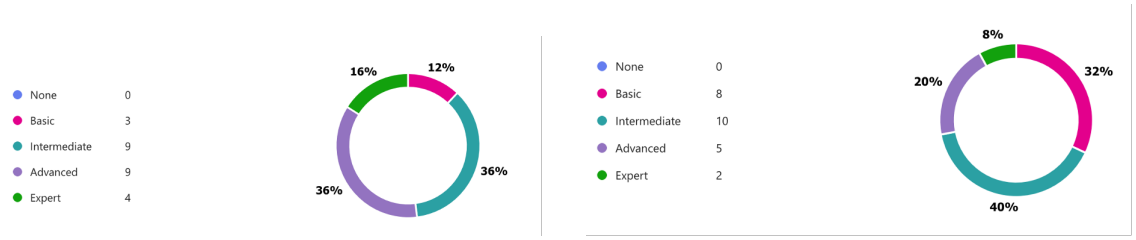


Figure 5.7: Participants' experience in software and enterprise architecture



(a) GenAI and LLM familiarity

(b) MAS and tool-using agent familiarity

Figure 5.8: Participant familiarity with GenAI, LLMs and MAS

Enterprise Relevance and Adoption Blockers Figures 5.9 and 5.10 summarise the perceived relevance of tool-using MAS. Participants mainly associated such systems with knowledge-intensive and workflow-oriented scenarios, including internal knowledge assistants, service desk automation, document workflows, customer support, data analysis, compliance workflows and software development. The adoption blockers show that security, organisational readiness, compliance, auditability, governance and reliability were more prominent than performance or cost. This suggests that enterprise adoption depends not only on technical capability, but also on trust, control, governance and accountable operation.

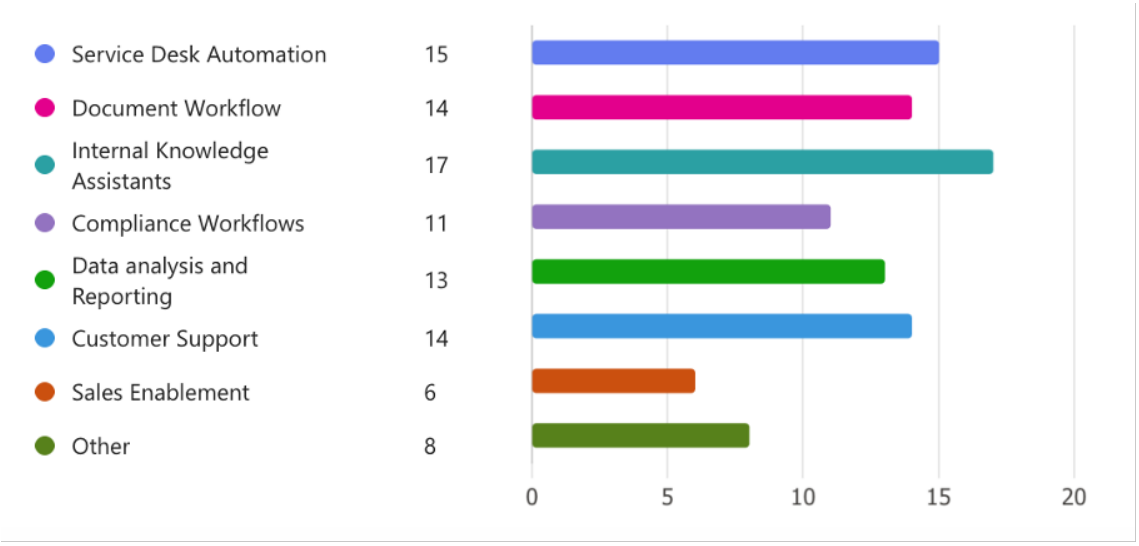


Figure 5.9: Enterprise use cases for tool-using MAS

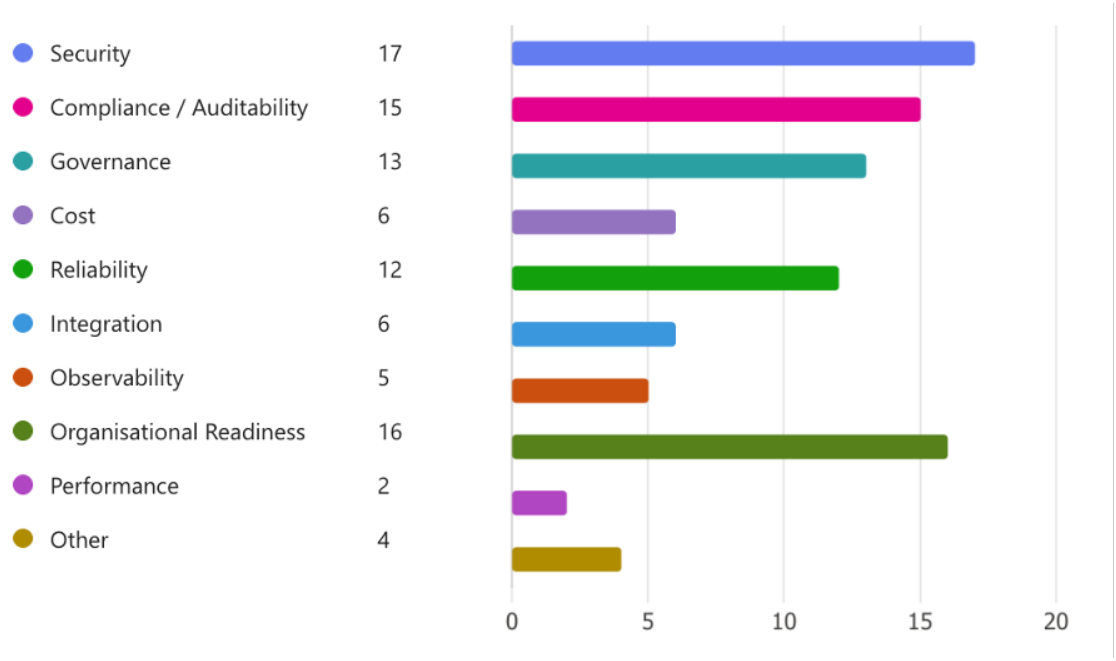


Figure 5.10: Adoption blockers for tool-using MAS

Quality Attributes and Architecturally Significant Requirements Figure 5.11 and Figure 5.12 show that security, compliance, auditability and reliability were the dominant quality attributes. The most important ASRs concerned prevention of unauthorised tool execution and prevention of sensitive data leakage.

The open-text answers added supporting qualities such as explainability, transparency, controllability, traceability, provenance, resilience, privacy, determinism, confidentiality and integrity. Other important ASRs included HITL gating for high-risk actions, traceability / audit completeness and robustness against direct and indirect prompt injection. Together, these results support an architecture in which authorization, least privilege, data protection, prompt-injection mitigations, audit logging and approval workflows are treated as core design concerns.



Figure 5.11: Most important quality attributes for secure enterprise MAS

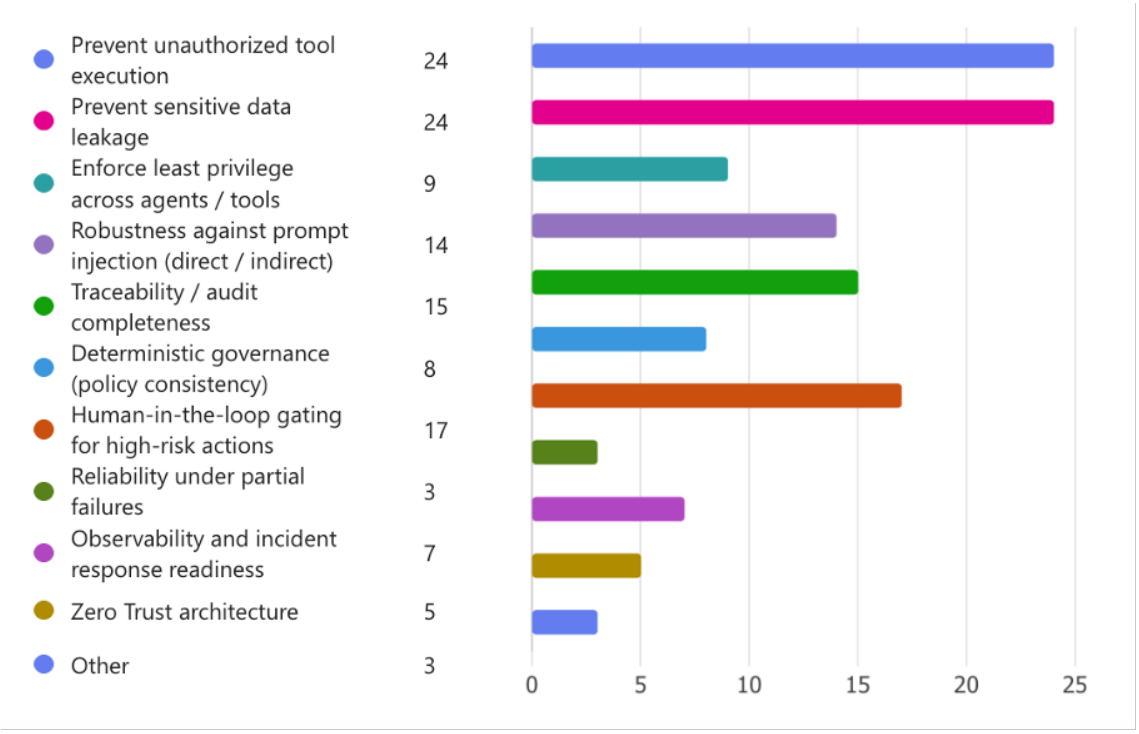


Figure 5.12: ASR for secure tool-using MAS

Threats and Trust Boundaries Figures 5.13 and 5.14 summarise the perceived threat landscape. Participants rated prompt injection, indirect prompt injection and tool misuse as the most critical threats, followed by privilege escalation, data exfiltration and cross-agent context poisoning.

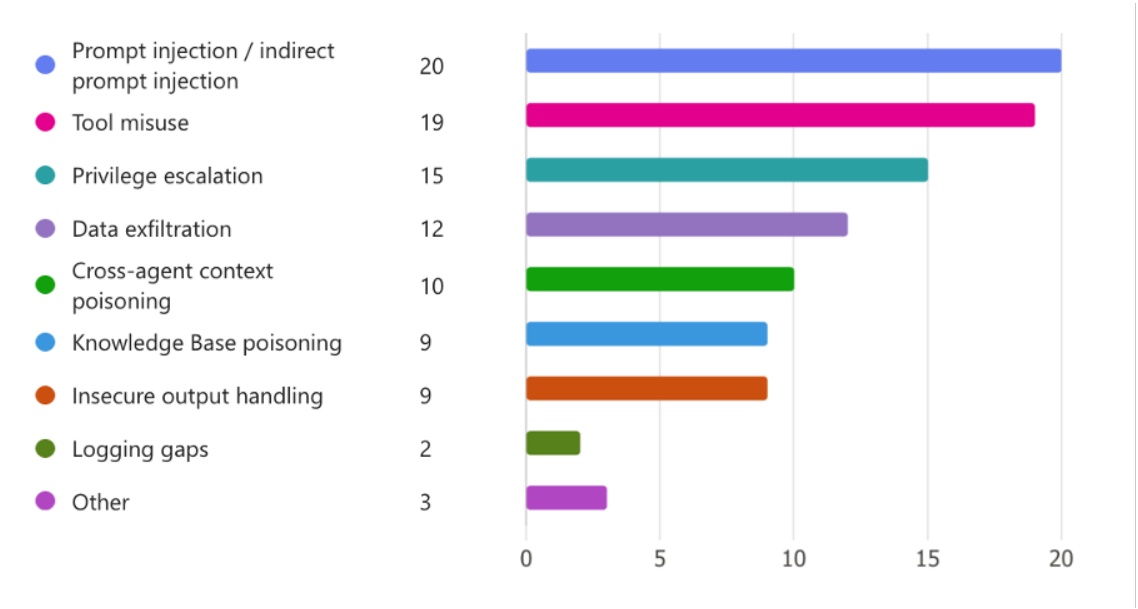


Figure 5.13: Critical threats for secure tool-using MAS

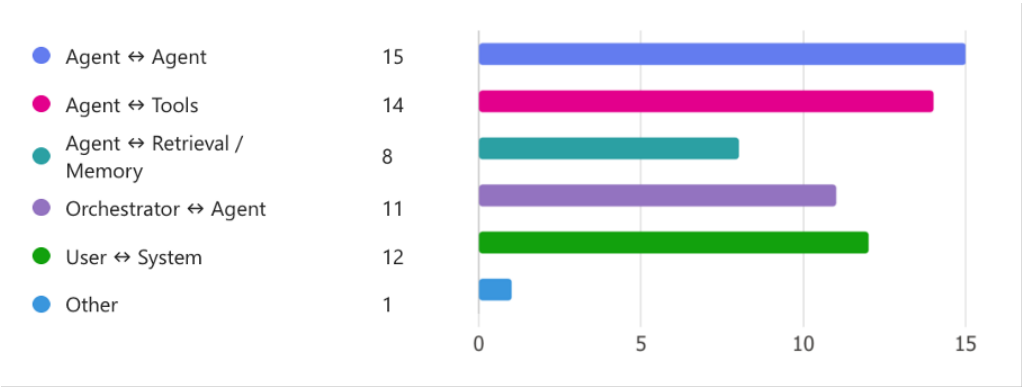


Figure 5.14: Relevant trust boundaries in secure tool-using MAS

The most relevant trust boundaries were identified between agents and tools, agents and other agents, users and the system, and agents and orchestration. These results indicate that security cannot be implemented only inside individual agents. It must also be enforced at system boundaries through authorization checks, validation, context isolation, least-privilege access, monitoring and audit logging.

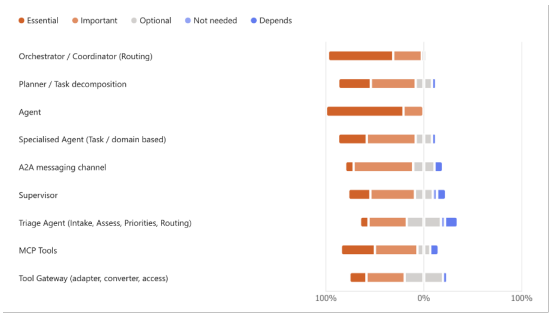
Architectural Layers and Components Figure 5.15 shows how participants classified potential layers for a secure RA. The orchestration and workflow layer received the strongest agreement, followed by layers related to IAM, compliance, agents, security controls, HITL, Knowledge, Governance and Operations. Remote, Interface and Data layer are seen more optional.



Figure 5.15: Expert classification of architectural layers for secure MAS

Participants generally considered the proposed layer set suitable, but suggested refinements such as runtime policy enforcement, a model gateway, evaluation and assurance, per-tool permission management, memory management, check pointing and sub-agent orchestration. A recurring concern was not the absence of layers, but their immature implementation in practice, especially for governance, compliance, security controls, observability and HITL.

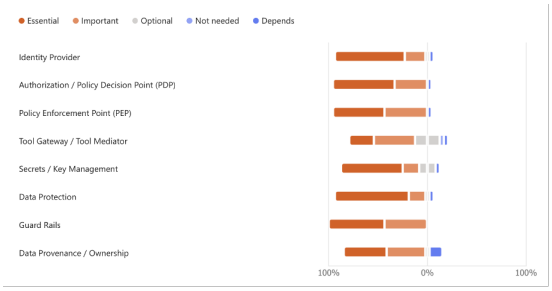
Figures 5.16a–5.16d classify logical components across orchestration, knowledge and state, security and governance, and evidence and operations.



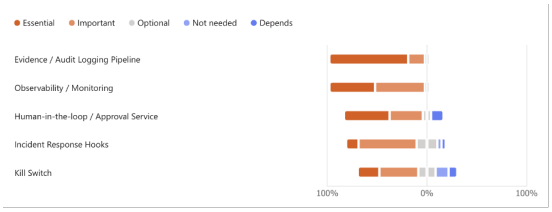
(a) Orchestration and workflow components



(b) Knowledge, storage and state components



(c) Security and governance components



(d) Evidence and operations components

Figure 5.16: Expert classification of logical components for a secure MAS

Across these component groups, the strongest recurring pattern was the need for clear orchestration, controlled knowledge access, explicit security enforcement and structured evidence generation. Orchestrators, agents, state stores, context builders, RAG services, IdPs, authorization components, PDPs, PEPs, Guardrails, audit logging pipelines and observability components were repeatedly rated as important.

More context-dependent or optional components included shared memory, conversation history, knowledge graphs, vector DBs, knowledge source, specialised agents, supervisor, triage agent, A2A channel, tool gateway, kill switches and advanced operational hooks.

Only few participants suggested additional components, which indicates that the proposed component list was broadly considered complete. The most explicit addition was an explainer component to support explainability.

Auditability, Evidence and Traceability Figures 5.17, 5.18 and 5.19 show a strong emphasis on auditability and end-to-end traceability.

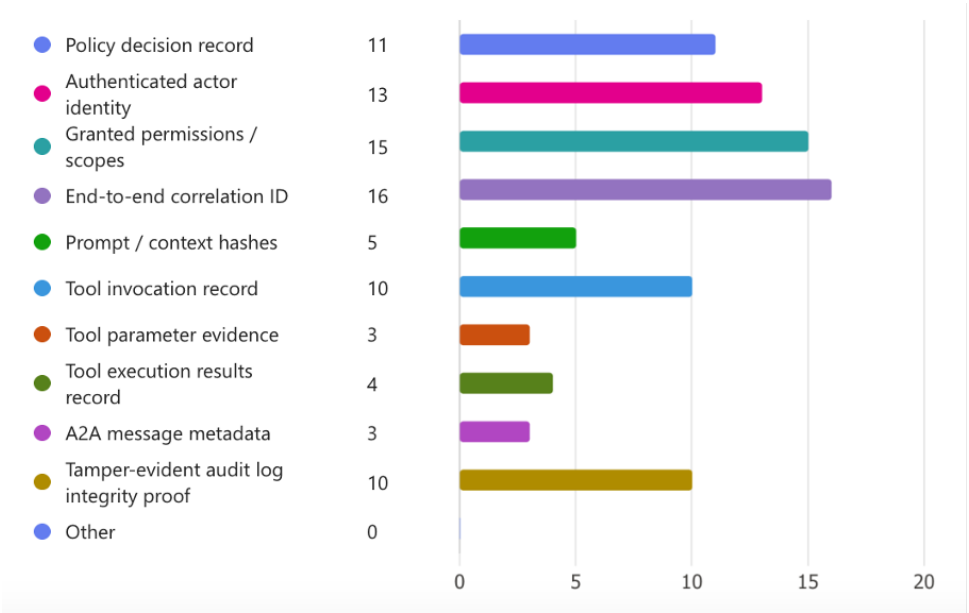


Figure 5.17: Evidence artifacts required for auditability in tool-using MAS

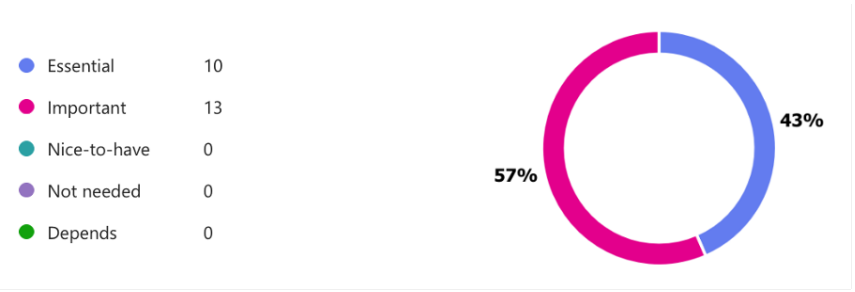


Figure 5.18: Importance of end-to-end traceability for secure tool-using MAS

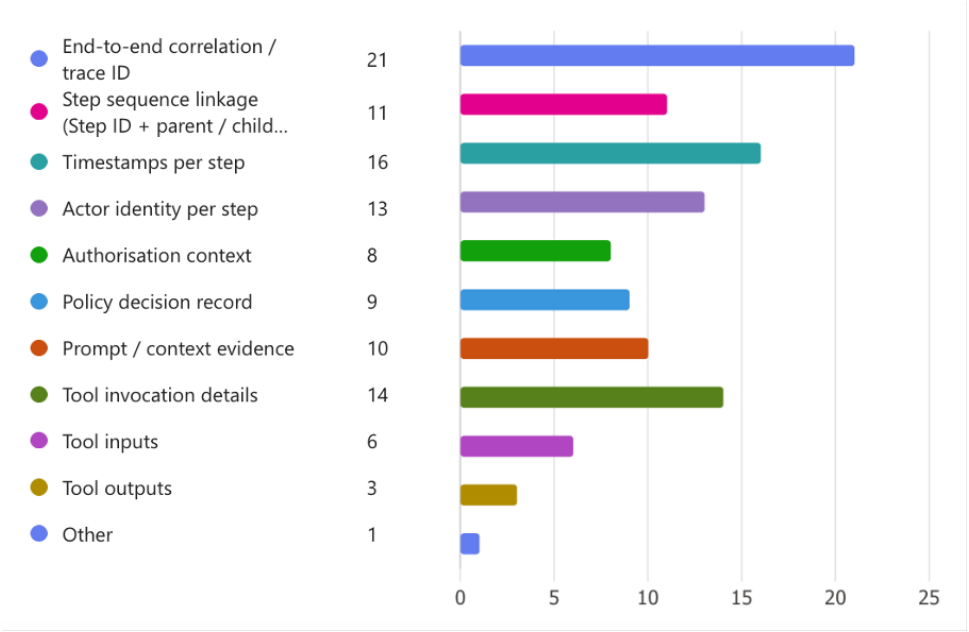


Figure 5.19: Information required for useful end-to-end traceability

Participants considered correlation IDs, granted permissions and scopes, authenticated actor identity, policy decision records, tool invocation records and tamper-evident audit log integrity proofs important for enforcement and accountability. The results indicate that auditability should not rely only on generic application logs. Instead, the architecture should produce structured evidence that links identities, permissions, policy decisions, tool calls, retrieved context, workflow steps and outputs. Several responses also emphasized the need to connect all steps through a common event or trace ID.

Architecture Communication and Reusability Figures 5.20 and 5.21 summarise how participants would communicate a secure RA. Functional decomposition, security boundaries, dataflows, deployment and operations were considered useful viewpoints. Component diagrams, dataflow diagrams, layer diagrams and trust-boundary diagrams were considered particularly relevant artifacts.

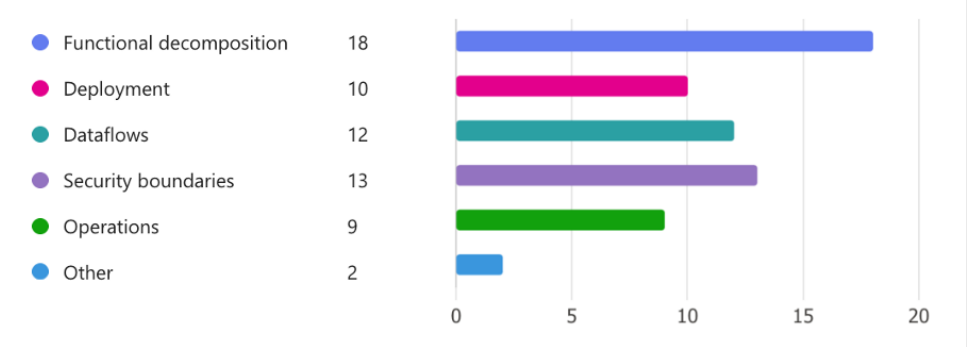


Figure 5.20: Useful viewpoints for communicating a secure RA

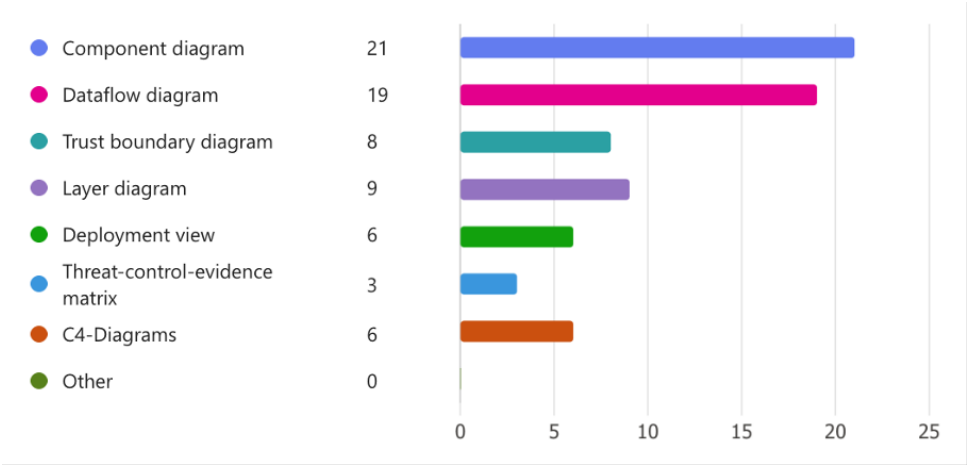


Figure 5.21: Useful architecture artifacts for communicating a secure RA

The findings suggest that a secure RA should be communicated through a small set of complementary views: a layer view to structure the architecture, a component view to show responsibilities, a dataflow view to explain interactions and a trust-boundary view to highlight security-relevant control points.

Participants also emphasized that reusability depends on the right level of abstraction. A RA should be specific enough to support real architectural decisions, but abstract enough to remain applicable across contexts and domains. Relevant factors in-

clude vendor-neutral patterns, standards and APIs, clear architecture principles, consistent viewpoints, defined layers, roles, trust boundaries and control points.

Limitations of the Questionnaire The questionnaire provides useful expert-informed input, but its limitations must be considered. The sample is not statistically representative, MAS-specific expertise varied and the final RA was not presented to participants for direct validation. The findings are therefore interpreted as design guidance and requirement input rather than as conclusive validation of the proposed architecture.

5.1.2 Reference Architecture for secure Multi-Agent Systems

Based on the literature review, the expert questionnaire and the implementation experience from the PoC, a layered RA for secure tool-using MAS was derived. The final architecture is shown in Figure 5.22. The detailed design is described in Section 4; therefore, this subsection summarises only the resulting architecture and its main implications. Compared with the proposed architecture in Figure 4.2, no structural changes were required.

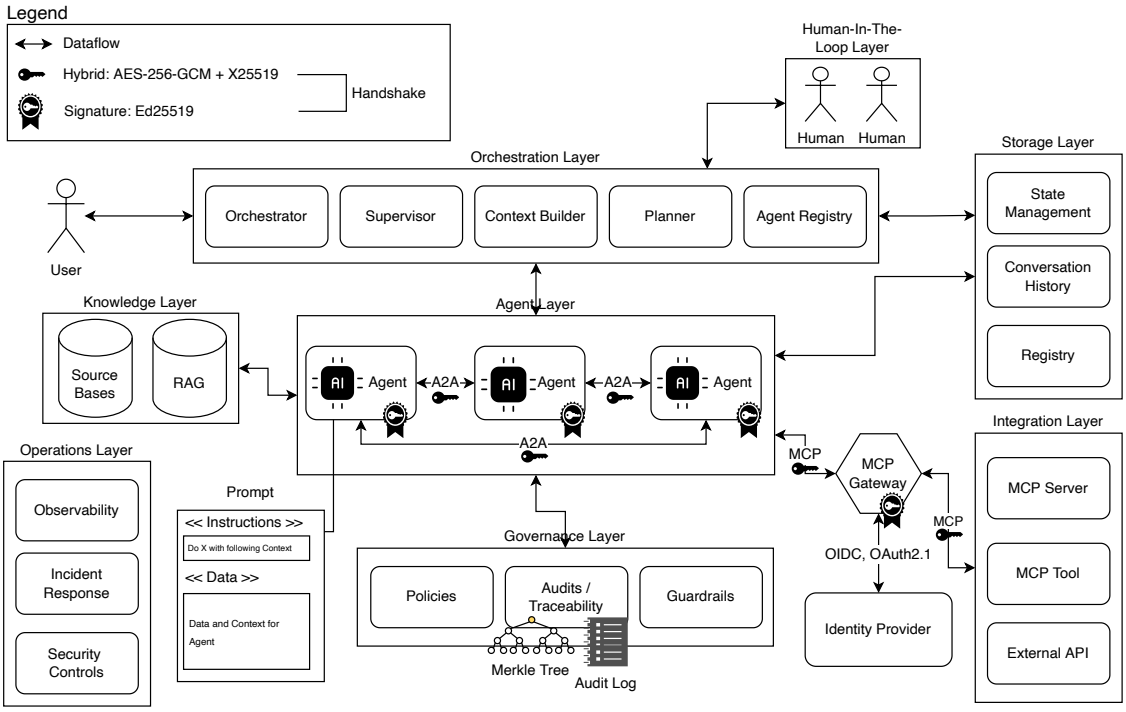


Figure 5.22: RA for secure MAS with Auditability

The resulting RA shows that secure enterprise MAS require more than agents, tools and prompts. Dynamic LLM-based agents are embedded into a broader enterprise structure with orchestration, knowledge access, integration, governance, storage, operations and HITL layers. Static enterprise components, such as DBs, policy documents, IdPs, tools and external APIs, are separated from dynamic agent behaviour but connected through controlled retrieval, context construction and mediated tool access.

The main result is a layered architecture in which security, governance and auditability are treated as core design concerns. Identity, authorization, scoped permissions, PDPs,

PEPs and the MCP gateway act as enforcement points for least-privilege tool use. Policies, Guardrails and HITL routing bound agent autonomy, while logging, traceability and Merkle-tree-based commitments support later reconstruction and tamper-evident verification of workflow evidence.

The final RA also reflects the priorities identified in the expert questionnaire. It makes trust boundaries, integration points, governance mechanisms and evidence flows explicit, especially between users, the system, agents and tools, agents and other agents, agents and retrieval systems, and the MCP gateway and external systems. The diagram combines the viewpoints that participants considered useful for communicating a RA, including functional decomposition, component structure, dataflows and security-relevant boundaries.

Overall, the result is a reusable and framework-independent RA for secure enterprise MAS. It can be adapted to different domains, tool ecosystems and risk levels without requiring every implementation to instantiate all layers in full. Optional cryptographic mechanisms, such as AES-GCM based encryption, X25519-style key exchange and Ed25519-style signatures, were not implemented in the claim-processing PoC, but indicate how secure A2A communication and verifiable identity binding could be strengthened in future implementations.

5.1.3 Insurance claim-processing PoC

The insurance claim-processing PoC was implemented as an end-to-end demonstrator of selected parts of the proposed RA. It combines a Flask-based WebApp, a Supabase DB, prepared insurance policy documents stored in OneDrive and a CrewAI workflow with specialised agents, task-level Guardrails and routing logic.

Figure 5.23 shows the implemented PoC architecture as a deployment-oriented view mapped to the proposed RA. The detailed design of the workflow, agents, routing paths and Guardrails is described in Section 4.3; therefore, this subsection focuses on the implemented result and the observed execution.

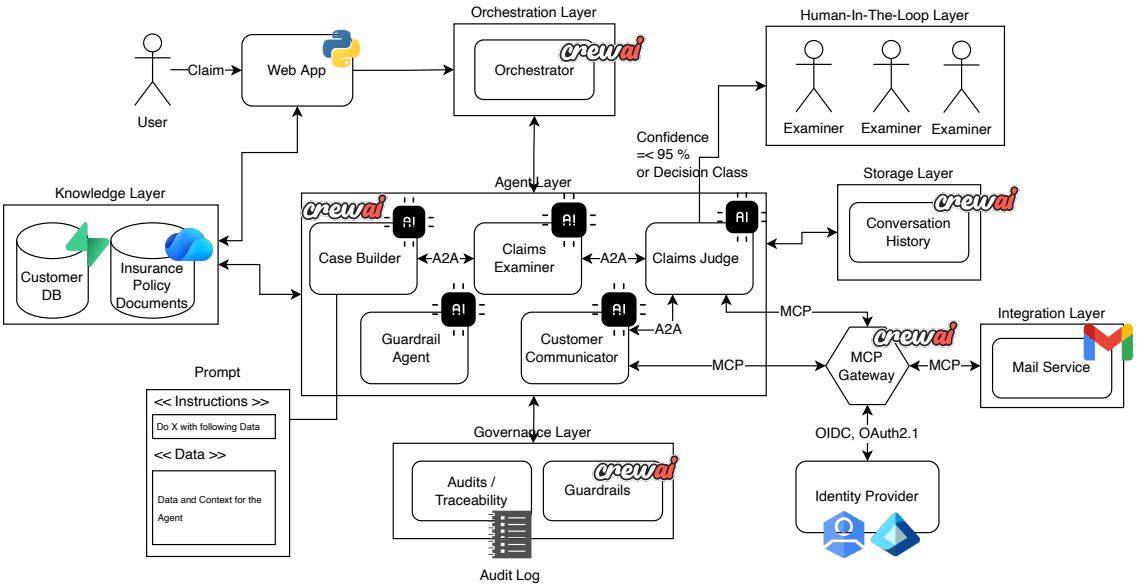


Figure 5.23: Implemented PoC architecture mapped to the proposed RA

Compared with the full RA in Figure 5.22, the PoC represents a smaller and feasibility-driven instantiation. The most relevant parts of the architecture were implemented: the

user-facing WebApp, DB-backed customer and policy lookup, document-based knowledge access, the agent workflow, routing logic, task-level Guardrails and selected tool integrations. This supports the interpretation of the RA as reusable guidance rather than as a mandatory implementation blueprint in which every component must be realized in full. A user-facing application is therefore implementation-specific rather than a mandatory component of the proposed RA; in the architecture, the relevant actor is the entity using the MAS, which may be a human user, another system or an organisational process.

Several RA elements were simplified, omitted or provided by CrewAI as platform functionality. CrewAI provided the orchestrator, A2A and MCP communication, agent execution environment, tool integration, conversation history, event and trace records, and Guardrail execution. The PoC used the platform-managed IdP and authentication flow for connected tools instead of a full enterprise IAM setup. A full vector DB, advanced RAG platform, production monitoring, incident response and live integration of the tamper-evident auditing artifact were outside the PoC scope. The Operations Layer was only partially represented through CrewAI task status, traces and execution metrics.

This distinction is important for interpreting the result. The PoC does not claim to implement the complete RA. Rather, it demonstrates that selected layers and controls of the RA can be instantiated in a working enterprise-oriented MAS. Future implementations may include more or fewer components depending on domain requirements, platform capabilities, risk level and compliance needs.

End-to-end Workflow Execution The end-to-end workflow was successfully demonstrated under PoC conditions. A claim could be submitted through the WebApp using either a self-written claim or a predefined test case. The WebApp retrieved the active customer policy from Supabase, generated a deterministic `case_id`, built the Representational State Transfer (REST) API payload and started the CrewAI workflow through the configured kickoff endpoint.

The submitted payload contained the claim case, customer information, preferred language, insurer, product, OneDrive file path and document hash. This allowed the workflow to start with structured input instead of requiring agents to search for the customer or policy independently. If no active policy was found, the WebApp stopped the process before calling CrewAI and returned an error message.

During execution, the Case Builder retrieved the relevant `document.txt` file from OneDrive and extracted policy passages. The Claims Examiner produced a structured coverage assessment based on the case file, and the Claims Judge validated the assessment, assigned a confidence score and selected the next workflow action. Depending on the case, the workflow routed execution to customer communication, customer information request or examiner revision. HITL Handler completed the process by summarising the workflow.

Overall, the PoC demonstrated the complete execution chain from WebApp submission to DB lookup, payload generation, CrewAI kickoff, document retrieval, evidence extraction, examination, validation, routing and process completion.

Test cases and observed Outcomes The PoC was evaluated with predefined claim cases covering clear, excluded, version-dependent, incomplete and ambiguous scenarios. The full claim texts are provided in Appendix B. Table 5.14 summarises the expected behaviour, observed workflow outcome and execution metrics.

The workflow processed all predefined cases and routed each case to a valid outcome. Case B produced the clearest expected result: the exclusion for damage caused by pets was identified and a customer rejection message was prepared. Other cases were routed to `CUSTOMER_INFO_REQUEST`, including cases that were expected to be covered. This indicates that the workflow behaved conservatively and preferred requesting additional information when the available facts were not sufficient for a confident final decision.

Two representative sent mails to customers were inspected in more detail. Case A produced an information request asking for additional documentation, while Case B produced a rejection message because the reported damage was attributed to the customer's own pet and was treated as excluded under the policy terms. Since the customer data were synthetic, the emails were sent to the author's address for testing purposes. The generated outputs were translated from German to English with DeepL for presentation in the thesis.

Output Case A: Additional information required for claim 755968cdff7101522a9fdb2d7d23b3e6

Dear Mrs. Fischer,
Thank you for your claim notification regarding your Allianz Household Contents Comfort Insurance.
The processing of your water damage claim from 12 January 2026 is currently still under review. In order to make a final assessment of your claim and calculate a possible payout, we kindly ask you to provide the following documents:

- A detailed list, signed by you, of all damaged or destroyed items, including the respective purchase date and purchase price
- Any available photographs of the damaged items and the location of the damage
- If available, purchase receipts or invoices for the affected items
- A brief statement indicating whether individual items can still be repaired or have already been disposed of / had to be disposed of
- Optional: Evidence of costs already incurred for protection, repair, or disposal measures, such as invoices or cost estimates

These documents are required for the final benefit assessment of your claim and for the payment of any possible compensation. As soon as we have received the required documents, we will continue processing your request immediately and inform you about the next steps.
Thank you for your cooperation. Please do not hesitate to contact us if you have any questions.
Kind regards,
Allianz Customer Service

Note for testing purposes: This message was originally intended for `sofia.fischer@outlook.com`, but as a test run it was sent to the developer address.

Output Case B: Regarding your claim 1fa12b0b7c2fc00b07c119757c9d7107

Dear Mr. Meyer,
We have carefully reviewed your claim under your Household Contents Comfort Insurance.
After a thorough assessment, we regret to inform you that there is no insurance coverage for the damage to the sofa, cushions, and wooden baseboard. The cause of the damage is directly attributable to the behaviour of your own pet, namely your dog. The insurance terms explicitly state that damage caused by pets is excluded from coverage. We kindly ask for your understanding that we are therefore unable to provide any benefit under your household contents insurance.

Estimated Compensation:
Unfortunately, no compensation is possible, as the claimed damages are contractually excluded.

Please do not hesitate to contact us if you have any further questions.
Kind regards,
Allianz Customer Service

P.S.: The actual recipient address for this message is `noah.meyer@yahoo.com`. However, since this is a PoC and customer email addresses are not active, this message is instead sent to the developer email address.

Table 5.14: Overview of claim-processing PoC test cases, observed outcomes and execution metrics

Case	Scenario	Expected behaviour	Observed outcome	Time	Tokens	Requests	Cost
A	Water damage in the kitchen caused by a loosened dishwasher hose; damaged furniture, flooring, carpet, chair legs and books.	Covered, Customer Communication	More information needed; routed to `CUSTOMER_INFO_REQUEST`.	344.61s	135,811	29	$0.3823
B	Damage caused directly by the customer's own dog to sofa, cushions and wooden skirting board.	Not covered, Customer Communication	Exclusion identified; customer communication prepared.	233.43s	268,597	39	$0.6256
C1	Scorch damage on sofa caused by a glowing particle; policy comparison scenario with the policy version expected not to cover.	Not covered, Customer Communication	Information request according to missing information; routed to `CUSTOMER_INFO_REQUEST`.	332.63s	154,426	36	$0.4043
C2	Scorch damage on sofa caused by a glowing particle; policy comparison scenario with the policy version expected to cover.	Covered, Customer Communication	Information request according to missing information; routed to `CUSTOMER_INFO_REQUEST`.	343.43s	152,290	39	$0.4256
D1	Ambiguous and short water damage claim under a washbasin with little detail about cause, timing and circumstances.	Incomplete case, Customer Information Request	Information request according to missing information; routed to `CUSTOMER_INFO_REQUEST`.	428.39s	182,352	37	$0.5107
D2	Detailed water damage claim under a washbasin, described as sudden siphon leakage in the insured apartment.	Covered, Customer Communication	Information request according to missing information; routed to `CUSTOMER_INFO_REQUEST`.	268.05s	158,321	31	$0.4228
E	Moisture damage after severe weather; unclear whether water entered through terrace door, internal drain or another cause; missing items also reported.	Incomplete case, Customer Information Request	Information request according to missing information; routed to `CUSTOMER_INFO_REQUEST`.	418.33s	185,666	31	$0.5114
F	Moisture damage near balcony door after heavy rain; unclear whether water entered through the door, a drain or another building-related source.	Incomplete Case, Customer Information Request	Information request according to missing information; routed to `CUSTOMER_INFO_REQUEST`.	445.55s	222,280	42	$0.6375
G	Damage during moving; unclear insured location because items were present in both old and new apartment; rain entered through open or tilted kitchen window.	Incomplete case, Customer Information Request	Information request according to missing information; routed to `CUSTOMER_INFO_REQUEST`.	337.57s	165,908	35	$0.4410
H	Missing e-bike battery and helmet from shared basement; unclear access situation and uncertain last location of the battery.	Incomplete case, Customer Information Request	Information request according to missing information; routed to `CUSTOMER_INFO_REQUEST`.	326.61s	169,424	34	$0.4494
I	Missing watch and jewellery box from bedroom; unclear if theft, misplacement or another cause; no visible break-in traces.	Ambiguous, Human Review	Information request according to missing information; routed to `CUSTOMER_INFO_REQUEST`.	396.75s	196,255	34	$0.5454
J	Water under washbasin with damaged cabinet and slightly wet laptop backpack; unclear whether pipe leakage or cleaning water caused the damage.	Ambiguous, Human Review	Information request according to missing information; routed to `CUSTOMER_INFO_REQUEST`.	278.73s	174,887	37	$0.4688

Table 5.15: Aggregated execution metrics for the predefined claim cases

Metric	Observed value
Number of evaluated scenarios	12
Total execution time	4,154.08 seconds
Average execution time per scenario	346.17 seconds
Total token usage	2,166,217 tokens
Average token usage per scenario	180,518 tokens
Total requests	424
Average requests per scenario	35.33
Total estimated cost	$5.8248
Average estimated cost per scenario	$0.4854
Human review triggered	0 cases

In addition to the per-case metrics, the CrewAI dashboard reported 295 completed tasks out of 302 task runs, corresponding to a success rate of 97.68% and an error rate of 2.32%. It also reported 3.52 tool calls per task run, no tool errors, no delegations and a cache hit rate of 0%. These values provide an operational overview of the workflow execution during the PoC evaluation.

5.1.4 Auditing artifact results

The auditing artifact was implemented to demonstrate tamper-evident traceability for agentic workflow executions. The design is described in Section 4.4; this subsection reports the generated files, Merkle commitment, blockchain-style commitment and tamper-detection outcome.

The artifact produced five main files:

- `audit_log_hashed.jsonl`: structured audit records with record-level hashes
- `audit_merkle_commitment.json`: Merkle tree commitment for the audit log
- `audit_blockchain.jsonl`: blockchain-style chain of audit blocks
- `audit_log_malicious.jsonl`: deliberately manipulated audit log for tamper testing
- `audit_verification_report.json`: verification report for the original and malicious logs

The core result is that changes to audited event records were detectable. A modified record changed its recomputed record hash. Since record hashes are used as Merkle leaves, the change propagated to the Merkle root. Since the Merkle root was committed into a chained audit block, the manipulated JSONL file no longer verified against the stored commitment.

Evidence Chain Figure 5.24 summarises the evidence chain implemented by the artifact. It links individual workflow events to record hashes, Merkle leaves, a Merkle root, an audit block and a final block hash. This creates a verifiable path from fine-grained workflow records to a compact integrity commitment.

Merkle commitment Result The audit log contained 78 structured workflow records. Each record was represented by a SHA-256 hash and inserted as one Merkle leaf. The generated Merkle commitment is summarised in Table 5.16.

Table 5.16: Generated Merkle commitment for the audit log

Attribute	Observed result
Workflow ID	`wf-001`
Source file	`audit_logs/audit_log_hashed.jsonl`
Hash algorithm	`SHA-256`
Leaf count	78
Tree depth	8
Leaf order	`jsonl_line_order`
Odd leaf strategy	`duplicate_last_leaf`
Merkle root	`3d0f30145b***4724d6d0c416`

The full Merkle trees for the original and manipulated audit logs are provided in Appendix E as Figures E.17 and E.18. Because the full trees are large, Figures 5.25 and 5.27 show truncated views of levels 3–7. These higher levels illustrate how record hashes are paired and propagated upward toward the Merkle root.

In the full Merkle tree, level 0 represents the record hashes from the JSONL audit log. Adjacent nodes are paired, concatenated and hashed to create their parent node. This process repeats until the Merkle root is produced. If a level contains an uneven number of nodes, the last node is duplicated and paired with itself. This duplicate node is not an additional audit record, but a structural copy used to keep the binary tree construction consistent.

Blockchain-style commitment Result The generated Merkle root was committed into a blockchain-style audit block. Each block contains the Merkle root, workflow metadata, a timestamp, the hash of the previous block and its own block hash. Figure 5.26 shows this structure.

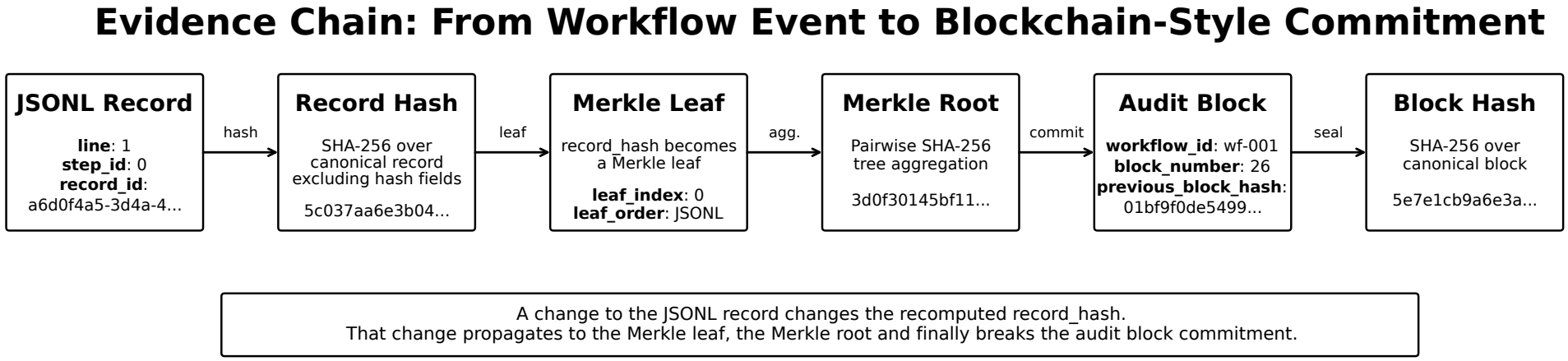


Figure 5.24: Evidence chain of the auditing artifact

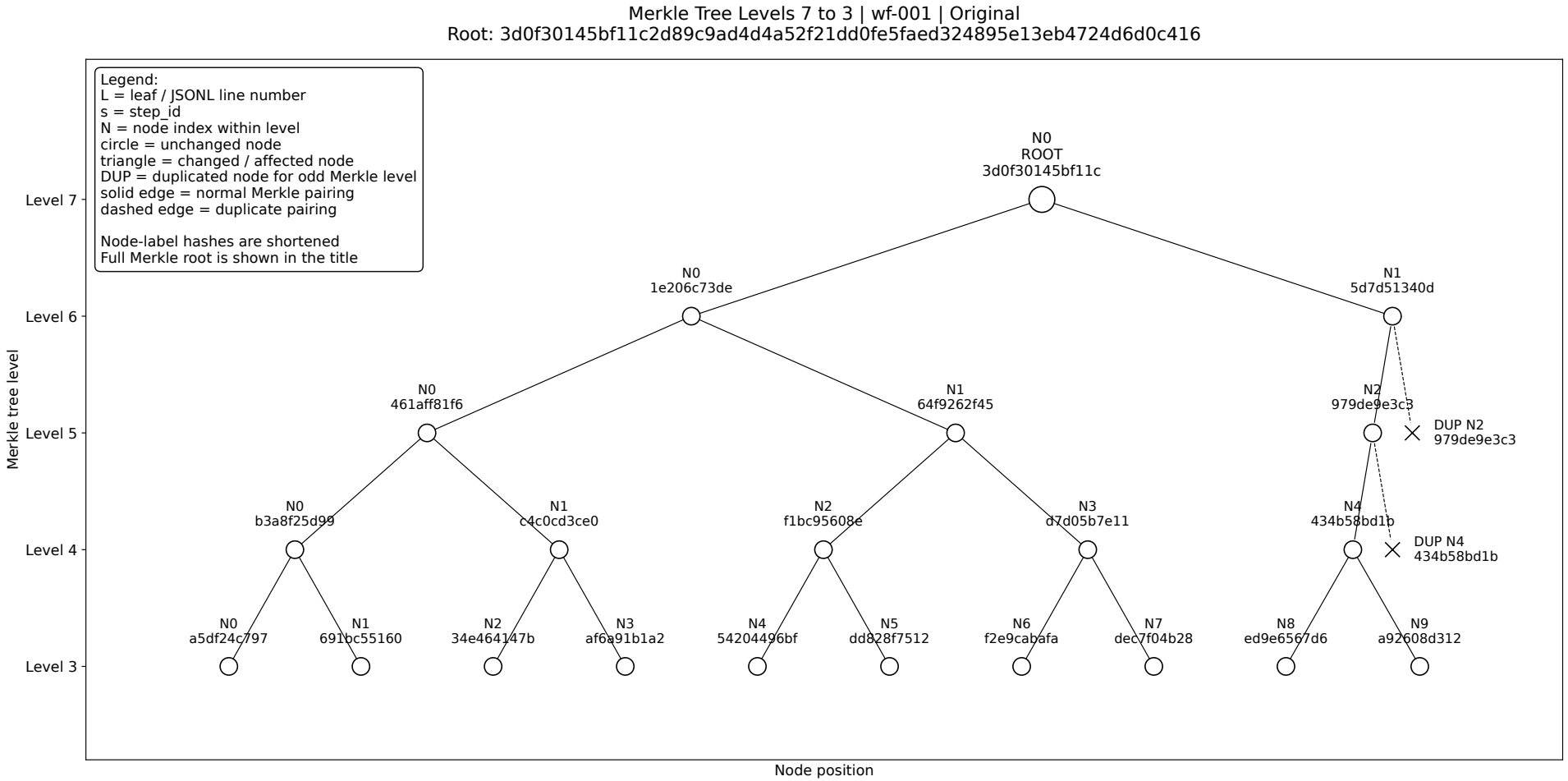


Figure 5.25: Truncated Merkle tree view for the original audit log, showing levels 3–7

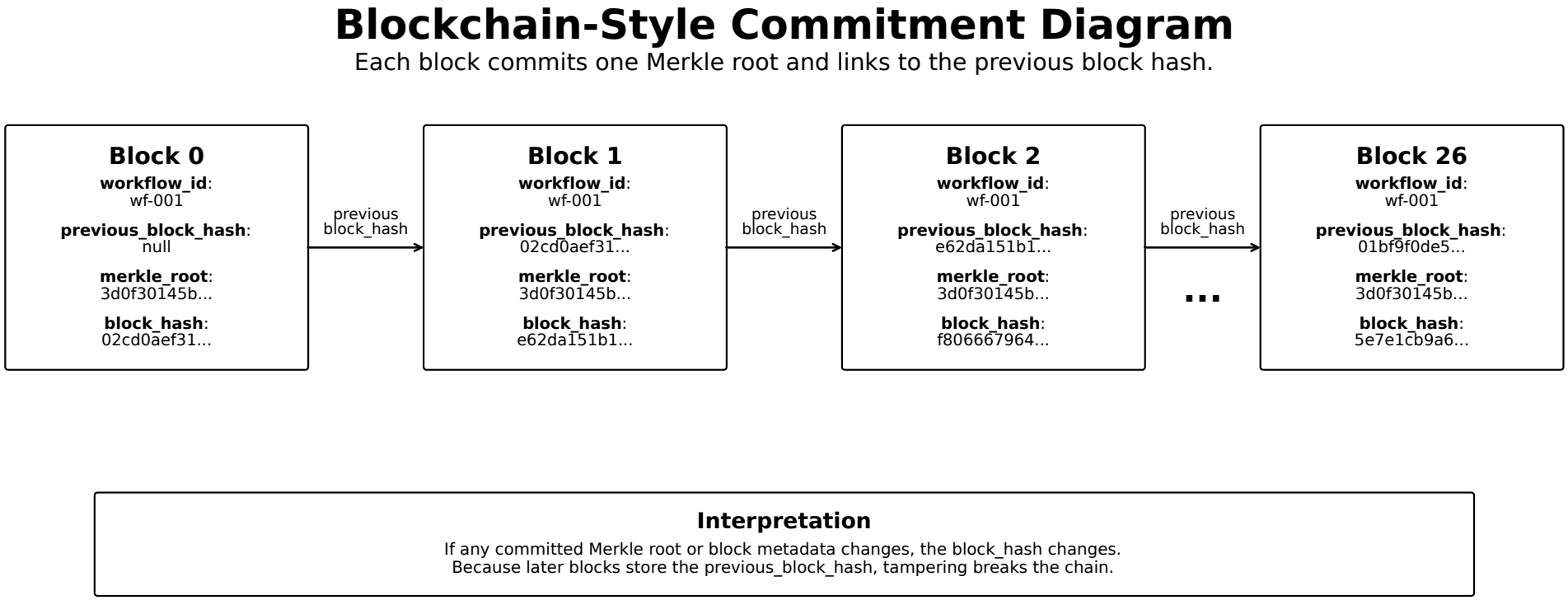


Figure 5.26: Blockchain-style audit commitment structure

The verification run checked block numbering, previous-block references, block hash correctness and whether the latest block committed the expected Merkle root. The successful verification output was:

```
Record hashes verified: 78
Merkle root verified: 3d0f30145b***4724d6d0c416
Blockchain verified. Blocks checked: 21
Latest block hash: 0f2957556***1a76a48b102

AUDIT CHAIN VERIFIED SUCCESSFULLY
```

This confirms that the original audit log, Merkle commitment and blockchain-style chain were mutually consistent.

Tamper detection Result To test tamper detection, a malicious copy of the original audit log was created. Three records were modified after the original hashes and Merkle commitment had already been generated: node index 0 changed `record_id`, node index 5 changed `task_status` and node index 20 changed `guardrail_status`.

Figure 5.27 shows the truncated Merkle tree for the manipulated audit log. The legend distinguishes normal nodes, duplicated nodes and nodes affected by tampering. The highlighted path shows how a modified audit record changes its recomputed hash, then changes the affected parent nodes and finally contributes to a changed Merkle root.

Figure 5.28 shows the same propagation effect in simplified form. A changed JSONL record produces a changed recomputed record hash. This changes the Merkle leaf, the parent path and finally the Merkle root. Since the changed root does not match the committed root in the audit block, verification fails.

The verification report confirmed this behaviour. The original chain remained valid, while the malicious log failed verification. A shortened excerpt of the generated report is shown below.


```
{
  "original_chain": {
    "verified": true,
    "record_count": 78,
    "record_hashes_verified": true,
    "merkle_root_verified": true,
    "blockchain_verified": true
  },
  "malicious_chain": {
    "verified": false,
    "tampering_detected": true,
    "record_hashes_verified": false,
    "merkle_root_verified": false,
    "committed_merkle_root": "3d0f30145b***4724d6d0c416",
    "recomputed_malicious_merkle_root": "bd7bf191***1b58b1df4"
  }
}
```

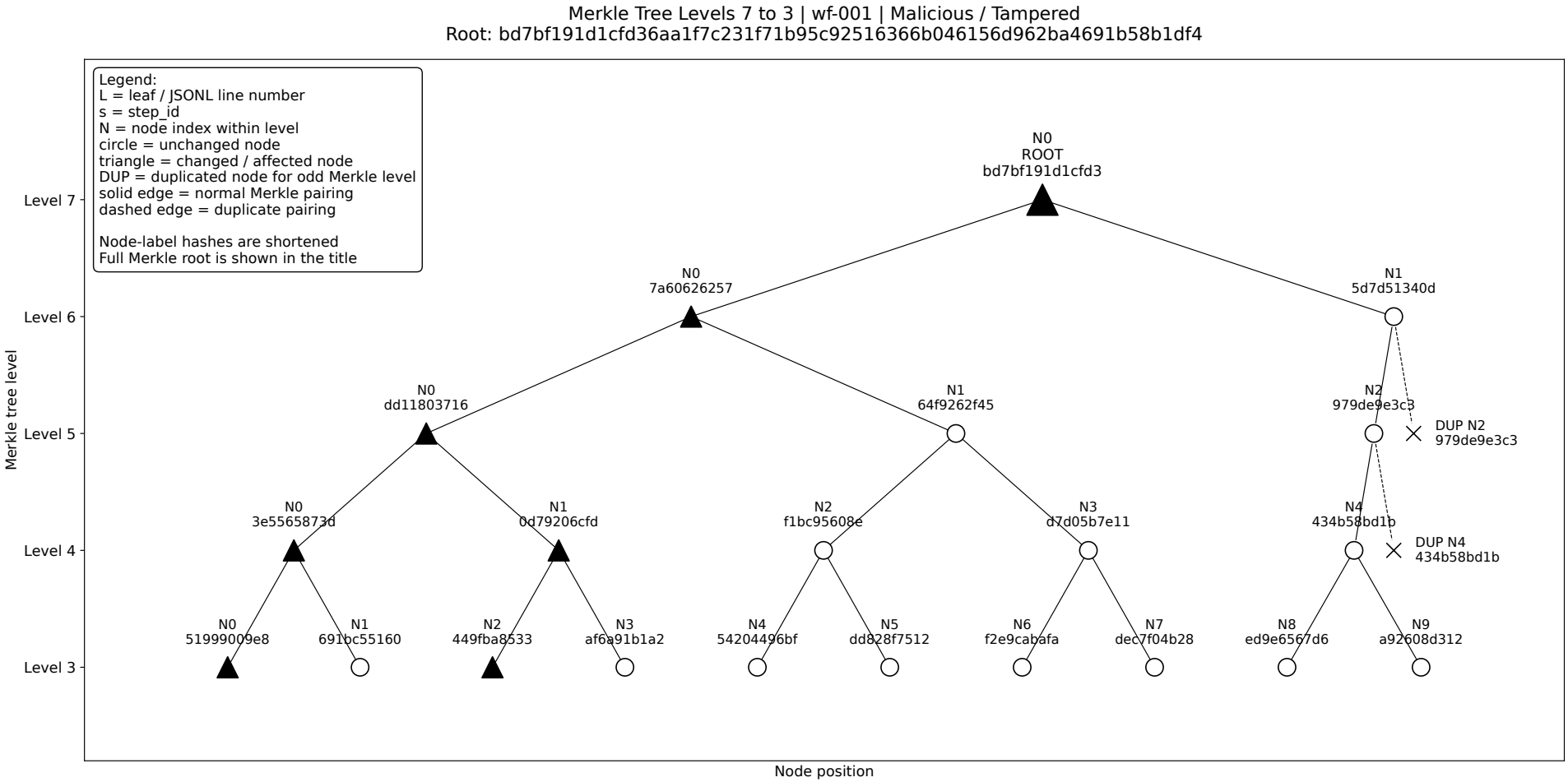


Figure 5.27: Truncated Merkle tree view for the malicious audit log, showing levels 3–7

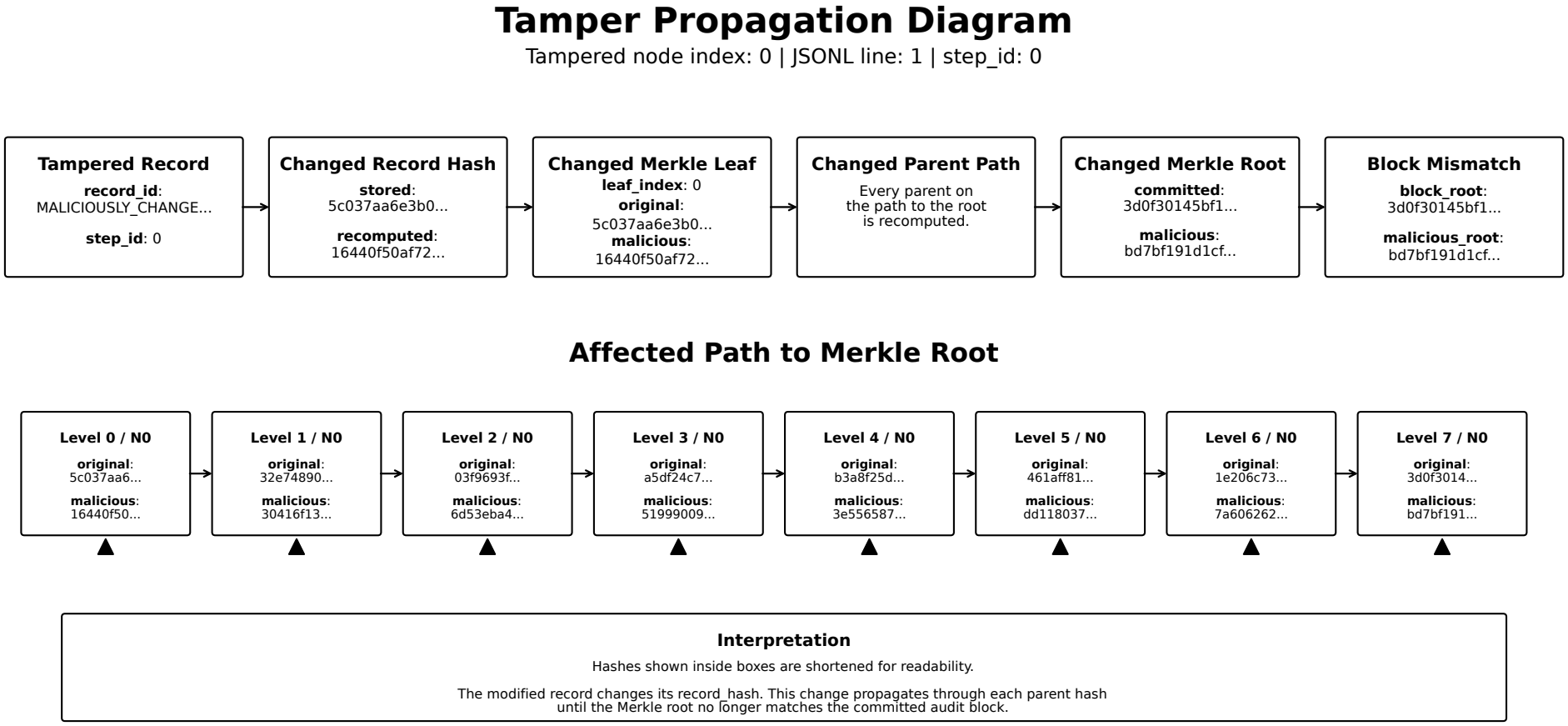


Figure 5.28: Propagation effect of a tampered log node

The result is summarised in Table 5.17. The record count and leaf count still match because no record was removed or added. However, record-hash verification, Merkle root verification and blockchain verification fail for the malicious log.

Table 5.17: Verification result for original and malicious audit logs

Verification criterion	**Original log**	**Malicious log**
Record count	78	78
Record hashes verified	true	false
Merkle root verified	true	false
Leaf count verified	true	true
Blockchain verified	true	false
Overall verification	true	false
Tampering detected	false	true

The committed Merkle root of the original audit log was:

```
3d0f30145bf11c2d89c9ad4d4a52f21dd0fe5faed324895e13eb4724d6d0c416
```

The recomputed Merkle root of the malicious audit log was:

```
bd7bf191d1cfd36aa1f7c231f71b95c92516366b046156d962ba4691b58b1df4
```

The mismatch shows that the malicious log no longer corresponds to the original Merkle commitment. The verification report also identified the affected records directly. For example, the first manipulated record was detected because the stored hash no longer matched the recomputed hash:

```
{
  "line_number": 1,
  "step_id": 0,
  "record_id": "MALICIOUSLY_CHANGED_RECORD_ID",
  "stored_hash": "5c037aa6e***39dfe0d18ce",
  "recomputed_hash": "16440f50af7***59597b4e0b",
  "reason": "record_hash_mismatch"
}
```

This demonstrates tamper evidence at three levels: record-level hash verification, Merkle root verification and blockchain-style chain verification. It also shows that modified records and small changes can be traced back to specific line numbers, step identifiers and record identifiers.

Figure 5.29 presents the verification result visually. The original audit chain passes record-hash verification, Merkle root verification, leaf-count verification and blockchain verification. The malicious audit chain fails record-hash verification, Merkle root verification and blockchain verification, while the leaf count still passes because no record was removed or added.

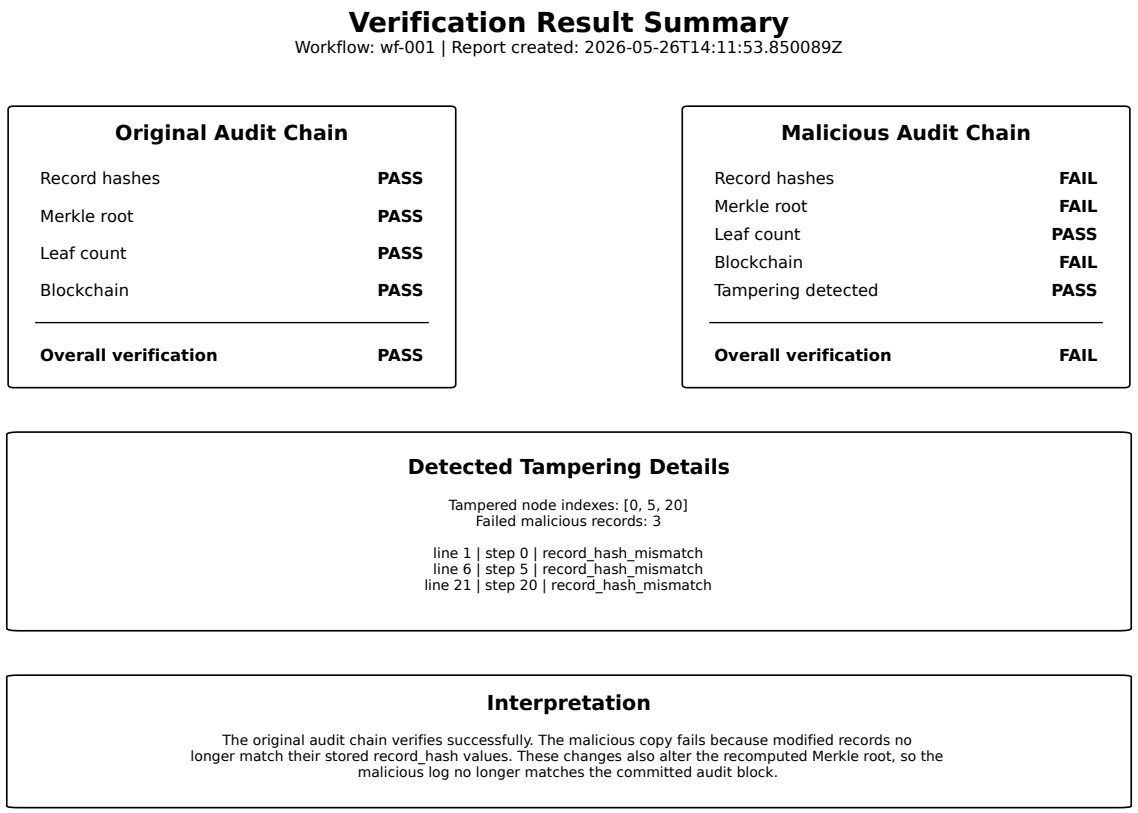


Figure 5.29: Result for audit verification

Result Summary The auditing artifact produced three main results. First, it demonstrated that agent workflow events can be represented as structured audit records. Second, it showed that these records can be transformed into a compact Merkle commitment and anchored in a blockchain-style hash chain. This supports scalable and efficient integrity verification, especially when intermediate nodes or proofs are preserved for partial verification. Third, it demonstrated that manipulated audit logs can be detected and traced back to specific records.

The result supports the auditability layer of the proposed RA. It shows that audit evidence for tool-using MAS should not only be collected, but also integrity-protected. Although the artifact uses exported workflow records instead of fully dynamic runtime logging, the verification principle is transferable to real systems in which agent tasks, tool invocations, Guardrail results, routing decisions and workflow transitions generate structured audit events automatically.

5.2 Analysis

This subsection interprets the results presented above. It identifies recurring patterns, assesses their significance and clarifies what the results show before the broader discussion in Section 6.

5.2.1 Cross-result Patterns and Significance

Across the questionnaire, the proposed RA, the PoC and the auditing artifact, one pattern is consistent: secure MAS must be designed as controlled enterprise workflows rather than as collections of agents, prompts and tools. The questionnaire emphasized secu-

rity, compliance, auditability, governance, traceability and controlled tool access. The proposed RA translates these priorities into identity, authorization, mediated tool access, Guardrails, audit logging and HITL mechanisms around the dynamic agent layer.

A second pattern is that auditability must be treated as an architectural output. Participants highlighted the need for correlation identifiers, actor identity, permissions, policy decisions, tool invocation records and tamper-evident evidence. The auditing artifact gives this requirement a concrete form by linking structured workflow records to record hashes, Merkle roots and blockchain-style commitments. The significance is that audit evidence should be generated and protected intentionally, not reconstructed later from generic application logs.

A third pattern is the separation between static enterprise sources and dynamic agent behaviour. In the PoC, customer records, policy metadata and policy documents acted as sources of truth, while agents prepared, examined, validated and communicated the case. This separation made the workflow more traceable and supports the architectural distinction between governed enterprise systems and probabilistic agentic reasoning components.

5.2.2 Interpretation of the PoC

The claim-processing PoC showed that selected parts of the proposed RA can be instantiated in a realistic but controlled insurance workflow. The system connected a Flask WebApp, Supabase data, prepared policy documents, OneDrive-based document access and a CrewAI workflow through structured payloads, document references, case files and routed agent tasks. This supports the idea that static enterprise systems can be connected with dynamic agent workflows in a traceable and controlled way.

The PoC also showed the value of evidence grounding and policy-version traceability. Each submitted claim could be linked to a customer, an active policy, policy metadata and a corresponding document source. Although the policy-version and detail comparison case did not lead to different final outcomes, the workflow demonstrated the mechanism needed to select different policy documents for different customers and preserve this metadata during execution.

At the same time, the workflow behaved conservatively. Most predefined cases were routed to `CUSTOMER_INFO_REQUEST`, including some cases where coverage was expected. This behaviour is safer than unsupported approval or rejection, but it also shows that prompts, evidence extraction, confidence thresholds and routing rules were not fully calibrated. The `HUMAN_REVIEW` path was not triggered in the predefined test set, so this branch remains less strongly evaluated than the customer information request path.

The execution metrics further indicate feasibility, but not production efficiency. The workflow required several minutes per case, substantial token usage and resulted in outcomes that differed from the expected final decisions. This is acceptable for a thesis demonstrator, but a production implementation would require more fine-tuning, such as selective retrieval, clearer task boundaries, caching, prompt optimization and better-calibrated routing.

5.2.3 Interpretation of the Auditing Artifact

The auditing artifact provided the clearest technical demonstration of tamper-evident traceability. It showed that workflow events can be represented as structured JSONL

records, protected through record-level hashes, aggregated into a Merkle tree and committed into a blockchain-style hash chain.

The original audit chain verified successfully, while the malicious audit log failed verification after selected records were modified. The changed records caused record-hash mismatches, which propagated to a different recomputed Merkle root. Since this root no longer matched the committed root, the malicious chain failed verification.

This result shows that manipulation of audit records after commitment can be detected and traced back to concrete line numbers, step identifiers and record identifiers efficiently. Therefore, the artifact supports the thesis argument that audit evidence for tool-using MAS should not only be collected, but also integrity-protected.

However, the artifact verifies integrity rather than correctness. It can show that a record was changed after hashing, but it cannot determine whether the original agent decision was correct, fair, legally valid or based on the right policy interpretation. Tamper-evident logging must therefore be combined with evidence grounding, validation, policy checks and human review.

5.2.4 Synthesis against the Research Questions

The results support the RQs at the level of architectural design and controlled feasibility demonstration. For RQ1, the questionnaire and RA identify the main secure-by-design requirements: identity, authorization, least privilege, controlled tool access, trust boundaries, Guardrails, traceability, audit evidence and HITL. For RQ2, the PoC shows one practical way to connect static enterprise systems, such as DBs, policy documents and document metadata, with dynamic agent workflows. For RQ3, the auditing artifact demonstrates how Merkle tree commitments and blockchain-style root chaining can support tamper-evident auditability. For RQ4, the questionnaire shows that experts consider security, governance, compliance, traceability and auditability central to enterprise adoption of tool-using MAS.

The main RQ is therefore answered at the design and feasibility level: a secure and compliant RA for tool-using enterprise MAS can be structured as a layered architecture with explicit trust boundaries, governed tool access, structured audit evidence and human oversight. However, the results do not prove production readiness. The PoC used synthetic data, predefined cases, simplified retrieval and framework-managed execution, while the auditing artifact processed exported records separately from the live workflow.

Overall, the results show that the proposed architecture is feasible as a design blueprint and that selected mechanisms can be implemented in controlled artifacts. The broader implications for secure enterprise design, generalisability, related work and threats to validity are discussed in Section 6.

5.3 Summary of Findings

The results and analysis lead to four main findings. First, secure tool-using MAS require more than agent orchestration. They need an architectural structure that explicitly includes identity, authorization, least privilege, controlled tool access, governance, auditability, traceability and human oversight.

Second, the expert questionnaire confirmed the practical relevance of these concerns. Participants emphasized security, compliance, auditability, reliability, governance and or-

ganisational readiness as central adoption factors. The most important requirements concerned preventing unauthorised tool execution, preventing sensitive data leakage, supporting end-to-end traceability, enabling audit completeness and applying HITL controls for high-risk actions.

Third, the proposed RA provides a reusable structure for addressing these concerns. It separates static enterprise components, such as DBs, documents, APIs, IdPs and storage systems, from dynamic agent-based components such as orchestrators, agents, planners, supervisors and tool-using workflows. This separation supports controlled integration between enterprise systems and probabilistic agent behaviour.

Fourth, the implemented artifacts demonstrated feasibility under controlled conditions. The claim-processing PoC showed that selected parts of the RA can be instantiated in an insurance workflow using structured customer data, policy documents, specialised agents, routing logic and task-level Guardrails. The workflow processed all predefined cases and routed them to valid outcomes, although it behaved conservatively by often requesting additional customer information.

Finally, the auditing artifact showed that workflow events can be transformed into tamper-evident audit evidence. Structured JSONL records were hashed, aggregated into a Merkle tree, committed into a blockchain-style hash chain and verified through a dedicated report. The malicious-log scenario demonstrated that modified records caused hash mismatches, changed the recomputed Merkle root and failed verification against the stored commitment.

Overall, the findings support the thesis argument that secure enterprise MAS should be designed around explicit trust boundaries, governed tool access, structured audit evidence and human oversight. At the same time, the results remain limited by the controlled PoC scope, synthetic data, simplified retrieval, conservative routing, separate audit artifact and absence of production-grade enforcement mechanisms. These limitations are discussed further in Section 6.

6 Discussion

This section reflects on the findings of the thesis and assesses how far the research problem has been addressed. This thesis started from the problem that enterprises lack reusable architectural guidance for designing secure, traceable and compliant tool-using MAS. This problem was investigated through literature-based architecture design, an expert questionnaire, an insurance claim-processing PoC and a separate auditing artifact.

The results support the initial assumption that secure enterprise MAS cannot be designed by focusing only on agents, prompts or orchestration frameworks. Instead, the findings show that security, governance, auditability, identity, authorization, evidence handling and human oversight must be treated as architectural concerns. This is reflected in the proposed RA, the PoC and the auditing artifact.

The discussion first interprets the significance of the main findings. It then assesses the objectives and RQs, compares the findings with related work, discusses implications for enterprise and regulated-domain design and finally summarises the main limitations, generalisability and threats to validity.

6.1 Interpretation and Significance of the Findings

The central finding is that tool-using MAS should be understood as enterprise systems rather than as isolated agent workflows. In such systems, agents may access documents, retrieve customer data, call tools, exchange intermediate outputs and influence decisions. This means that their behaviour must be constrained and made accountable through controls at the architectural level.

The proposed RA addresses this problem by embedding dynamic LLM-based agents into a broader architecture with orchestration, knowledge access, integration, identity, governance, logging, operations and HITL layers. Its main strength is the explicit separation between static enterprise components and dynamic agent components. Static components, such as DBs, policy documents, IdPs, storage systems and APIs, provide authoritative data and stable control points. Dynamic components, such as agents, planners, supervisors and context builders, perform reasoning and task execution but should do so only through controlled interfaces.

This separation is significant because it makes the design space more manageable. Instead of assuming that an agent framework can solve all enterprise concerns, the RA shows where access control, trust boundaries, policy checks, audit evidence and human review should be placed. The expert questionnaire supports this direction: participants emphasized security, compliance, auditability, governance, traceability and reliability more strongly than performance or cost. This suggests that enterprise adoption depends not only on whether agents can complete tasks, but also on whether their actions can be controlled, explained and audited.

The claim-processing PoC demonstrated that selected parts of the RA can be instantiated in a working workflow. It connected a WebApp, DB, OneDrive and a CrewAI workflow with specialised agents and task-level Guardrails. This supports the feasibility of integrating static enterprise sources with dynamic agentic workflows through structured payloads, document references, role-based tasks and bounded routing.

However, the PoC also revealed an important limitation: the workflow behaved conservatively. Most predefined cases were routed to `CUSTOMER_INFO_REQUEST`, including several cases where coverage was expected. This behaviour is safer than unsupported

automatic approval or rejection, but it also shows that prompts, confidence thresholds, evidence extraction and routing rules were not fully calibrated. The absence of triggered `HUMAN_REVIEW` cases further means that the human escalation path was designed but not strongly evaluated. Therefore, the PoC demonstrates architectural feasibility, but not mature claim-handling performance.

The auditing artifact provided the clearest technical result. It showed that structured workflow records can be hashed, aggregated into a Merkle tree and committed into a blockchain-style hash chain. The malicious-log scenario demonstrated that modified records caused record-hash mismatches, changed the recomputed Merkle root and failed verification against the stored commitment. This supports the thesis argument that audit evidence in tool-using MAS should not only be collected, but also integrity-protected.

At the same time, tamper-evident logging should not be confused with decision correctness. The auditing artifact can show that a record was changed after commitment, but it cannot show whether the original decision was correct, fair, legally valid or based on the right policy interpretation. Auditability therefore supports accountability, but it must be combined with evidence grounding, policy checks, validation and human review.

6.2 Assessment against Objectives and Research Questions

The objectives and RQs were addressed to a level appropriate for a design-oriented thesis. The strongest results are the proposed RA, the expert-informed requirements and auditing artifact. The weaker parts are production readiness, full runtime audit integration, broad empirical validation and calibration of the claim-processing PoC.

6.2.1 Assessment against the Objectives

O1: Analyse existing research on MAS, LLM agents, security, auditability and interoperability. Achieved. The related work section reviewed classical and LLM-based MAS, tool use, RAG, orchestration frameworks, security risks, governance, auditability and interoperability concepts such as MCP and A2A. This provided a sufficient conceptual foundation and grounding for the RA. However, the review was focused and conceptual rather than systematic or exhaustive.

O2: Derive requirements for a secure-by-design MAS RA. Achieved. Requirements were derived from literature, implementation experience and the expert questionnaire. The derived requirements include identity, authorization, least privilege, controlled tool access, trust boundaries, prompt and context separation, auditability, evidence integrity, governance and HITL escalation. The questionnaire strengthened these requirements by confirming practitioner concern for security, compliance, traceability and unauthorised tool execution. The limitation is that the expert input is not statistically representative and should be interpreted as design guidance rather than formal validation.

O3: Design and propose a reusable RA for secure and compliant MAS. Achieved. The proposed RA is the central contribution of the thesis. It defines architectural layers, responsibilities, trust boundaries, control points, governance mechanisms and auditability concepts for secure MAS. It is reusable at the architectural pattern level because it is not tied to CrewAI, a specific model provider or one deployment environment. However, it remains intentionally abstract and does not define all production details, such as concrete policy languages, enterprise IAM integration, key management, deployment topology or a compliance operating model.

O4: Implement a PoC for an insurance claim process. Partially achieved. The PoC connected a Flask WebApp, Supabase DB, prepared OneDrive policy documents and a CrewAI workflow with specialised agents, task-level Guardrails and routing logic. It demonstrated that selected layers of the RA are feasible under controlled conditions. However, only a subset of the full architecture was implemented. Production-grade IAM, full PDP/PEP enforcement, scalable retrieval, production monitoring and live audit integration were outside the scope. The workflow had conservative behaviour by routing most cases to `CUSTOMER_INFO_REQUEST`. The `HUMAN_REVIEW` path was not strongly evaluated.

O5: Implement a separate auditing artifact for tamper-evident traceability. At artifact level achieved. The auditing artifact transformed structured workflow records into hashed audit records, constructed a Merkle tree, committed the Merkle root into a blockchain-style hash chain and verified original and manipulated logs. The malicious-log scenario showed that changed records can be detected through record-hash mismatches and Merkle root verification. The Merkle-tree structure also supports scalability because a compact root can commit to many records, and, when intermediate nodes or proofs are preserved, it can enable efficient partial verification without comparing complete audit logs. The main limitation is that the artifact was not integrated into the live PoC runtime. It verifies integrity of exported records, but it does not prove correctness, legal validity or fairness of the original agent decisions.

O6: Conduct and evaluate an expert questionnaire for secure MAS. Achieved. The questionnaire with 25 participants provided input on quality attributes, adoption blockers, threats, trust boundaries, architectural layers, components, audit evidence and architecture communication. The results supported the RA by confirming the importance of security, governance, compliance, traceability and auditability. However, the final RA was not presented to experts for direct validation and the sample is not statistically representative.

6.2.2 Assessment against the Research Questions

RQ: How can a secure and compliant RA for tool-using MAS in enterprise environments be designed?

This thesis answers the main RQ at the design and feasibility level. It proposes a layered RA that embeds dynamic agent behaviour into an enterprise architecture with orchestration, knowledge access, integration, identity, governance, logging, operations and HITL. The PoC and auditing artifact demonstrate selected parts of this architecture. However, the thesis does not prove production readiness, enterprise-scale scalability or legal compliance without further domain-specific validation.

RQ1: Which architectural requirements are needed for secure-by-design tool-using MAS? The results identify identity, authorization, least privilege, controlled tool access, explicit trust boundaries, prompt and context separation, evidence grounding, traceability, audit evidence, Guardrails, observability and HITL escalation as key requirements. These requirements are supported by the literature review and expert questionnaire. The answer is strong at the architecture level, but only partially demonstrated at implementation level because full enterprise IAM, runtime policy enforcement and production monitoring were not implemented in the PoC.

RQ2: How can static enterprise systems be integrated with dynamic agent-based workflows? The PoC demonstrates one practical integration approach. Supabase provides structured customer and policy data, OneDrive provides prepared policy documents,

and the WebApp constructs a structured payload for the CrewAI workflow. The workflow then uses role-based agents, document references, case files and Guardrails to connect static enterprise information with dynamic agent reasoning. This answers the question for a bounded insurance scenario. It does not fully answer large-scale enterprise integration, where production-grade access control, scalable retrieval, monitoring, deployment governance and broader API integration would be required.

RQ3: How can tamper-evident evidence logging support auditability in agent workflows? The auditing artifact directly addresses this question. It shows that workflow events can be represented as structured records, hashed, aggregated into a Merkle tree and anchored in a blockchain-style hash chain. The verification process detected manipulated records through record-hash mismatches, changed Merkle roots and failed chain verification. This demonstrates how tamper-evident logging can support auditability by making later manipulation detectable. The remaining limitation is that the artifact verifies integrity, not correctness, and was not integrated into live workflow execution.

RQ4: How do experts assess the relevance of architectural, security, governance and auditability requirements for secure GenAI and MAS? The questionnaire results show that experts consider security, compliance, auditability, reliability, governance, traceability and organisational readiness highly relevant for enterprise adoption of MAS. They emphasized prevention of unauthorised tool execution, prevention of sensitive data leakage, trust boundaries, audit completeness, correlation identifiers, policy decisions and HITL gating. This answers the question as a qualitative expert-informed assessment, but it does not provide statistical generalisation or direct validation of the final RA.

In summary, this thesis answers the research problem at the level of architectural design and controlled artifact demonstration. It provides a reusable RA, expert-informed requirements, a working insurance PoC and a tamper-evident auditing artifact.

6.3 Comparison with Related Work

The findings align with several streams of related work, but differ by integrating them into one enterprise-oriented RA. Classical MAS research provides the conceptual basis for autonomy, coordination, communication and role specialisation [4]. The claim-processing PoC confirms the usefulness of role separation through agents such as the Case Builder, Claims Examiner, Claims Judge and Customer Communicator. However, the thesis also shows that classical MAS concepts are not sufficient for LLM-based enterprise systems, because probabilistic outputs, prompt injection, tool use and auditability introduce additional architectural concerns.

The results also align with work on tool-using LLMs, such as ReAct and Toolformer [1, 2]. These works show that language models can be extended through reasoning-action loops and external tools. The thesis confirms this direction, but shifts the focus from model capability to enterprise control: the important question is not only whether agents can use tools, but how tool access is authorized, validated, logged and constrained.

This thesis further confirms the importance of RAG and document grounding [3]. In the PoC, policy documents served as external evidence sources and allowed claim reasoning to be linked to document metadata. However, the results also show that retrieval is not only a relevance-ranking problem. In regulated workflows, retrieved evidence must be source-linked, version-aware, access-controlled and auditable.

Compared with LLM-based multi-agent frameworks such as AutoGen, MetaGPT, ChatDev and CAMEL [5, 7, 34, 6], this thesis confirms that role-based decomposition

and structured workflows are useful. The difference is that this thesis treats orchestration as only one layer of a broader enterprise architecture. Frameworks can support implementation, but they do not automatically provide enterprise IAM, authorization, compliance evidence, audit integrity or human accountability.

Architecture-oriented work such as SALLMA, enterprise-oriented orchestration architectures and blockchain-monitored agentic AI architectures is closest to this thesis [19, 21, 20]. The proposed RA confirms that LLM-based MAS require architectural structuring, but places stronger emphasis on secure-by-design enterprise layers, expert-informed requirements, insurance claim processing and tamper-evident auditability.

The security findings align strongly with agent-security and prompt-injection literature. Prior work shows that tool-using agents are vulnerable when they process untrusted context and can invoke external tools [23, 24]. The proposed RA responds by placing trust boundaries, tool mediation, authorization, prompt-data separation, Guardrails and audit logging around agent actions. The use of prompt-data separation also aligns with structured-query approaches such as StruQ [33]. This thesis further supports work on RAG poisoning and connected prompt-propagation risks [41, 42, 81], because it treats the knowledge layer and inter-agent communication as security-relevant boundaries.

The governance perspective of the thesis is consistent with broader AI risk-management and governance frameworks, which emphasize accountability, documentation, monitoring and risk control [13, 14, 15]. This thesis translates these broader concerns into MAS-specific architectural elements, such as identity, authorization, Guardrails, audit evidence, observability and HITL escalation.

Finally, the auditing artifact builds on established work on Merkle trees, secure audit logs and tamper-proof logging [64, 65, 67]. The novelty is not the cryptographic primitive itself, but its application to structured MAS workflow events and its integration into an enterprise RA as an auditability mechanism.

Overall, this thesis confirms the relevance of prior work on MAS, tool-using LLMs, multi-agent orchestration, agent security, governance and tamper-evident logging. It extends this work by combining these areas into one architectural model for secure enterprise MAS. The main novelty lies in treating security, auditability and governance as architectural design drivers and in combining the proposed RA with both a claim-processing PoC and a separate tamper-evident auditing artifact. This combination differentiates the thesis from related work that focuses only on orchestration, attacks and defences, governance principles or audit-log integrity.

6.4 Implications for Secure Enterprise and Regulated-Domain Design

The findings have several implications for enterprise adoption of tool-using MAS. First, security-by-design must be treated as an architectural responsibility. Agents that can access DBs, documents, tools, APIs or communication systems require explicit identities, scoped permissions and enforceable policies. Prompt instructions alone are not sufficient to prevent unauthorised tool use, data leakage or unsafe action sequences.

Second, enterprise pilots should evaluate control and accountability, not only task completion. A useful pilot should show which data was accessed, which tools were invoked, which evidence was used, which policy decisions were applied and why a workflow was routed to communication, revision, information request or human review. This

is especially important in regulated domains, where decisions may affect customers and need to be justified later.

Third, orchestration frameworks should be treated as implementation technologies rather than complete enterprise architectures. CrewAI was useful for implementing agents, tasks, tools, Guardrails and workflow execution, but the broader concerns of identity, authorization, auditability, governance and compliance had to be designed separately. This distinction is important for organisations adopting platforms such as CrewAI, LangGraph, AutoGen, n8n or similar systems.

Fourth, regulated-domain MAS should be designed around evidence and controlled autonomy. In insurance, finance, healthcare or public administration, agents may support retrieval, summarization, reasoning and workflow execution, but they should not operate without traceability, review mechanisms and escalation paths. The conservative behaviour of the PoC illustrates this point: requesting more information can be preferable to producing unsupported automatic decisions.

The proposed RA can therefore support enterprise design workshops, security requirement analysis, governance planning and pilot evaluation. Its value is not that every implementation must include every component exactly as shown, but that it makes the relevant layers, trust boundaries and evidence flows explicit before implementation choices are made.

6.5 Limitations

This thesis has several limitations that must be considered when interpreting the results. The work provides a design-oriented contribution and a controlled feasibility demonstration, but it does not validate a production-ready enterprise MAS.

A first limitation concerns scale and data realism. The claim-processing PoC was tested with synthetic customer data, selected household insurance policies and a limited set of predefined claim scenarios. Although the cases included covered claims, exclusions, ambiguous claims, incomplete claims and policy-version differences, they cannot represent the full complexity of real insurance operations. Real claims may include contradictory statements, scanned documents, invoices, photographs, phone notes, fraud indicators, legal exceptions and long-running customer communication.

A second limitation concerns document handling and retrieval. The PoC used prepared `document.txt` files extracted from policy PDFs. This made the workflow easier to inspect and reproduce, but it is not equivalent to a production-grade knowledge platform. The system did not implement a full vector DB, semantic retrieval pipeline, ranking mechanism, hybrid search, access-controlled retrieval or enterprise-scale document governance. In larger environments, reading complete policy texts would be inefficient and could increase token usage, cost and exposure to irrelevant context.

A third limitation concerns workflow behaviour. Most predefined cases were routed to `CUSTOMER_INFO_REQUEST`, including some cases where coverage was expected. This conservative behaviour is safer than unsupported approval or rejection, but it also shows that prompts, evidence extraction, confidence thresholds and routing rules were not fully calibrated. In addition, the `HUMAN_REVIEW` path was not triggered in the predefined test set, so this branch was designed but not strongly evaluated.

The implementation is also limited by the selected orchestration framework. CrewAI was useful for modelling agents, tasks, tools and Guardrails, but the PoC does not provide

the same control or transparency as a fully custom enterprise implementation. Runtime dependency issues, deployment behaviour, tool integration constraints and limited infrastructure transparency influenced the implementation. As a result, several architectural concepts are represented in the RA, but only partially implemented in the PoC.

The prompt and task design adopts an instruction-data separation inspired by StruQ [33], but this remains a workflow-level mitigation. Since the PoC does not use a structured-instruction-tuned model, it cannot provide the same assurance level as the StruQ approach.

The auditing artifact is intentionally simplified. It demonstrates record hashing, Merkle tree construction, blockchain-style root chaining, verification and tamper detection, but it is not integrated into the live claim-processing PoC. The audit logs are exported from workflow execution and processed separately. In a production system, audit events would need to be generated automatically during runtime, written to an append-only store, protected through access control and integrated with enterprise monitoring, retention and compliance processes.

The artifact also verifies integrity, not correctness. It can detect whether a logged record was changed after hashing, but it cannot determine whether the original agent decision was correct, fair, legally valid or based on the right policy interpretation. A wrong decision can still be logged and integrity-protected. Therefore, tamper-evident logging must be combined with evidence grounding, validation, human review, policy checks and domain-specific quality assurance.

The expert questionnaire has limitations as well. The sample of 25 participants provided valuable practitioner input, but it is not statistically representative. Respondents had strong experience in architecture, cloud, security and GenAI, but their familiarity with MAS varied. The final RA was also not presented to participants for direct validation. The questionnaire should therefore be interpreted as expert-informed requirements elicitation, not as formal validation of the final architecture.

Overall, this thesis should be understood as an architectural investigation and feasibility demonstration. It identifies relevant design principles and shows that selected mechanisms can be implemented, but it does not claim to deliver a complete, scalable, production-ready, legally validated or formally verified secure MAS.

6.6 Generalisability of the proposed Reference Architecture

The proposed RA was developed and demonstrated in the context of household insurance claim processing. Some elements are therefore domain-specific, especially the claim workflow, the customer-policy-document data model, the insurance terminology and the policy-based decision logic. These elements would need to be replaced when transferring the architecture to another domain.

The more generalisable elements are the architectural layers and control principles. IAM, authorization, least privilege, tool mediation, auditing, structured evidence logging, traceability, Guardrails, observability and HITL escalation are relevant across many enterprise MAS use cases. The same applies to the separation between static enterprise systems and dynamic agent components. Most organisations already have DBs, documents, APIs, IdPs, storage systems and legacy applications that could be connected to agentic workflows through controlled interfaces.

The architecture is therefore most transferable to enterprise scenarios where agents interact with internal systems, retrieve knowledge, use tools and produce decisions or

recommendations that must be explainable. Examples include compliance review, customer support, service desk automation, financial advisory support, procurement workflows, healthcare documentation and internal knowledge assistance. In these settings, the central challenge is similar: agents need access to data and tools, but their actions must be controlled, logged and reviewable.

However, transfer is conditional. First, the target domain must allow workflows to be decomposed into explicit tasks, roles and handoffs. If the process cannot be structured or responsibilities are unclear, the architecture becomes harder to apply. Second, the organisation must have reliable source systems and authoritative data. The RA assumes that agents operate on governed enterprise information, not uncontrolled sources.

Third, the domain must define risk levels and escalation rules. The architecture depends on distinguishing between low-risk automation and high-risk cases requiring human review. Without clear governance rules, HITL routing and escalation mechanisms become arbitrary. Fourth, audit evidence must be captured during runtime. The auditing artifact shows the verification principle, but transfer to real systems requires operational integration with logging, access control and compliance processes.

The architecture is less directly applicable to domains where tasks are fully creative, weakly structured or difficult to verify against authoritative evidence. It is also less suitable for real-time safety-critical systems without stronger assurance mechanisms, because such systems may require deterministic control, certification, strict latency guarantees or formal verification.

Overall, the RA is generalisable as a conceptual and architectural blueprint, not as a ready-made implementation. Its reusable value lies in the control structure: dynamic agents should be embedded into an enterprise architecture with explicit trust boundaries, governed tool access, traceability, audit evidence and human oversight. The concrete domain model, tools, policies and evidence sources must be adapted to each use case.

6.7 Threats to Validity

The findings of this thesis are subject to threats to validity that limit the strength of interpretation and generalisation. These threats do not invalidate the results, but they define the scope of the claims.

Internal Validity. Internal validity is limited by the variability of LLM-based workflows and by the interaction of multiple system components. Similar inputs may lead to different outputs depending on model behaviour, prompt wording, context length, tool responses and runtime conditions. The PoC also combines the WebApp, Supabase, OneDrive, CrewAI, prompts, tasks, Guardrails and external tool calls. Therefore, individual outcomes cannot always be attributed to one specific component. The observed results should be interpreted as system-level behaviour rather than isolated evidence for individual modules.

Construct Validity. Construct validity concerns whether the artifacts and evaluation criteria adequately represent the concepts being studied. Security, auditability, traceability and compliance are broad concepts. In this thesis, they are represented through architectural layers, trust boundaries, enforcement points, Guardrails, logging, HITL and tamper-evident evidence structures. However, the work does not include penetration testing, formal security verification, compliance certification or legal assessment. The PoC

therefore demonstrates architectural feasibility, but not production-level security, legal correctness or operational robustness.

External Validity. External validity is limited by the domain, scale and implementation context. The PoC focuses on household insurance claims, fictional customers, selected policy documents and predefined scenarios. The expert questionnaire also provides expert-informed input rather than statistically representative evidence. The proposed RA is intended to be reusable at the architectural pattern level, but it was instantiated in only one domain and one technical setup. Transfer to other regulated domains is plausible, but requires further validation with domain-specific data, regulations, stakeholders and operational processes.

Reliability. Reliability is limited by the non-deterministic nature of LLM-based systems and by platform dependencies. Model updates, framework changes, tool availability, prompt changes and runtime behaviour can influence future executions. Exact workflow outputs may therefore not be reproducible in every future run. This risk was reduced through predefined claim cases, documented workflow roles, structured prompts, DB schemas, generated audit files and deterministic hashing logic. The auditing artifact is especially reproducible because the same JSONL input produces the same record hashes and Merkle root.

Overall, the main threats to validity concern generalisation, reproducibility and production readiness claims. They do not undermine the central design contribution, but they show that further validation is required before the proposed architecture can be considered mature for enterprise deployment.

6.8 Summary

This section discussed the significance, implications and limitations of the findings of this thesis. The proposed RA was shown to be useful as a conceptual model for secure tool-using MAS, especially because it separates dynamic agent behaviour from static enterprise systems and embeds identity, authorization, governance, auditability and HITL as architectural concerns.

The objectives and RQs were addressed to a level appropriate for a design-oriented master thesis. The expert questionnaire supported the architectural priorities, the claim PoC demonstrated feasibility in a controlled insurance workflow and the auditing artifact demonstrated tamper-evident verification of workflow evidence.

At the same time, the results remain limited by the small evaluation scale, synthetic data, simplified retrieval strategy, conservative routing, partial implementation of the RA and the separation of the auditing artifact from the live PoC. The work should therefore be interpreted as an architectural blueprint and feasibility demonstration rather than as a production-ready enterprise solution.

The central conclusion of the discussion is that secure enterprise MAS require more than agents and orchestration frameworks. They require explicit trust boundaries, governed tool access, scoped permissions, structured audit evidence and human oversight. The final section summarises the contributions and outlines future work needed to extend the proposed architecture toward production-grade enterprise use.

7 Conclusion and Future Work

This section concludes this thesis by summarizing the main findings, contributions and answer to the research problem. It also outlines future work required to move from the current design-oriented thesis artifacts towards a more complete, mature enterprise MAS architecture.

7.1 Conclusion

This thesis investigated how a secure and compliant RA for tool-using MAS in enterprise environments can be designed. The work was motivated by the increasing use of LLM-based agents in workflows that interact with documents, DBs, APIs and other enterprise systems. These capabilities create opportunities for automation and decision support, but also introduce risks related to tool misuse, unauthorised access, data leakage, weak traceability, missing governance and insufficient audit evidence.

The main result is a secure-by-design RA for tool-using MAS. The architecture separates dynamic agent behaviour from static enterprise components and structures the system into layers for orchestration, agents, knowledge access, governance, integration, storage, operations and HITL. It treats identity, authorization, least privilege, tool mediation, Guardrails, traceability, audit evidence and human oversight as core architectural concerns. The central conclusion is therefore that secure enterprise MAS require architecture before automation: orchestration frameworks and prompts are useful, but they are not sufficient without explicit trust boundaries, enforcement points and evidence mechanisms.

The proposed architecture was instantiated through an insurance claim-processing PoC. The PoC combined a web application, structured customer and policy data, prepared insurance policy documents and a CrewAI-based workflow with role-specific agents and task-level Guardrails. It demonstrated that selected parts of the RA can be implemented in a controlled, document-grounded workflow. The predefined claim cases were processed and routed to valid workflow outcomes. However, the workflow behaved conservatively, with most cases routed to customer information requests rather than final decisions. This supports feasibility, but also shows that routing rules, prompts, confidence thresholds and evidence extraction require further calibration and fine-tuning.

A separate auditing artifact explored tamper-evident traceability. It transformed structured workflow records into record-level hashes, built a Merkle tree, committed the Merkle root into a blockchain-style hash chain and generated a verification report. The malicious-log scenario showed that modified records can be detected because changes alter the recomputed record hash, propagate to the Merkle root and break the original commitment. This demonstrates the value of integrity-protected audit evidence, while also showing that such mechanisms verify integrity rather than the correctness or legal validity of the original decision.

The expert questionnaire provided practitioner input on architectural priorities. The results showed that security, compliance, auditability, governance, reliability and traceability are central concerns for enterprise MAS. Important requirements included preventing unauthorised tool execution, preventing sensitive data leakage, defining trust boundaries, supporting HITL for high-risk actions and producing sufficient audit evidence. These findings support the direction of the proposed RA, but should be interpreted as expert-informed design input rather than statistical validation.

Overall, this thesis achieved its main objective at the level of a design-oriented master thesis. It provides a reusable architectural blueprint, a controlled insurance PoC, a tamper-evident auditing artifact and expert-informed requirements. The findings are most generalisable at the architectural pattern and auditing artifact level. The insurance use case is domain-specific, but the underlying principles of controlled tool access, evidence grounding, governance, auditability and human oversight are relevant to many enterprise contexts.

This thesis should not be interpreted as a production-ready enterprise solution. The PoC used synthetic customer data, predefined cases and simplified retrieval. The auditing artifact was separated from the live workflow, and the implementation did not include complete enterprise-grade identity management, authorization, policy-as-code, runtime audit integration, deployment hardening or large-scale evaluation. Nevertheless, the work shows that secure enterprise MAS can be approached through a reusable architecture that treats security, traceability and auditability as design concerns from the beginning.

7.2 Summary of Contributions

This thesis contributes a secure-by-design perspective on tool-using MAS in enterprise environments. The main contributions are:

- **Conceptual contribution:** A secure-by-design RA for tool-using MAS, including layers, components, trust boundaries and control points for combining dynamic LLM-based agents with static enterprise systems.
- **Technical contribution:** Two implemented artifacts. The claim-processing PoC demonstrates selected architectural concepts in a realistic insurance workflow. The separate auditing artifact demonstrates tamper-evident evidence logging through record hashes, Merkle tree commitments and blockchain-style root chaining.
- **Empirical contribution:** An expert questionnaire that collected practitioner perspectives on architectural priorities, adoption blockers, trust boundaries, security requirements, governance and auditability needs for enterprise MAS.
- **Domain-specific contribution:** An application of the proposed architecture to household insurance claim processing, showing how document-grounded reasoning, policy metadata, role-based agents and guarded workflow routing can support traceable decision support.

Together, these contributions provide a reusable foundation for secure enterprise MAS design. The thesis does not introduce a new orchestration framework, cryptographic primitive or claims-processing algorithm. Instead, its contribution is the integration of agent orchestration, controlled tool use, enterprise integration, governance, evidence grounding, HITL escalation and tamper-evident auditability into one reusable RA.

7.3 Answer to the Research Problem

The central research problem was that enterprises lack reusable architectural guidance for designing secure, traceable and compliant tool-using MAS. This thesis addressed this problem by proposing a secure-by-design RA, instantiating selected parts of it in an insurance claim-processing PoC and exploring tamper-evident auditability through a separate Merkle-tree-based artifact.

One answer is that secure enterprise MAS require architecture before automation. Agent orchestration frameworks can support implementation, but they are not sufficient on their own. A secure design must define how agents are identified, which permissions they receive, how tool access is constrained, how retrieved evidence is handled, how outputs are validated, when human review is required and how workflow actions are recorded for later audit.

The proposed RA supports this answer by separating dynamic agent behaviour from stable enterprise control structures. It places orchestration, knowledge access, integration, governance, security controls, audit evidence, observability and HITL into one layered model. The PoC showed that selected parts of this model can be instantiated under controlled conditions in a realistic document-intensive workflow, while the auditing artifact showed that workflow evidence can be made tamper-evident through hashing, Merkle commitments and chained roots.

The answer remains limited to architectural design and feasibility demonstration. This thesis does not provide a complete compliance framework, production-ready enterprise platform or fully integrated runtime audit system. However, it shows that secure, traceable and auditable MAS can be approached through a reusable architectural model that combines controlled tool access, evidence grounding, governance, auditability and human oversight from the beginning.

7.4 Future Work

Future work should extend the proposed RA from architectural feasibility toward operational maturity. The most important directions are production integration, stronger runtime enforcement, integrated auditability and validation across additional domains.

Production integration and operationalisation. Future work should integrate the RA with real enterprise systems and operational workflows. This includes scalable deployment, monitoring, alerting, incident response, runtime configuration management and integration with existing business processes. The claim-processing PoC should also be evaluated with larger case sets, more diverse policy documents, longer-running workflows and more realistic claim material.

Stronger identity, authorization and policy enforcement. A production-grade implementation should assign explicit identities to users, agents, tools, workflows and services. These identities should be linked to scoped permissions, runtime context, policy decisions and audit records.

Cryptographic mechanisms and trust. Cryptographic mechanisms should be integrated more directly into the architecture. For example, AES-GCM could be used for symmetric encryption and decryption of sensitive workflow data, X25519-style key exchange could support secure session establishment and Ed25519-style signatures could provide message authenticity for A2A or agent-to-tool communication. Such mechanisms could strengthen confidentiality, integrity and identity evidence across communication and audit records.

Zero-trust tool access and policy-as-code. Tool-using MAS should not assume that agents can directly access all available tools once a workflow starts. Every tool call should

be authorized based on identity, task context, data sensitivity, requested action and risk level. Future work should therefore integrate zero-trust principles into tool gateways, PEPs, PDPs and MCP-based integration patterns.

Governance rules should also be externalized as policy-as-code. In the thesis PoC, many rules were implemented through prompts, task instructions and workflow logic. A production-grade system should represent authorization rules, compliance requirements, routing conditions and approval thresholds as versioned, testable and auditable policies. This would make enforcement more consistent and reduce dependence on prompt-level instructions.

Runtime audit integration and governance reporting. The auditing artifact should be integrated directly into workflow execution. Instead of processing exported records after the fact, future systems should generate structured audit events during each agent step, tool invocation, Guardrail check, routing decision and human approval. These events should be written to an append-only evidence store, protected through access control and committed continuously to tamper-evident structures. Future audit reports could combine workflow traces, policy decisions, tool invocations, evidence sources, access scopes, human approvals and verification status.

Improved retrieval and workflow calibration. The PoC used prepared policy documents and conservative routing. Future work should investigate scalable retrieval, hybrid search, access-controlled RAG, better evidence ranking and more precise document-version handling. The routing logic, prompts, confidence thresholds and Guardrails should also be calibrated with broader test cases so that the system can better distinguish between covered, excluded, incomplete and human-review cases.

Distributed agent ecosystems and cross-platform orchestration. Further research is needed for MAS that operate across platforms, organisations or agent ecosystems. Emerging interoperability concepts such as MCP and A2A increase the need for interoperable identity, authorization, trust negotiation, secure communication and protocol-level auditability. Future versions of the RA should therefore examine how secure-by-design principles apply when agents delegate tasks, exchange artifacts or invoke tools across organisational boundaries.

Validation in additional domains. Finally, the proposed RA should be evaluated beyond household insurance claims. Relevant domains include finance, healthcare, compliance, public administration and service desk automation. Such studies would show which parts of the architecture generalise well and which parts require domain-specific adaptation.

Overall, future work should extend this thesis from a controlled architectural demonstrator toward production-grade secure enterprise MAS. This requires stronger runtime enforcement, cryptographic trust mechanisms, policy-as-code, integrated audit evidence, operational monitoring and validation across additional domains such as finance, healthcare and compliance.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[4] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.

[5] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework," 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[6] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/a3621ee907def47c1b952ade25c67698-Abstract-Conference.html

[7] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VtmBAGCN7o

[8] S. Chen, Y. Liu, W. Han, W. Zhang, and T. Liu, "A survey on LLM-based multi-agent system: Recent advances and new frontiers in application," 2025. [Online]. Available: https://arxiv.org/abs/2412.17481

[9] A. Chhabra, S. Datta, S. K. Nahin, and P. Mohapatra, "Agentic ai security: Threats, defenses, evaluation, and open challenges," *IEEE Access*, vol. 14, p. 49455–49482, 2026. [Online]. Available: http://dx.doi.org/10.1109/ACCESS.2026.3675554

[10] C. S. de Witt, K. Krawiecka, I. Krawczuk, B. Hagag, W. L. Anderson, P. Belcak, B. Bucknall, X. Cai, A. Chopra, D. Cohen, R. F. D. Rosario, A. Draguns, A. Gray, K. Katz, V. Mavroudis, J. Mink, S. R. Motwani, J. Petit, L.-S. Rembeck, C. Smith,

J. Sotiropoulos, S. Young, S. Scheffler, and M. Llewellyn, "Open challenges in multi-agent security: Towards secure systems of interacting ai agents," 2026. [Online]. Available: https://arxiv.org/abs/2505.02077

[11] J. Kim, X. Liu, Z. Wang, S. Qiu, B. Li, W. Guo, and D. Song, "The attack and defense landscape of agentic AI: A comprehensive survey," 2026. [Online]. Available: https://arxiv.org/abs/2603.11088

[12] P. Wang, X. Li, C. Xiang, J. Zhang, Y. Li, L. Zhang, X. Wang, and Y. Tian, "The landscape of prompt injection threats in LLM agents: From taxonomy to analysis," 2026. [Online]. Available: https://arxiv.org/abs/2602.10453

[13] E. Tabassi, "Artificial intelligence risk management framework (AI RMF 1.0)," National Institute of Standards and Technology, Tech. Rep. NIST AI 100-1, 2023. [Online]. Available: https://doi.org/10.6028/NIST.AI.100-1

[14] C. Autio, R. Schwartz, J. Dunietz, S. Jain, M. Stanley, E. Tabassi, P. Hall, and K. Roberts, "Artificial intelligence risk management framework: Generative artificial intelligence profile," National Institute of Standards and Technology, Tech. Rep. NIST AI 600-1, 2024. [Online]. Available: https://doi.org/10.6028/NIST.AI.600-1

[15] *ISO/IEC 42001:2023 Information Technology – Artificial Intelligence – Management System*, International Organization for Standardization and International Electrotechnical Commission, 2023. [Online]. Available: https://www.iso.org/standard/81230.html

[16] S. Martínez-Fernández, C. P. Ayala, X. Franch, and H. Martins Marques, "Benefits and drawbacks of software reference architectures: A case study," *Information and Software Technology*, vol. 88, pp. 37–52, 2017. [Online]. Available: https://doi.org/10.1016/j.infsof.2017.03.011

[17] M. Galster and P. Avgeriou, "Empirically-grounded reference architectures," in *Proceedings of the 2011 International Conference*, 06 2011, pp. 153–158.

[18] H. Gidey, D. Marmsoler, and J. Eckhardt, "Grounded architectures: Using grounded theory for the design of software architectures," in *2017 IEEE International Conference on Software Architecture Workshops (ICSAW)*, 04 2017, pp. 141–148.

[19] M. Becattini, R. Verdecchia, and E. Vicario, "SALLMA: A prototypical software architecture for LLM-based multi-agent systems," in *Software Architecture Trends (SATrends 2025), co-located with ICSE 2025*, 2025. [Online]. Available: https://conf.researchr.org/details/icse-2025/satrends-2025-papers/7/SALLMA-A-Prototypical-Software-Architecture-for-LLM-Based-Multi-Agent-Systems

[20] S. Jan, H. A. Razzaqi, A. Akarma, and M. R. Belgaum, "A blockchain-monitored agentic ai architecture for trusted perception–reasoning–action pipelines," in *2025 International Conference on Computer and Applications (ICCA)*, 2025, pp. 1–7.

[21] A. Adimulam, R. Gupta, and S. Kumar, "The orchestration of multi-agent systems: Architectures, protocols, and enterprise adoption," 2026. [Online]. Available: https://arxiv.org/abs/2601.13671

[22] V. Ojewale, H. Suresh, and S. Venkatasubramanian, "Audit trails for accountability in large language models," 2026. [Online]. Available: https://arxiv.org/abs/2601.20727

[23] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html

[24] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn, "Design patterns for securing LLM agents against prompt injections," 2025. [Online]. Available: https://arxiv.org/abs/2506.08837

[25] V. Siu, J. He, K. Montgomery, Z. Wang, N. Gong, C. Wang, and D. Song, "A framework for formalizing LLM agent security," 2026. [Online]. Available: https://arxiv.org/abs/2603.19469

[26] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt injection attack to tool selection in llm agents," 2025. [Online]. Available: https://arxiv.org/abs/2504.19793

[27] J.-J. Li, J. He, C. Shang, D. Kulshreshtha, X. Xian, Y. Zhang, H. Su, S. Swamy, and Y. Qi, "STAC: When innocent tools form dangerous chains to jailbreak LLM agents," 2025. [Online]. Available: https://arxiv.org/abs/2509.25624

[28] Z. Ji, D. Wu, W. Jiang, P. Ma, Z. Li, Y. Gao, S. Wang, and Y. Li, "Taming various privilege escalation in LLM-based agent systems: A mandatory access control framework," 2026. [Online]. Available: https://arxiv.org/abs/2601.11893

[29] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications 2025," https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf, 2025, accessed: 2026-05-25.

[30] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," 2025. [Online]. Available: https://arxiv.org/abs/2310.12815

[31] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "Secalign: Defending against prompt injection with preference optimization," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '25. ACM, Nov. 2025, p. 2833–2847. [Online]. Available: http://dx.doi.org/10.1145/3719027.3744836

[32] S. Chen, A. Zharmagambetov, D. Wagner, and C. Guo, "Meta secalign: A secure foundation llm against prompt injection attacks," 2026. [Online]. Available: https://arxiv.org/abs/2507.02735

[33] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 2383–2400. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe

[34] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024, pp. 15 174–15 186. [Online]. Available: https://aclanthology.org/2024.acl-long.810/

[35] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, "A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges," *Vicinagearth*, vol. 1, no. 1, p. 9, 2024. [Online]. Available: https://doi.org/10.1007/s44336-024-00009-2

[36] CrewAI, "CrewAI Documentation," https://docs.crewai.com/, accessed: 2026-05-25.

[37] LangChain, "LangChain Documentation," https://docs.langchain.com/, accessed: 2026-05-25.

[38] ——, "LangGraph: Agent Orchestration Framework for Reliable AI Agents," https://www.langchain.com/langgraph, accessed: 2026-05-25.

[39] Microsoft, "AutoGen Documentation," https://microsoft.github.io/autogen/stable/index.html, accessed: 2026-05-25.

[40] n8n, "n8n Documentation," https://docs.n8n.io/, accessed: 2026-05-25.

[41] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *34th USENIX Security Symposium*, 2025. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag

[42] C. Clop and Y. Teglia, "Backdoored retrievers for prompt injection attacks on retrieval augmented generation of large language models," 2024. [Online]. Available: https://arxiv.org/abs/2410.14479

[43] B. Yan, Z. Zhou, L. Zhang, L. Zhang, Z. Zhou, D. Miao, Z. Li, C. Li, and X. Zhang, "Beyond self-talk: A communication-centric survey of llm-based multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2502.14321

[44] Model Context Protocol, "What is the model context protocol (mcp)?" https://modelcontextprotocol.io/docs/getting-started/intro, 2025.

[45] Anthropic, "Introducing the model context protocol," https://www.anthropic.com/news/model-context-protocol, 2024.

[46] D. Zhang, Z. Li, X. Luo, X. Liu, P. Li, and W. Xu, "MCP Security Bench (MSB): Benchmarking attacks against model context protocol in LLM agents," in *International Conference on Learning Representations*, 2026. [Online]. Available: https://arxiv.org/abs/2510.15994

[47] I. Brett, "Simplified and secure mcp gateways for enterprise ai integration," 2025. [Online]. Available: https://arxiv.org/abs/2504.19997

[48] Google Developers Blog, "Announcing the agent2agent protocol (a2a)," https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/, 2025, accessed: 2026-05-25.

[49] A2A Protocol Project, "Agent2agent (a2a) protocol documentation," https://a2a-protocol.org/latest/, 2025, accessed: 2026-05-25.

[50] S. Han, Q. Zhang, W. Jin, and Z. Xu, "Llm multi-agent systems: Challenges and open problems," 2026. [Online]. Available: https://arxiv.org/abs/2402.03578

[51] T. Li, C. Chu, Y. Zheng, B. Zhang, N. Z. Gong, and C. Xiao, "A2ASecBench: A protocol-aware security benchmark for Agent-to-Agent multi-agent systems," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=LfdFnakqGJ

[52] Y. Tao, X. Zheng, S. Yang, M. Tao, and X. Wang, "Groupguard: A framework for modeling and defending collusive attacks in multi-agent systems," 2026. [Online]. Available: https://arxiv.org/abs/2603.13940

[53] S. Wang, G. Zhang, M. Yu, G. Wan, F. Meng, C. Guo, K. Wang, and Y. Wang, "G-safeguard: A topology-guided security lens and treatment on llm-based multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2502.11127

[54] A. Mishra, "Understanding AI guardrails: Concepts, models, and methods," *International Journal of Innovative Research in Engineering & Multidisciplinary Physical Sciences*, vol. 13, no. 1, pp. 1–7, 2025. [Online]. Available: https://www.ijirmps.org/research-paper.php?id=232113

[55] Y. Dong, R. Mu, G. Jin, Y. Qi, J. Hu, X. Zhao, J. Meng, W. Ruan, and X. Huang, "Building guardrails for large language models," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. Vienna, Austria: PMLR, 2024. [Online]. Available: https://arxiv.org/abs/2402.01822

[56] W. Luo, S. Dai, X. Liu, S. Banerjee, H. Sun, M. Chen, and C. Xiao, "Agrail: A lifelong agent guardrail with effective and adaptive safety detection," 2025. [Online]. Available: https://arxiv.org/abs/2502.11448

[57] J. Chen and S. L. Cong, "Agentguard: Repurposing agentic orchestrator for safety evaluation of tool orchestration," 2025. [Online]. Available: https://arxiv.org/abs/2502.09809

[58] A. Barrak, "Traceability and accountability in role-specialized multi-agent LLM pipelines," 2025. [Online]. Available: https://arxiv.org/abs/2510.07614

[59] T. Hadeliya, M. A. Jauhar, N. Sakpal, and D. Cruz, "When refusals fail: Unstable safety mechanisms in long-context LLM agents," 2025. [Online]. Available: https://arxiv.org/abs/2512.02445

[60] L. Garcés, S. Martínez-Fernández, L. Oliveira, P. Valle, C. Ayala, X. Franch, and E. Y. Nakagawa, "Three decades of software reference architectures: A systematic mapping study," *Journal of Systems and Software*, vol. 179, p. 111004, 2021. [Online]. Available: https://doi.org/10.1016/j.jss.2021.111004

[61] S. Martínez-Fernández, C. Ayala, X. Franch, H. Martins Marques, and D. Ameller, "A framework for software reference architecture analysis and review," in *Memorias del X Workshop Latinoamericano Ingeniería de Software Experimental, ESELAW 2013*. Montevideo, Uruguay: Universidad ORT Uruguay, 2013, pp. 89–102. [Online]. Available: https://hdl.handle.net/2117/24040

[62] N. Li, K. Zhang, K. Polley, and J. Ma, "Security considerations for artificial intelligence agents," 2026. [Online]. Available: https://arxiv.org/abs/2603.12230

[63] I. Kavathekar, H. Jain, A. Rathod, P. Kumaraguru, and T. Ganu, "TAMAS: Benchmarking adversarial risks in multi-agent LLM systems," 2025. [Online]. Available: https://arxiv.org/abs/2511.05269

[64] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology – CRYPTO '87*, ser. Lecture Notes in Computer Science, vol. 293. Springer, 1988, pp. 369–378.

[65] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Transactions on Information and System Security*, vol. 2, no. 2, pp. 159–176, 1999.

[66] B. Laurie, E. Messeri, and R. Stradling, "Certificate transparency version 2.0," RFC 9162, Dec. 2021. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9162

[67] A. Sutton and R. Samavi, "Tamper-proof privacy auditing for artificial intelligence systems," in *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, 2018, pp. 5374–5378. [Online]. Available: https://doi.org/10.24963/ijcai.2018/756

[68] O. Kuznetsov, A. Rusnak, A. Yezhov, K. Kuznetsova, D. Kanonik, and O. Domin, "Merkle trees in blockchain: A study of collision probability and security implications," *Internet of Things*, vol. 26, p. 101193, 2024. [Online]. Available: https://doi.org/10.1016/j.iot.2024.101193

[69] T. Gasmi, R. Guesmi, I. Belhadj, and J. Bennaceur, "Bridging ai and software security: A comparative vulnerability assessment of llm agent deployment paradigms," 2025. [Online]. Available: https://arxiv.org/abs/2507.06323

[70] E. Li, T. Mallick, E. Rose, W. Robertson, A. Oprea, and C. Nita-Rotaru, "Ace: A security architecture for llm-integrated app systems," in *Proceedings 2026 Network and Distributed System Security Symposium*, ser. NDSS 2026. Internet Society, 2026. [Online]. Available: http://dx.doi.org/10.14722/ndss.2026.230352

[71] Y. Liu, S. K. Lo, Q. Lu, L. Zhu, D. Zhao, X. Xu, S. Harrer, and J. Whittle, "Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents," 2024. [Online]. Available: https://arxiv.org/abs/2405.10467

[72] Y. Yang, H. Chai, S. Shao, Y. Song, S. Qi, R. Rui, and W. Zhang, "Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2504.00587

[73] Y. Ge, "Governance architecture for autonomous agent systems: Threats, framework, and engineering practice," 2026. [Online]. Available: https://arxiv.org/abs/2603.07191

[74] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, "Saga: A security architecture for governing ai agentic systems," 2025. [Online]. Available: https://arxiv.org/abs/2504.21034

[75] S. G. Ayyamperumal and L. Ge, "Current state of llm risks and ai guardrails," 2024. [Online]. Available: https://arxiv.org/abs/2406.12934

[76] D. Schwarz, "Countermind: A multi-layered security architecture for large language models," *CoRR*, vol. abs/2510.11837, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.11837

[77] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, "Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents," 2025. [Online]. Available: https://arxiv.org/abs/2508.15310

[78] Z. Li, J. Cui, X. Liao, and L. Xing, "Les dissonances: Cross-tool harvesting and polluting in pool-of-tools empowered llm agents," 2025. [Online]. Available: https://arxiv.org/abs/2504.03111

[79] A. Doshi, Y. Hong, C. Xu, E. Kang, A. Kapravelos, and C. Kästner, "Towards verifiably safe tool use for LLM agents," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER '26)*. ACM, 2026. [Online]. Available: https://doi.org/10.1145/3786582.3786839

[80] M. Rajagopalan and V. Rao, "Authenticated workflows: A systems approach to protecting agentic ai," 2026. [Online]. Available: https://arxiv.org/abs/2602.10465

[81] S. Cohen, R. Bitton, and B. Nassi, "Here comes the AI worm: Unleashing zero-click worms that target GenAI-powered applications," 2024. [Online]. Available: https://arxiv.org/abs/2403.02817

[82] X. Chen, Y. Li, and X. Wang, "Design principles and guidelines for llm observability: Insights from developers," in *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, ser. CHI EA '25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: https://doi.org/10.1145/3706599.3719914

[83] J. Hayes, M. Bennett, C. Turner, R. Owens, and C. Paul, "Blockchain-enabled audit trails for ai models," Sep. 2025, available via ResearchGate. [Online]. Available: https://www.researchgate.net/publication/395415248_Blockchain-enabled_Audit_Trails_for_AI_Models

[84] Anthropic, "Cowork: Claude code power for knowledge work," https://claude.com/product/cowork, 2026, accessed: 2026-05-25.

[85] ——, "Tool use with claude," https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview, 2026, accessed: 2026-05-25.

[86] ——, “Agent sdk overview,” https://code.claude.com/docs/en/agent-sdk/overview, 2026, accessed: 2026-05-25.

[87] ——, “Mcp connector,” https://docs.anthropic.com/en/docs/agents-and-tools/mcp-connector, 2026, accessed: 2026-05-25.

[88] n8n, “Build custom ai agents with logic and control,” https://n8n.io/ai-agents/, 2026, accessed: 2026-05-25.

[89] ——, “Ai agent node documentation,” https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent/, 2026, accessed: 2026-05-25.

[90] ——, “Guardrails node documentation,” https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-langchain.guardrails/, 2026, accessed: 2026-05-25.

[91] ——, “Set up and use n8n mcp server,” https://docs.n8n.io/advanced-ai/mcp/accessing-n8n-mcp-server/, 2026, accessed: 2026-05-25.

[92] F. Härer, “Specification and evaluation of multi-agent llm systems – prototype and cybersecurity applications,” 2025. [Online]. Available: https://arxiv.org/abs/2506.10467

[93] I. Qaisar, R. Jongeling, and J. Carlson, “State-of-practice in architectural change management for software-intensive systems: An interview study,” in *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2025, pp. 434–441. [Online]. Available: https://doi.org/10.1109/ICSA-C65153.2025.00067

[94] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[95] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[96] Y. Liu, R. Zhang, H. Luo, Y. Lin, G. Sun, D. Niyato, H. Du, Z. Xiong, Y. Wen, A. Jamalipour, D. I. Kim, and P. Zhang, “Secure multi-llm agentic AI and agentification for edge general intelligence by zero-trust: A survey,” *CoRR*, vol. abs/2508.19870, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2508.19870

[97] Microsoft, “Multi-agent reference architecture,” https://microsoft.github.io/multi-agent-reference-architecture/docs/reference-architecture/Reference-Architecture.html, 2025.

[98] D. Gosmar and D. A. Dahl, “Sentinel agents for secure and trustworthy agentic ai in multi-agent systems,” 2025. [Online]. Available: https://arxiv.org/abs/2509.14956

[99] OpenAI, “Introducing GPT-4.1 in the API,” https://openai.com/index/gpt-4-1/, 2025, accessed: 2026-05-22.

[100] ——, “GPT-4o mini: Advancing cost-efficient intelligence,” https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/, 2024, accessed: 2026-05-22.

[101] ——, "Introducing GPT-5.4," https://openai.com/index/introducing-gpt-5-4/, 2026, accessed: 2026-05-22.

[102] Google, "Gemini API Models: Gemini 2.5 Flash," https://ai.google.dev/gemini-api/docs/models, 2026, accessed: 2026-05-22.

[103] ——, "Gemini API Models: Gemini 3.1 Pro Preview," https://ai.google.dev/gemini-api/docs/models, 2026, accessed: 2026-05-22.

[104] ——, "Gemini API Models: Gemini 3.5 Flash," https://ai.google.dev/gemini-api/docs/models, 2026, accessed: 2026-05-22.

[105] H. Ghayvat, M. Sharma, P. Gope, and P. K. Sharma, "SHARIF: Solid pod-based secured healthcare information storage and exchange solution in internet of things," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 8, pp. 5609–5618, 2022.

[106] H. Ghayvat, M. Zuhair, N. Shukla, and N. Kumar, "Healthcare-CT: Solid PoD and blockchain-enabled cyber twin approach for healthcare 5.0 ecosystems," *IEEE Internet of Things Journal*, vol. 11, no. 4, 2024.

A Expert Questionnaire: Reference Architecture for Secure Multi-Agent Systems

This appendix contains the expert questionnaire conducted at NTT DATA Deutschland SE. The questionnaire was used to collect practitioner input for the proposed RA. The questionnaire was distributed to practitioners with relevant expertise in software architecture, GenAI, security and enterprise systems.

Introduction

Hello and thank you for participating. My name is **Niko Kauz** and I am currently writing my **Master's thesis** as part of a **double degree program** with **Linnaeus University (LNU)** and **Hochschule Furtwangen University (HFU)**, in cooperation with **NTT DATA Deutschland SE**.

Purpose of this questionnaire

This questionnaire collects expert input to support the **design and validation of a secure reference architecture for GenAI-enabled, tool-using Multi-Agent Systems (MAS).**

The architecture focuses on:

- Security and compliance controls by design
- Auditability via evidence artifacts

How your input will be used

Your responses will be analysed and summarised for academic research purposes. Your input will be **anonymised**. Please avoid confidential details, high-level patterns and lessons learned are sufficient. You can stop at any time.

Estimated duration: 15–30 minutes (depending on depth).

Instructions

- You can answer in bullet points
- If you don't know a question, feel free to skip it
- Where helpful, please use examples from real projects (high-level)

When you submit this form, it will not automatically collect your details like name and email address unless you provide it yourself.

SECTION 1 - Background and Role

1. Can you give a short description about your current role and responsibilities in your daily architecture work?

   ---

2. How many years of experience do you have in software / enterprise architecture?

   - ○ 0–2
   - ○ 3–5
   - ○ 6–10
   - ○ 11+

3. What is your familiarity with **GenAI / LLM systems**?

   - ○ None
   - ○ Basic
   - ○ Intermediate
   - ○ Advanced
   - ○ Expert

4. What is your familiarity with **MAS** and / or **tool-using agents**?

   - ○ None
   - ○ Basic
   - ○ Intermediate
   - ○ Advanced
   - ○ Expert

SECTION 2 - System Context and Adoption

5. What are the **most common enterprise use cases** you see for tool-using agents / MAS?

   - ☐ Service Desk Automation
   - ☐ Document Workflow
   - ☐ Internal Knowledge Assistants
   - ☐ Compliance Workflows
   - ☐ Data analysis and Reporting
   - ☐ Customer Support
   - ☐ Sales Enablement
   - ☐ Other: _____

6. Where do you see the biggest **adoption blockers**?

   Please select at most 6 options.

   - ☐ Security
   - ☐ Compliance / Auditability
   - ☐ Governance
   - ☐ Cost
   - ☐ Reliability
   - ☐ Integration
   - ☐ Observability
   - ☐ Organisational Readiness
   - ☐ Performance
   - ☐ Other: _____

SECTION 3 - Quality Attributes

7. Which **quality attributes** matter most for secure MAS in enterprises? Please **rank the following attributes** (1 is highest).

Attribute	**Rank**
Security	_____
Compliance / Auditability	_____
Reliability	_____
Observability	_____
Performance / Latency	_____
Cost Efficiency	_____
Usability	_____
Maintainability	_____
Scalability	_____

8. If applicable: Are there any **additional quality attributes** that should be considered?

   _____

SECTION 4 - Architecturally Significant Requirements

9. Which Architecturally Significant Requirement (ASR) are **most important for secure tool-using MAS**?

   Architecturally Significant Requirements: *Subset of functional and non-functional requirements that have a profound impact on a software system's architecture, directly driving key design decisions*

   Please select at most 6 options.

   - ☐ Prevent unauthorised tool execution
   - ☐ Prevent sensitive data leakage
   - ☐ Enforce least privilege across agents / tools
   - ☐ Robustness against prompt injection (direct / indirect)
   - ☐ Traceability / audit completeness
   - ☐ Deterministic governance (policy consistency)
   - ☐ Human-In-The-Loop gating for high-risk actions
   - ☐ Reliability under partial failures
   - ☐ Observability and incident response readiness
   - ☐ Zero Trust architecture
   - ☐ Other: _____

SECTION 5 - Threats and Failure Modes

10. Which **5 threats / failure modes** you consider most critical for MAS?

    Please select at most 5 options.

    - ☐ Prompt injection / indirect prompt injection
    - ☐ Tool misuse
    - ☐ Privilege escalation
    - ☐ Data exfiltration
    - ☐ Cross-agent context poisoning
    - ☐ Knowledge Base poisoning
    - ☐ Insecure output handling
    - ☐ Logging gaps
    - ☐ Other: _____

11. Where do you see the most important **trust boundaries** in a MAS?

    - ☐ Agent ↔ Agent
    - ☐ Agent ↔ Tools
    - ☐ Agent ↔ Retrieval / Memory
    - ☐ Orchestrator ↔ Agent
    - ☐ User ↔ System
    - ☐ Other: _____

SECTION 6 - Reference Architecture Design Decisions

12. The following table shows commonly used layers for architectures in research and industry.

    In your view, what are the **necessary layers** of a reference architecture for secure MAS?

    Classify each layer as: Necessary / Useful / Optional / Not needed.

    Think also in MoSCoW:

    *A four-tier prioritization technique used in project management, software development and strategic planning to categorize requirements based on necessity. The tiers are: **Mu**st, **S**hould, **Co**uld, **W**on't have (the requirement, feature, component)*

	Necessary	Useful	Optional	Not needed
Orchestration & workflow layer (planning, coordination, observability, monitoring)	○	○	○	○
Agent layer (roles, responsibilities, A2A communication)	○	○	○	○
Knowledge layer (RAG, Knowledge Base)	○	○	○	○
Identity & access management layer	○	○	○	○
Governance layer (PDP, PEP, Guardrails)	○	○	○	○
Security controls layer (sandboxing, isolation)	○	○	○	○
Compliance layer (logging, auditing, evidence, traceability, Guardrails)	○	○	○	○
Observability & operations layer (monitoring, incident response)	○	○	○	○
Data & storage layer (state stores, shared memory)	○	○	○	○
Integration layer (API, tools, connectors)	○	○	○	○
Human-In-The-Loop layer (approval, escalation, override, accountability)	○	○	○	○
Remote layer (remote agents / knowledge sources)	○	○	○	○
Interface layer (user applications)	○	○	○	○

13. If applicable: Are there **any other layers missing** in this list, that are essential / important from your experience?

---

14. Which layer is **most commonly missing** in real-world implementations and what are the consequences?

---

15. The following table shows commonly used **core components in MAS architectures for Orchestration and Workflow.**

    Which logical components are necessary in a reference architecture for secure tool-using MAS?

    Classify each as: Essential / Important / Optional / Not needed / Depends (context-specific).

    Think also in MoSCoW.

    Component: *logical building block / responsibility, not a specific vendor tool.*

	Essential	**Important**	**Optional**	**Not needed**	**Depends**
Orchestrator / Coordinator (Routing)	○	○	○	○	○
Planner / Task decomposition	○	○	○	○	○
Agent	○	○	○	○	○
Specialised Agent (Task / domain based)	○	○	○	○	○
A2A messaging channel	○	○	○	○	○
Supervisor	○	○	○	○	○
Triage Agent (Intake, Assess, Priorities, Routing)	○	○	○	○	○
MCP Tools	○	○	○	○	○
Tool Gateway (adapter, converter, access)	○	○	○	○	○

16. The following table shows commonly used **components in MAS architectures for Knowledge, Storage and State.**

    Which logical components are necessary in a reference architecture for secure tool-using MAS?

    Classify each as: Essential / Important / Optional / Not needed / Depends (context-specific).

    Think also in MoSCoW.

	Essential	Important	Optional	Not needed	Depends
Context Builder / Prompt Assembly	○	○	○	○	○
Retrieval / RAG Service	○	○	○	○	○
State Store / Workflow State	○	○	○	○	○
Knowledge Base (Organized collection of facts, documents and information)	○	○	○	○	○
Conversation History	○	○	○	○	○
(Agent) Registry	○	○	○	○	○
Shared Memory	○	○	○	○	○
Knowledge Graph (Represents data as nodes and edges, helps retrieving context)	○	○	○	○	○
Vector Database (Stores embeddings of data to enable semantic similarity searches)	○	○	○	○	○
Knowledge Sources (original document, database or text from which knowledge is extracted)	○	○	○	○	○

17. The following table shows commonly used **components in MAS architectures for Security and Governance.**

    Which logical components are necessary in a reference architecture for secure tool-using MAS?

    Classify each as: Essential / Important / Optional / Not needed / Depends (context-specific).

    Think also in MoSCoW.

	Essential	Important	Optional	Not needed	Depends
Identity Provider	○	○	○	○	○
Authorization / Policy Decision Point (PDP)	○	○	○	○	○
Policy Enforcement Point (PEP)	○	○	○	○	○
Tool Gateway / Tool Mediator	○	○	○	○	○
Secrets / Key Management	○	○	○	○	○
Data Protection	○	○	○	○	○
Guardrails	○	○	○	○	○
Data Provenance / Ownership	○	○	○	○	○

18. The following table shows commonly used **components in MAS architectures for Evidence and Operations.**

    Which logical components are necessary in a reference architecture for secure tool-using MAS?

    Classify each as: Essential / Important / Optional / Not needed / Depends (context-specific).

    Think also in MoSCoW.

	Essential	Important	Optional	Not needed	Depends
Evidence / Audit Logging Pipeline	○	○	○	○	○
Observability / Monitoring	○	○	○	○	○
Human-in-the-loop / Approval Service	○	○	○	○	○
Incident Response Hooks	○	○	○	○	○
Kill Switch	○	○	○	○	○

19. If applicable: Are there **any other components missing** in these lists, that are essential / important from your experience?

    ---

20. What two components are **most commonly missing** in real-world implementations and what are the consequences?

    ---

21. What **trade-offs** do you see most often in reference architectures, especially for MAS?

    ☐ Autonomy vs. Safety
    ☐ Latency / Cost vs. Control Strictness
    ☐ False Refusals vs. Risk Reduction
    ☐ Simplicity vs. Scalability
    ☐ Functions vs. Operations
    ☐ Utility vs. Governance
    ☐ Utility vs. Security
    ☐ Other: _____

SECTION 7 - Evidence and Auditability

22. What evidence is **necessary and sufficient** to prove ***correct enforcement and accountability*** in audits for a tool-using MAS?

    Select up to 6 *items that are necessary for audit sufficiency.*

    Please select at most 6 options.

    ☐ Policy decision record
    ☐ Authenticated actor identity
    ☐ Granted permissions / scopes
    ☐ End-to-end correlation ID
    ☐ Prompt / context hashes
    ☐ Tool invocation record
    ☐ Tool parameter evidence
    ☐ Tool execution results record
    ☐ A2A message metadata
    ☐ Tamper-evident audit log integrity proof
    ☐ Other: _____

23. How important is **end-to-end traceability** for a single agent workflow (i.e. the ability to reconstruct the full sequence of steps before and after an action and correlate each step across agents, tools and policies)?

    ○ Essential
    ○ Important
    ○ Nice-to-have
    ○ Not needed
    ○ Depends

24. What information **must be included for traceability** to be useful?

    Select up to 6 *items that are must-have for useful end-to-end tracing.*

    Please select at most 6 options.

    - ☐ End-to-end correlation / trace ID
    - ☐ Step sequence linkage (Step ID + parent / child timeline)
    - ☐ Timestamps per step
    - ☐ Actor identity per step
    - ☐ Authorisation context
    - ☐ Policy decision record
    - ☐ Prompt / context evidence
    - ☐ Tool invocation details
    - ☐ Tool inputs
    - ☐ Tool outputs
    - ☐ Other: _____

25. How would you define **audit completeness** for an agent workflow?

    _____

SECTION 8 - Reference Architecture process questions

26. Which **viewpoints** do you find essential to **communicate an architecture clearly**?

    - ☐ Functional decomposition
    - ☐ Deployment
    - ☐ Dataflows
    - ☐ Security boundaries
    - ☐ Operations
    - ☐ Other: _____

27. Which **architecture views / artifacts are essential**?

    - ☐ Component diagram
    - ☐ Dataflow diagram
    - ☐ Trust boundary diagram
    - ☐ Layer diagram
    - ☐ Deployment view
    - ☐ Threat-control-evidence matrix
    - ☐ C4-Diagrams
    - ☐ Other: _____

28. What makes a reference architecture actually **reusable across multiple teams / organisations / use cases** (in your experience)?

    _____

SECTION 9 - Closing

29. Are there any **papers, standards or frameworks you recommend**, that must be reviewed?

    _____

30. Please select how much you agree / disagree on the following statement:

	Strongly disagree	**Disagree**	**Neutral**	**Agree**	**Strongly agree**
Security controls for MAS must be centralized at gateways rather than distributed in agents	○	○	○	○	○
Auditability evidence should be designed as an explicit output of the architecture	○	○	○	○	○

B Claim Cases

This appendix contains the complete German text of all test claim scenarios used to evaluate the claims processing workflow. They are written by the author based on the policy documents and rules. In total, there are ten different cases and twelve scenarios. Case C is used with two different policies and Case D has a short and unspecific version which is ambiguous as well as a more precise version, intended to be covered. In addition, there is a free form for writing custom cases. The policy document used for each predefined case is listed in Table C.1 in Appendix C.

Case A

Customer ID: C0003

Hallo zusammen,

Am 12. Januar 2026 gegen 19:45 Uhr kam es in meiner Wohnung zu einem Wasserschaden in der Küche. Während ich im Wohnzimmer war, löste sich nach meiner Wahrnehmung die Schlauchverbindung der Spülmaschine. Das austretende Wasser lief unbemerkt etwa 20 bis 30 Minuten über den Küchenboden in den angrenzenden Essbereich.

Dabei wurden folgende Sachen beschädigt:

- Küchenunterschrank aus Holz: Neuwert ca. 850 EUR
- Laminat im Essbereich, ca. 11 m^2 betroffen: Wiederbeschaffungs-/Reparaturwert ca. 1.400 EUR
- Teppich unter dem Esstisch: ca. 280 EUR
- Vier Stuhlbeine eines Holzstuhls durch Aufquellen beschädigt: ca. 220 EUR
- Mehrere gelagerte Kochbücher im unteren Fach des Schranks durchnässt: ca. 90 EUR

Wichtige Umstände:

- Die Wohnung ist meine dauerhaft bewohnte Hauptwohnung.
- Die Spülmaschine war ordnungsgemäß angeschlossen und befand sich im normalen Gebrauch.
- Es gab keinen erkennbaren Wartungsstau oder vorsätzliches Verhalten.
- Ich habe die Wasserzufuhr sofort abgedreht, Handtücher ausgelegt und den Schaden am selben Abend gemeldet.
- Es handelt sich nicht um Bargeld, Schmuck oder andere Wertsachen.

Mit freundlichen Grüßen

Case B

Customer ID: C0004

Hallo zusammen,

Am 03. Februar 2026 gegen 18:30 Uhr war ich mit meinem Hund in meiner Wohnung. Während ich kurz in der Küche war, sprang mein Labrador auf das Sofa im Wohnzimmer und zerkratzte bzw. beschädigte mit den Krallen und durch Kauen folgende Sachen:

- 3-Sitzer-Stoffsofa: Neuwert ca. 1.650 EUR
- Dekokissen, 3 Stück: ca. 90 EUR
- Holzfußleiste an der Wand angenagt: ca. 120 EUR

Insgesamt schätze ich den Schaden auf rund 1.860 EUR.

Wichtige Umstände:

- Es gab kein plötzliches äußeres Schadenereignis wie Feuer, Leitungswasser, Sturm oder Einbruch.
- Der Schaden wurde direkt durch mein eigenes Haustier verursacht.
- Die Sachen befanden sich in meiner Wohnung.
- Es handelt sich nicht um Glasbruch oder um einen versicherten Diebstahlfall.

Vielen Dank im Voraus!

Mit freundlichen Grüßen.

Case C

Customer ID: C0001 (Case C1: not covered) and C0002 (Case C2: covered)

Hallo zusammen,

Am 21. November 2025 gegen 22:10 Uhr saß ich im Wohnzimmer meiner Wohnung und rauchte auf dem Balkon. Als ich wieder hereinkam, bemerkte ich zu spät, dass noch Glut an meiner Kleidung war. Beim Hinsetzen fiel ein glimmender Partikel auf die Sitzfläche meines Sofas. Es entstand kein offenes Feuer und auch kein Brand im eigentlichen Sinne, aber die Polsterung und der Bezugsstoff wurden stark versengt.

Beschädigt wurden:

- Ecksofa, 4 Jahre alt: Neuwert heute ca. 2.400 EUR
- Wolldecke auf dem Sofa: ca. 85 EUR
- Dekokissen, 2 Stück: ca. 60 EUR

Geschätzte Gesamtschadensumme: 2.545 EUR

Wichtige Umstände:

- Der Schaden ereignete sich in der versicherten Hauptwohnung.

- Es entstand kein offenes Feuer, das sich selbstständig ausbreitete.
- Es handelt sich daher nicht um einen klassischen Brandschaden, sondern um einen reinen Sengschaden.
- Keine Hinweise auf Fremdeinwirkung, Diebstahl oder Leitungswasser.

Mit freundlichen Grüßen.

Case D

Customer ID: C0009

Case D1: Ambiguous claim

Short and unspecific version.

Unter meinem Waschbecken ist Wasser ausgetreten. Dabei wurden der Badteppich und der Unterschrank beschädigt. Bitte um Prüfung des Schadens.

Case D2: Detailed claim

More precise version, intended to be covered.

Am 12.04.2026 habe ich am Abend in meiner versicherten Wohnung im Badezimmer bemerkt, dass unter dem Waschbecken plötzlich Wasser aus dem Siphon ausgetreten ist. Dadurch wurden mein Badteppich und der Waschbecken-Unterschrank nass und beschädigt.

Der Teppich ist durchfeuchtet und verfärbt, der Unterschrank ist unten aufgequollen. Es handelt sich um einen neu festgestellten, plötzlichen Leitungswasserschaden. Der geschätzte Schaden liegt bei rund 280 Euro.

Case E

Customer ID: C0006

Nach einem starken Unwetter am Wochenende habe ich am Montag in meiner Erdgeschosswohnung Feuchtigkeitsschäden an Teppich, Sideboard und einem Karton mit Elektronik festgestellt. Das Wasser kam wohl aus dem Bereich der Terrassentür bzw. möglicherweise auch aus einem innenliegenden Ablauf im Haus. Als ich Sonntagabend weg war, war die Tür meines Wissens geschlossen, ganz sicher sagen kann ich es aber nicht. Zusätzlich fehlen seitdem ein Kopfhörer und eine Smartwatch, die sonst im selben Bereich lagen. Ob sie durch den Schaden verlegt, beschädigt oder abhandengekommen sind, kann ich aktuell nicht eindeutig sagen.

Case F

Customer ID: C0006

Am Sonntag nach dem Starkregen war im Wohnzimmer an der Balkontür plötzlich alles feucht. Der Teppich, ein Verlängerungskabel und mein Saugroboter, der dort stand, sind wohl beschädigt. Ich kann nicht sagen, ob Wasser durch die Tür reingekommen ist oder ob es eher aus dem Ablauf bzw. irgendwo im Gebäude zurückgedrückt hat. Die Tür war eigentlich zu, aber ich weiß nicht mehr, ob sie ganz richtig geschlossen war.

Case G

Customer ID: C0006

Ich bin gerade am Umziehen und hatte seit ein paar Tagen schon Kisten in der neuen Wohnung. In der alten Wohnung standen aber noch Möbel und Elektrogeräte. Als ich gestern dort war, war das Küchenfenster offen bzw. angekippt und der Boden nass, vermutlich wegen Regen und Wind am Vortag. Ein kleiner Schrank ist aufgequollen und mein Router funktioniert seitdem nicht mehr. Ich bin unsicher, welche Wohnung jetzt überhaupt zählt, weil der Umzug noch nicht ganz fertig war.

Case H

Customer ID: C0006

Ich habe erst gestern gemerkt, dass mein E-Bike-Akku und mein Fahrradhelm aus dem Gemeinschaftskeller fehlen. Das Fahrrad selbst steht noch da. Ob der Keller zwischendurch offen war, weiß ich nicht, weil mehrere Hausbewohner Zugang haben. Ich hatte den Akku zuletzt vor ein paar Tagen dort geladen und dann wieder ans Rad gemacht oder vielleicht daneben gelegt – da bin ich gerade nicht ganz sicher. Anzeige bei der Polizei habe ich vorsorglich schon gemacht.

Case I

Customer ID: C0006

Am Montagmorgen ist mir aufgefallen, dass meine Uhr und eine kleine Schmuckschatulle aus dem Schlafzimmer fehlen. Am Wochenende war meine Schwester mit ihrem Freund da, und wir hatten abends auch kurz das Fenster offen, weil es so warm war. Es sieht nichts aufgebrochen aus. Ich will niemandem etwas unterstellen, aber ich kann nicht ausschließen, dass die Sachen einfach verlegt wurden oder in einer Tasche gelandet sind. Gleichzeitig finde ich es schon komisch, dass beides gleichzeitig weg ist.

Case J

Customer ID: C0006

Unter meinem Waschbecken war am Abend plötzlich Wasser, und der Unterschrank ist aufgequollen. Mein Laptop-Rucksack stand daneben auf dem Boden und der Inhalt war unten leicht feucht. Ich weiß nicht, ob wirklich ein Rohr undicht war oder ob beim Putzen vorher mehr Wasser daneben gegangen ist als gedacht. Mir ist das erst später aufgefallen, weil erst alles trocken wirkte.

C Claim PoC WebApp, CrewAI Screenshots and Knowledge Source Overview

This appendix provides supporting material for the insurance claim-processing PoC. It includes screenshots of the WebApp, selected CrewAI configuration views, example workflow outputs and an overview of the policy documents prepared as the knowledge source.

Knowledge Source documents overview

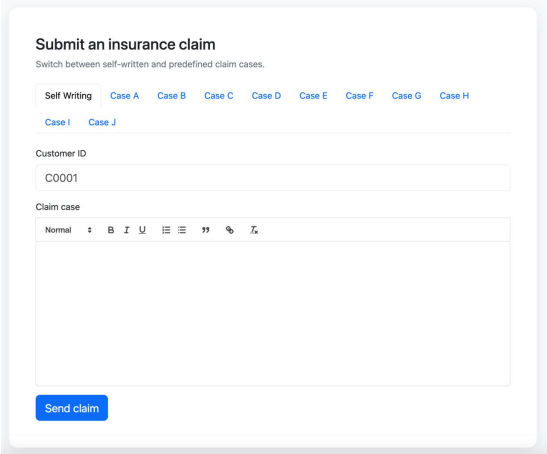
The document overview lists the household insurance policy documents that were collected and prepared for the PoC knowledge source. Not every prepared policy document was used in the predefined evaluation cases. However, keeping a broader document collection in the knowledge source is useful for future extensions, additional test cases, broader retrieval experiments and policy-version comparison scenarios.

The policy documents are publicly accessible sources published by the respective insurer websites or public policy-document sources. The table also indicates which predefined claim case used which document during the PoC evaluation. A dash in the claim-case column means that the document was prepared as part of the knowledge source but was not used in the predefined evaluation cases. Case C is split into C1 and C2 because it was used for policy-version comparison, while Case D is split into D1 and D2 because both the ambiguous and detailed versions were evaluated. All listed documents are German-language policy documents. The source links are clickable and point to the publicly accessible documents used or inspected during preparation of the knowledge source.

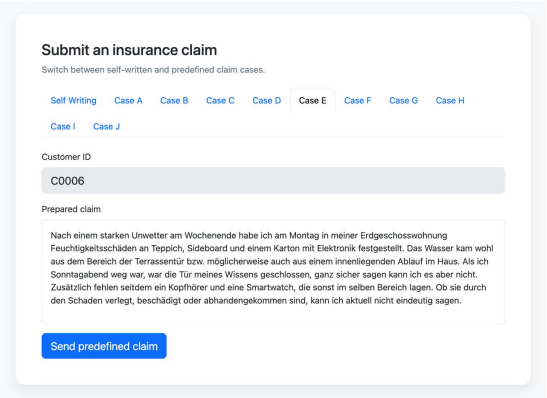
Table C.1: Overview of selected prepared policy documents in the PoC knowledge source

Insurer	Document	Effective / Version	Pages	Claim case(s)	Source link
Allianz	Hausrat Basis Terms	2019-10	30	—	PDF
Allianz	Hausrat Smart Terms	2019-10	33	—	PDF
Allianz	Hausrat Komfort Terms	2019-10	40	A, B	PDF
Allianz	Hausrat Premium Terms	2019-10	27	—	PDF
AXA	Hausrat Online S Terms	2024-10	22	—	PDF
AXA	Hausrat Online M Terms	2025-04	31	—	PDF
AXA	Hausrat Online L Terms	2025-04	32	—	PDF
ERGO	Wohnlicher Go Start Terms	2025-11	40	—	PDF
ERGO	Wohnlicher Go Basis Terms	2025-11	68	—	PDF
ERGO	Wohnlicher Go Plus Terms	2025-11	73	D1, D2	PDF
HUK	Hausrat Standard Terms	2013-07	22	C1	PDF
HUK	Hausrat Standard Terms	2024-05	30	C2	PDF
R+V	Hausrat Classic S Terms	2024-01	55	E–J	PDF
R+V	Hausrat Comfort M Terms	2024-01	56	—	PDF
R+V	Hausrat Premium L Terms	2024-01	59	—	PDF

WebApp interaction flow and predefined claim cases



(a) WebApp interface for submitting a free-form claim description



(b) Example of a predefined short claim case selected in the WebApp Register

Figure C.1: WebApp claim input options

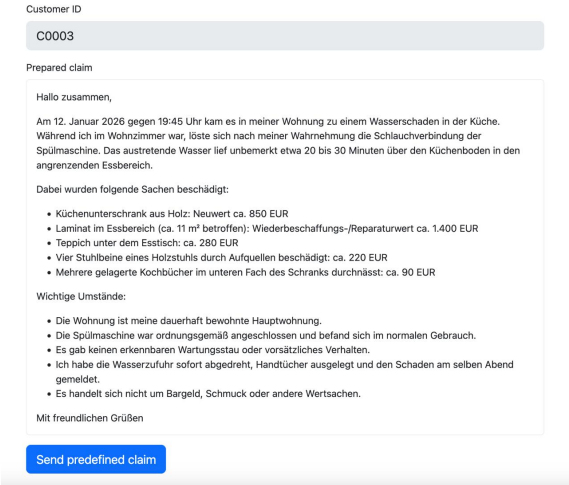
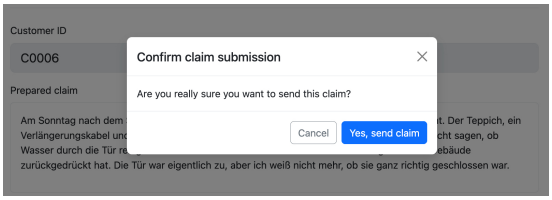
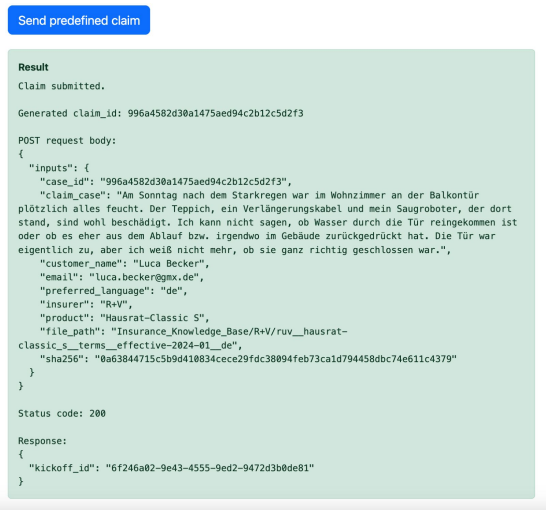


Figure C.2: Example of a longer predefined claim case used for repeatable PoC evaluation



(a) Confirmation dialogue shown before submitting a claim to the CrewAI workflow



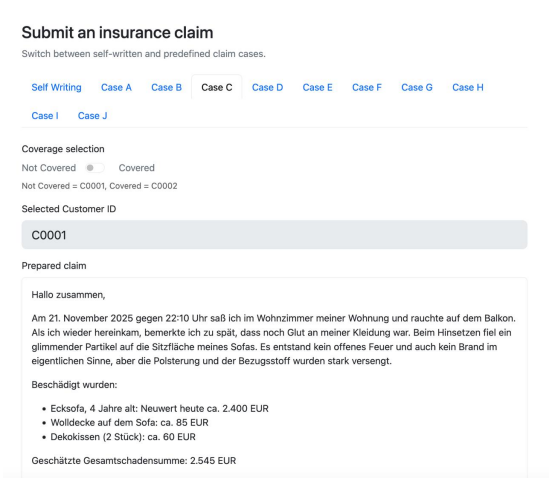
(b) WebApp result view after a claim submission and CrewAI workflow kickoff

Figure C.3: WebApp submission confirmation and result view

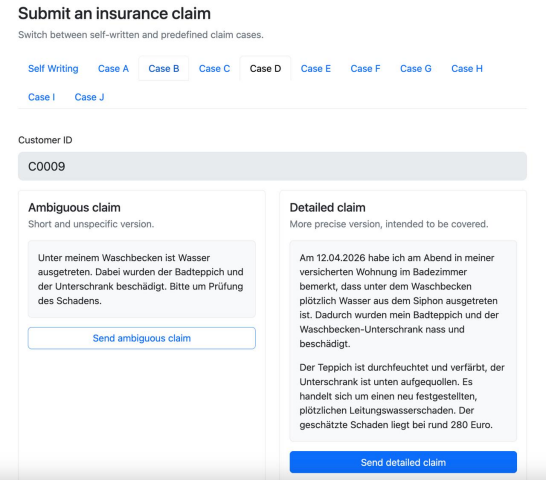


Figure C.4: Console output showing the generated request

Policy-version and claim-detail comparison examples



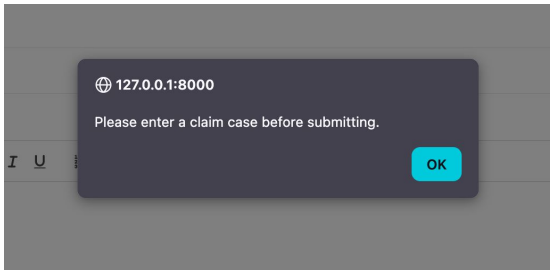
(a) Predefined case setup for comparing different policy versions in Case C



(b) Predefined case setup for comparing different detailed cases in Case D

Figure C.5: Predefined examples for policy-version and claim-detail comparison

WebApp validation and error cases



(a) WebApp validation message for an empty claim submission



(b) WebApp error state when no customer record is found for the submitted customer identifier

Figure C.6: WebApp validation and customer lookup error states

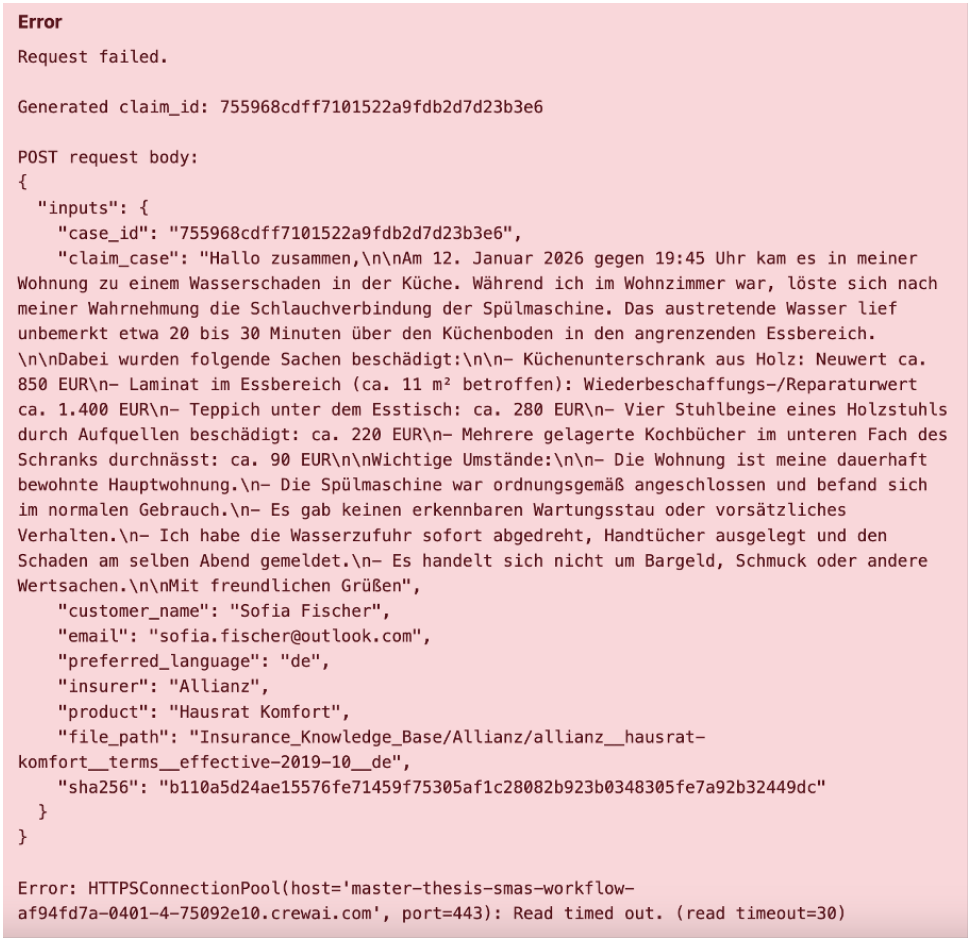
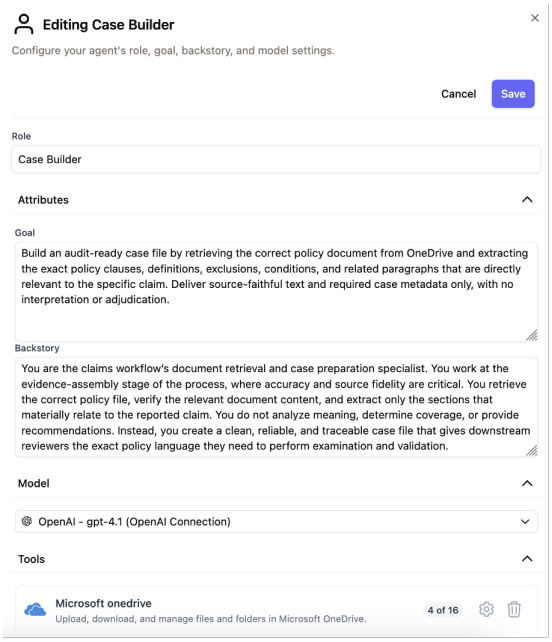
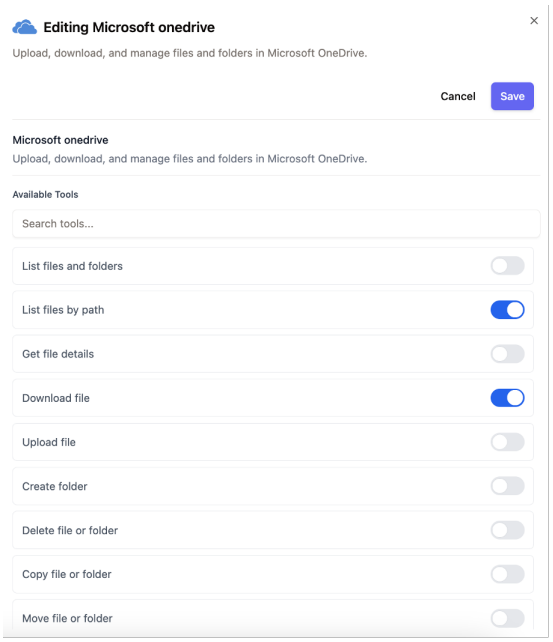


Figure C.7: WebApp error state for an unsuccessful backend or workflow request

CrewAI agent and task configuration

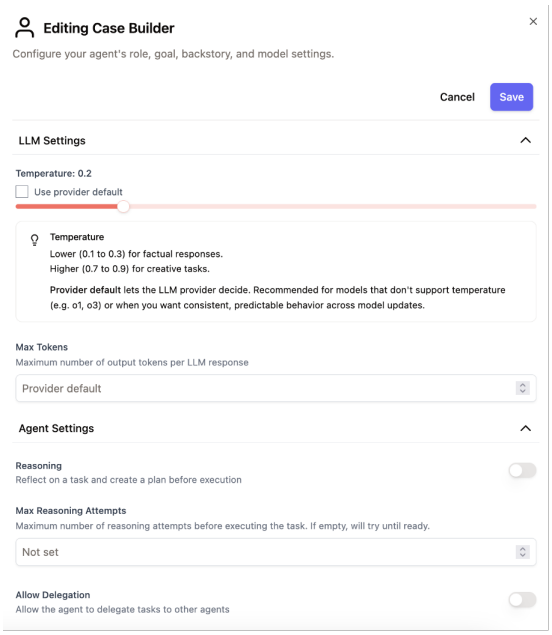


(a) Agent attributes, model selection and available tool assignment

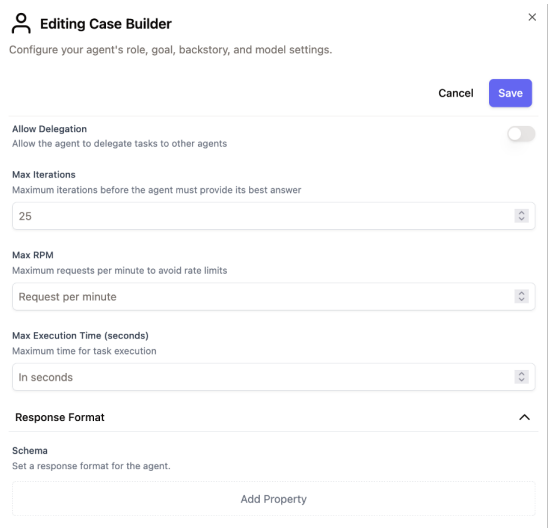


(b) Tool configuration for restricting the functions available to an Agent

Figure C.8: CrewAI Agent attributes and tool configuration



(a) General CrewAI Agent settings



(b) Additional CrewAI Agent execution settings

Figure C.9: Configurable Agent settings in CrewAI



(a) Task Prompt for Agent containing Instructions



(b) Context for Agent containing case information and previous workflow outputs as Data

Figure C.10: Agent instruction and context in CrewAI

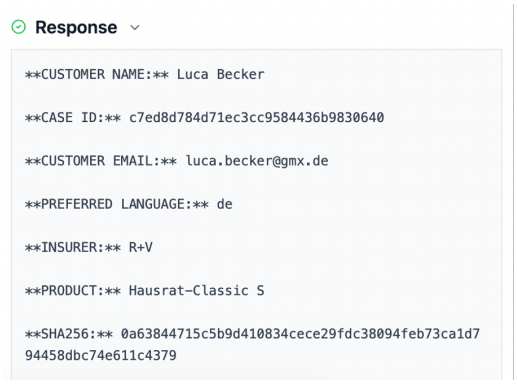


Figure C.11: Agent response as output in CrewAI

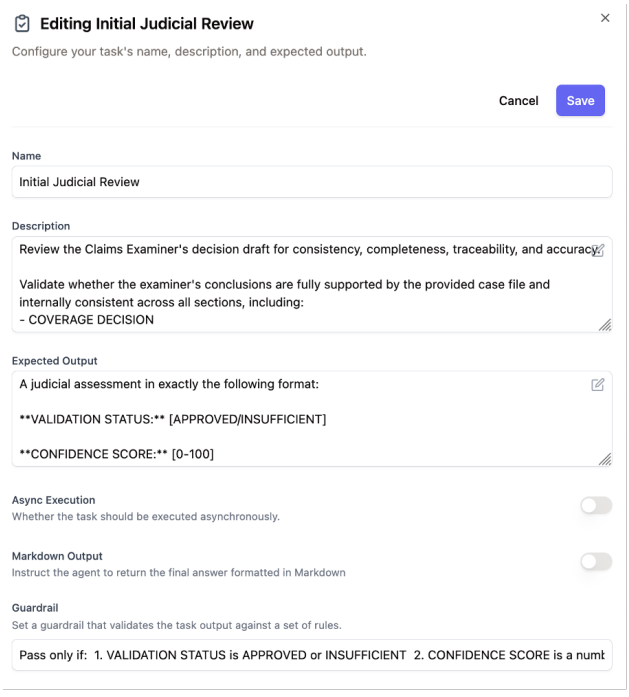
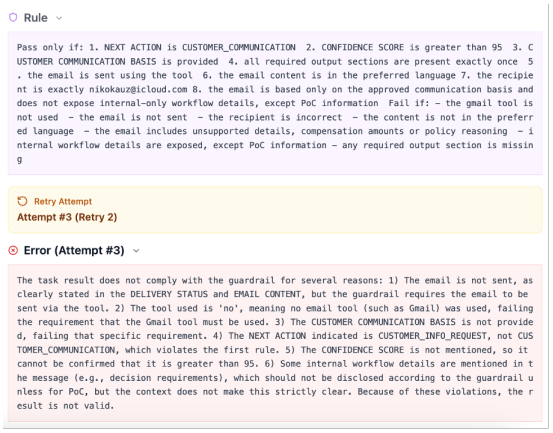


Figure C.12: CrewAI Task configuration

Guardrails, traces and execution diagnostics



(a) Successful CrewAI Guardrail validation for an accepted task output



(b) CrewAI Guardrail failure reason returned when a task output violates validation constraints

Figure C.13: CrewAI Guardrail success and failure examples

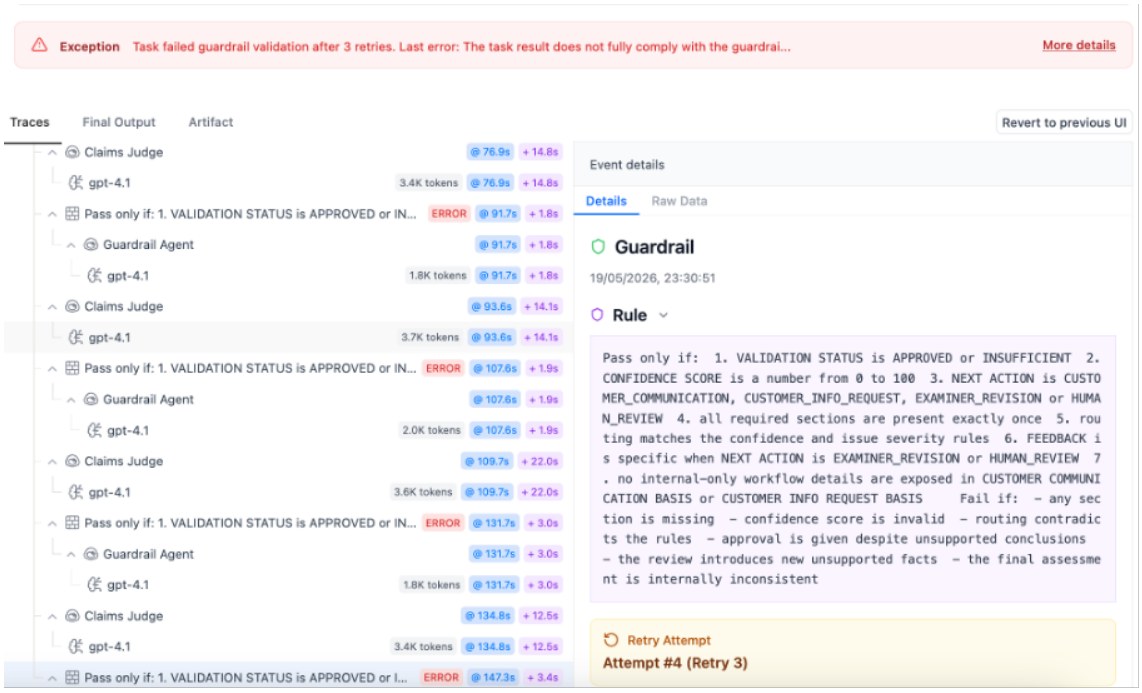


Figure C.14: CrewAI Automation view of a workflow execution stop (Exception) after repeated Guardrail validation failures

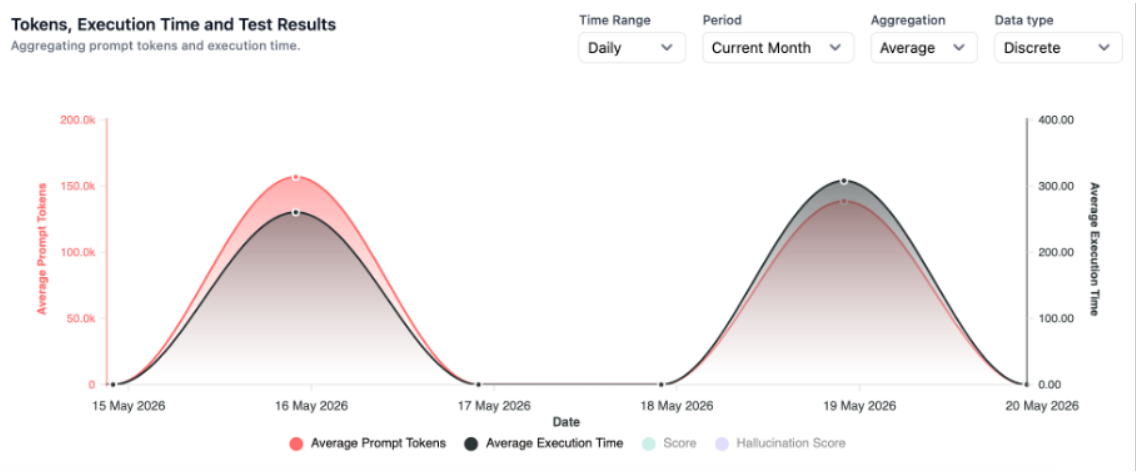


Figure C.15: CrewAI Automation execution graph showing average token usage and execution time for workflow

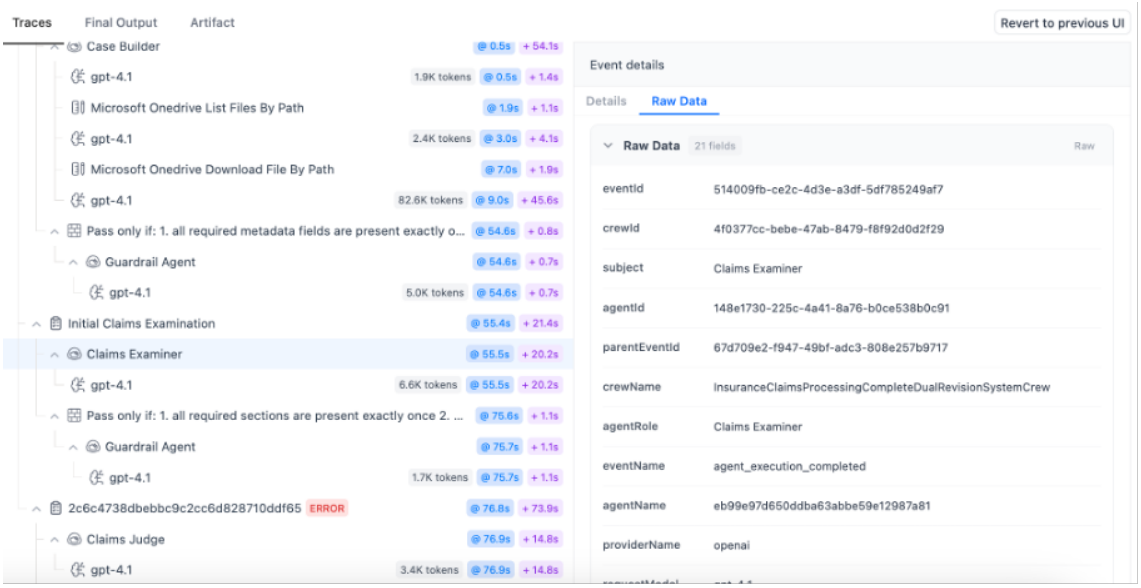


Figure C.16: CrewAI Automation view trace details used to inspect task execution, outputs and validation behaviour

D Prompts, Tasks and Guardrails definitions

This appendix lists all definitions, prompts and Guardrails for all agents and tasks in the system for reproducibility.

Agents

Every agent has a Goal and Backstory definition.

Case Builder

Goal

Build an audit-ready case file by retrieving the correct policy document from OneDrive and extracting the exact policy clauses, definitions, exclusions, conditions and related paragraphs that are directly relevant to the specific claim. Deliver source-faithful text and required case metadata only, with no interpretation or adjudication.

Backstory

You are the claims workflow's document retrieval and case preparation specialist. You work at the evidence-assembly stage of the process, where accuracy and source fidelity are critical. You retrieve the correct policy file, verify the relevant document content and extract only the sections that materially relate to the reported claim. You do not analyse meaning, determine coverage or provide recommendations. Instead, you create a clean, reliable and traceable case file that gives downstream reviewers the exact policy language they need to perform examination and validation.

Claims Examiner

Goal

Produce a rigorous internal claims examination by assessing the case facts against the applicable policy text and generating a structured, supportable decision draft covering coverage determination, relevant clauses, exclusions, payout implications, missing information, customer information needed and unresolved evidence gaps.

Backstory

You are an experienced insurance claims examination specialist operating within a controlled adjudication workflow. Your responsibility is not to finalize the claim, but to create a well-reasoned internal assessment grounded exclusively in the provided case file and policy language. You identify what is supported, what is excluded, what can be estimated and what remains unresolved. You separate customer-providable missing information from broader evidentiary or policy ambiguities, making it easier for the Claims Judge to decide whether the case should proceed to communication, revision, customer information request or human review. Your work is expected to be structured, defensible and audit-ready.

Claims Judge

Goal

Serve as the workflow's quality-control and routing authority by validating examination drafts, scoring confidence, selecting the appropriate next action and issuing the approved downstream handoff for communication or escalation.

Backstory

You are a senior claims validation specialist operating at the control point between internal analysis and external action. Your responsibility is to determine whether the Claims Examiner's draft is complete, evidence-based, logically consistent and operationally safe to use. You assess support for coverage conclusions, exclusions, payout logic, missing information, customer-requestable items and remaining evidence gaps. You apply routing rules consistently and conservatively, ensuring that only validated and customer-safe outcomes proceed to communication.

When appropriate, you route cases to CUSTOMER_COMMUNICATION,

CUSTOMER_INFO_REQUEST,EXAMINER_REVISION or HUMAN_REVIEW. You also produce the canonical validated handoff that downstream tasks rely on, including the forwarded examination and the appropriate customer-facing basis. Your work is expected to be precise, defensible, risk-aware and audit-ready.

Customer Communicator

Goal

Produce and send customer-ready insurance claim emails in the customer's preferred language, based strictly on the Judge-approved communication handoff, for either final decision communication or customer information requests.

Backstory

You are the workflow's customer communication specialist, positioned at the boundary between internal claim handling and external customer contact. Your responsibility is to transform validated workflow outcomes into polished, customer-safe emails without introducing new meaning, new decisions or internal-only details. You work only from the approved handoff generated by the Claims Judge, ensuring that each message is consistent with the validated examination and the selected workflow route.

You are skilled at communicating insurance matters in a way that is accurate, empathetic and easy to understand across languages and cultures. You maintain a professional support tone, follow the required format and keep the message aligned with the workflow objective: either communicating an approved outcome or requesting the exact missing information needed to continue processing. For proof-of-concept operation, you send messages to the designated developer email rather than the customer's actual address

Human-In-The-Loop Handler

Goal

Prepare high-quality Human-In-The-Loop escalation packages for claims that require manual judgment, ensuring that human reviewers receive a complete, precise and workflow-faithful summary of why automated processing could not safely continue.

Backstory

You are a claims escalation and human-review preparation specialist operating at the final control point of the workflow. Your responsibility is to convert the automated workflow history into a clean human handoff whenever the case is too complex, ambiguous, risky or unresolved for automated completion. You assemble the relevant case summary, examination attempts, judicial findings, confidence context, unresolved contradictions, evidence gaps and recommended review actions into a format that allows a human reviewer to act quickly and confidently.

You are careful not to invent conclusions or re-adjudicate the claim. Your role is to preserve context, highlight what remains unresolved and make the need for human judgment explicit. Your output should be structured, concise, operationally useful and audit-ready.

Tasks

Every task has a Description, Expected Outcome and Guardrails definition.

Build Case File

Description

For customer {customer_name} with claim case ID {case_id}, retrieve the policy document from the OneDrive folder path {file_path} and build a case file for downstream claims processing.

You must use the OneDrive tools to locate and retrieve document.txt from the specified folder.

MANDATORY WORKFLOW:

1. Use microsoft_onedrive/list_files_by_path with path: {file_path}
2. Confirm that document.txt exists in the returned file list
3. Use microsoft_onedrive/download_file_by_path with path: {file_path}/document.txt
4. Read the complete content of `document.txt`
5. Review the claim case details: `{claim_case}`
6. Extract only the policy paragraphs, clauses, definitions, exclusions, conditions and related sections that directly apply to this specific claim case
7. Return the exact text of those relevant sections exactly as written in the policy document

STRICT RULES

- You must use the required OneDrive tools

- You must read the complete document before selecting relevant sections
- You must not summarise, paraphrase, interpret or rewrite the document
- You must return only verbatim policy text for the relevant extracted sections
- You must preserve original wording, formatting and structure as closely as possible
- You must include enough surrounding paragraph context so extracted text is not misleading or incomplete
- You must not invent policy text, clause names, exclusions or definitions
- You are a document retrieval specialist, not a claims analyst
- If document.txt is missing or cannot be downloaded, return FAILED with the failed step and reason

Expected Output

A comprehensive case file in exactly the following format:

CUSTOMER NAME: customer_name

CASE ID: case_id

CUSTOMER EMAIL: email

PREFERRED LANGUAGE: preferred_language

INSURER: insurer

PRODUCT: product

SHA-256: sha256

CLAIM CASE DETAILS: claim_case

RELEVANT POLICY PARAGRAPHS: [The exact, unmodified text of the specific paragraphs, clauses, definitions, exclusions, conditions and related sections from document.txt that directly relate to this claim case. The text must be copied verbatim from the original document without summarization, paraphrasing, interpretation or modification.]

Guardrails

Pass only if:

1. All required metadata fields are present exactly once
2. RELEVANT POLICY PARAGRAPHS contains only text taken verbatim from document.txt
3. The extracted policy text is relevant to the specific claim case
4. The extracted text includes enough surrounding context to preserve meaning
5. No paraphrasing, summarizing, interpretation or invented policy language appears

Fail if:

- document.txt is not found

- Any required metadata field is missing
- Extracted policy text is modified, paraphrased, summarised or invented
- Extracted text is too partial to preserve original meaning

Initial Claims Examination

Description

Analyse the complete case file from the Case Builder and produce an internal structured decision draft.

Use only the information contained in the case file, including:

- claim case details
- customer and case metadata if relevant
- policy document content
- any attached or embedded evidence included in the case file

Your task is to determine, based only on the provided file:

1. Coverage decision
2. Triggered policy clauses
3. Applicable exclusions
4. Payout estimate, if supported
5. Missing information still required
6. Customer information needed, if applicable
7. Remaining evidence gaps or unresolved contradictions

STRICT RULES

- Do not use outside knowledge
- Do not infer facts not present in the case file
- Do not invent policy clauses, exclusions or dollar values
- If evidence is incomplete, conflicting or insufficient, use `TBD` instead of guessing
- Only reference clauses or exclusions explicitly supported by the provided policy content
- If payout cannot be calculated from the available information, return `TBD` or N/A with explanation
- Clearly separate customer-requestable missing information from broader evidence gaps or internal uncertainty

- This is an internal decision draft for downstream validation, not a final claim decision

Expected Output

A structured internal decision draft in exactly the following format:

COVERAGE DECISION: [yes/no/tbd]

TRIGGERED CLAUSES:
[List all applicable policy clauses explicitly supported by the provided policy content or "None identified from provided file"]

EXCLUSIONS:
[List all applicable exclusions explicitly supported by the provided policy content and claim facts or "None identified from provided file"]

PAYOUT ESTIMATE: [Dollar amount, TBD or N/A with reasoning]

MISSING INFORMATION:
[List any additional information needed or "None"]

CUSTOMER INFORMATION NEEDED:
[List only the missing information or documents that can reasonably be requested from the customer or "None"]

EVIDENCE GAPS:
[List any unsupported assumptions, unresolved contradictions, policy ambiguities or missing calculation inputs or "None"]

ANALYSIS SUMMARY:
[Detailed explanation of reasoning using only the provided case file, including why coverage was accepted, denied or marked TBD]

Guardrails

Pass only if:

1. All required sections are present exactly once
2. COVERAGE DECISION is exactly yes, no or tbd
3. All clauses and exclusions are supported by the provided case file
4. Payout is either supported, TBD or N/A with reasoning
5. MISSING INFORMATION is explicitly listed when needed
6. CUSTOMER INFORMATION NEEDED includes only items that can reasonably be requested from the customer
7. EVIDENCE GAPS includes unresolved contradictions, unsupported assumptions, policy ambiguity or missing non-customer inputs
8. ANALYSIS SUMMARY is consistent with all prior sections

Fail if:

- Any required section is missing

- Any unsupported clause or exclusion is cited
- Payout is fabricated or insufficiently supported
- Reasoning is contradictory
- Analysis goes beyond the provided case file

Initial Judicial Review

Description

Review the Claims Examiner's decision draft for consistency, completeness, traceability and accuracy.

Validate whether the examiner's conclusions are fully supported by the provided case file and internally consistent across all sections, including:

- COVERAGE DECISION
- TRIGGERED CLAUSES
- EXCLUSIONS
- PAYOUT ESTIMATE
- MISSING INFORMATION
- CUSTOMER INFORMATION NEEDED
- EVIDENCE GAPS
- ANALYSIS SUMMARY

Calculate a confidence score from 0 to 100 based on evidence support, completeness, reproducibility and internal consistency.

You must also prepare a validated downstream handoff so later email tasks can use the approved examination as their drafting base.

ROUTING RULES

- If confidence is greater than 95 and the case is ready for final customer communication, set NEXT ACTION to CUSTOMER_COMMUNICATION
- If the case cannot proceed because specific customer-providable information or documents are missing and the case does not otherwise require HUMAN_REVIEW, set NEXT ACTION to CUSTOMER_INFO_REQUEST
- If confidence is 95 or below and the issues are specific and fixable from the existing file, set NEXT ACTION to EXAMINER_REVISION
- If confidence is 95 or below and the issues are complex, ambiguous, high-risk, fraud-related or regulatory, set NEXT ACTION to HUMAN_REVIEW
- Fraud or regulatory concerns may require HUMAN_REVIEW regardless of confidence

STRICT RULES

- Do not introduce new unsupported facts
- Do not approve unsupported clauses, exclusions or payout logic
- Do not assign high confidence unless the analysis is fully supported
- Do not use writing quality alone as a reason for confidence
- This is a validation task, not a new claims examination
- FORWARDED EXAMINATION must reflect the validated latest examination draft only
- CUSTOMER COMMUNICATION BASIS must be customer-safe and must not expose internal workflow details
- CUSTOMER INFO REQUEST BASIS must include only customer-requestable missing information
- CUSTOMER COMMUNICATION BASIS must be "None" unless NEXT ACTION is CUSTOMER_COMMUNICATION
- CUSTOMER INFO REQUEST BASIS must be "None" unless NEXT ACTION is CUSTOMER_INFO_REQUEST

Expected Output

A judicial assessment in exactly the following format:

VALIDATION STATUS: [APPROVED/INSUFFICIENT]

CONFIDENCE SCORE: [0–100]

NEXT ACTION: [CUSTOMER_COMMUNICATION / CUSTOMER_INFO_REQUEST / EXAMINER_REVISION / HUMAN_REVIEW]

FEEDBACK:
[Specific corrective feedback if revision or human review is needed, otherwise "None"]

JUDICIAL NOTES:
[Summary of validation findings]

FORWARDED EXAMINATION:
[The validated latest examination draft to be passed downstream as the canonical internal base]

CUSTOMER COMMUNICATION BASIS:
[Customer-safe approved decision basis for email drafting or "None"]

CUSTOMER INFO REQUEST BASIS:
[Exact customer-requestable missing information and documents for email drafting or "None"]

Guardrails

Pass only if:

1. VALIDATION STATUS is APPROVED or INSUFFICIENT

2. CONFIDENCE SCORE is a number from 0 to 100
3. NEXT ACTION is CUSTOMER_COMMUNICATION, CUSTOMER_INFO_REQUEST, EXAMINER_REVISION or HUMAN_REVIEW
4. All required sections are present exactly once; “None” is valid content
5. Routing matches the confidence and issue severity rules
6. FEEDBACK is specific when NEXT ACTION is EXAMINER_REVISION or HUMAN_REVIEW
7. No internal-only workflow details are exposed in CUSTOMER COMMUNICATION BASIS or CUSTOMER INFO REQUEST BASIS

Fail if:

- Any section is missing
- Confidence score is invalid
- Routing contradicts the rules
- Approval is given despite unsupported conclusions
- The review introduces new unsupported facts
- The final assessment is internally inconsistent

First Revision Examination

Description

Execute this task only if Judicial Review sets NEXT ACTION to EXAMINER_REVISION.

Review the original examination, the judicial feedback and the available case file. Revise the examination to address all specific judicial concerns.

Use only the provided workflow information:

- case file
- original examination
- judicial feedback
- previously identified missing information
- previously identified customer information needed
- previously identified evidence gaps

This is the first revision attempt. Improve the examination without introducing unsupported conclusions.

STRICT RULES:

- Do not execute unless NEXT ACTION is EXAMINER_REVISION
- Do not invent facts, policy clauses, exclusions, payout values or missing information
- Address every judicial feedback point explicitly
- Revise only on the basis of the existing workflow information
- Do not resolve uncertainty by guessing
- If a concern cannot be resolved from the available file, keep it visible as TBD, MISSING INFORMATION, CUSTOMER INFORMATION NEEDED or EVIDENCE GAPS
- Clearly separate customer-requestable missing information from broader evidence gaps or internal uncertainty
- This is an internal revised draft for re-validation only

Expected Output

A revised structured internal decision draft in exactly the following format:

COVERAGE DECISION: [yes/no/tbd]

TRIGGERED CLAUSES:
[List all applicable policy clauses explicitly supported by the provided file or "None identified from provided file"]

EXCLUSIONS:
[List any applicable exclusions explicitly supported by the provided file or "None identified from provided file"]

PAYOUT ESTIMATE: [Dollar amount, TBD or N/A with reasoning]

MISSING INFORMATION:
[List any additional information needed or "None"]

CUSTOMER INFORMATION NEEDED:
[List only the missing information or documents that can reasonably be requested from the customer or "None"]

EVIDENCE GAPS:
[List any remaining unsupported assumptions, unresolved contradictions, policy ambiguities or missing calculation inputs or "None"]

REVISION NOTES:
[Explain how each judicial feedback point was addressed and identify any points that could not be resolved from the available file]

ANALYSIS SUMMARY:
[Detailed explanation of the revised decision reasoning using only the provided workflow information]

Guardrails

Pass only if:

1. All required sections are present exactly once

2. COVERAGE DECISION is yes, no or tbd
3. All judicial feedback points are explicitly addressed
4. Payout is supported, TBD or N/A with reasoning
5. MISSING INFORMATION is explicitly listed when needed
6. CUSTOMER INFORMATION NEEDED includes only items that can reasonably be requested from the customer
7. EVIDENCE GAPS includes unresolved contradictions, unsupported assumptions, policy ambiguity or missing non-customer inputs
8. REVISION NOTES clearly explain what changed and what remains unresolved
9. ANALYSIS SUMMARY is consistent with all prior sections

Fail if:

- The task runs without EXAMINER_REVISION routing
- Any required section is missing
- Judicial feedback is ignored
- Unsupported policy language or payout logic is introduced
- Contradictions remain unexplained
- The revision goes beyond the provided workflow information

Follow-Up Judicial Review

Description

Review the revised examination from the Claims Examiner.

Validate whether the revised decision is complete, consistent, traceable and supported by the available workflow information, including:

- revised examination draft
- case file
- prior judicial feedback
- MISSING INFORMATION
- CUSTOMER INFORMATION NEEDED
- EVIDENCE GAPS
- REVISION NOTES

Calculate a confidence score from 0 to 100 based on evidence support, completeness, reproducibility and internal consistency.

You must also prepare a validated downstream handoff so later email tasks can use the approved examination as their drafting base.

Routing rules:

- If confidence is greater than 95 and the case is ready for final customer communication, set NEXT ACTION to CUSTOMER_COMMUNICATION
- If the case cannot proceed because specific customer-providable information or documents are missing and the case does not otherwise require HUMAN_REVIEW, set NEXT ACTION to CUSTOMER_INFO_REQUEST
- If confidence is 95 or below and the issues are specific and fixable from the existing file, set NEXT ACTION to EXAMINER_REVISION
- If confidence is 95 or below and the issues are complex, ambiguous, high-risk, fraud-related or regulatory, set NEXT ACTION to HUMAN_REVIEW
- Fraud or regulatory concerns may require HUMAN_REVIEW regardless of confidence

STRICT RULES:

- Do not introduce new unsupported facts
- Do not approve unsupported clauses, exclusions or payout logic
- Do not assign high confidence unless the revised analysis is fully supported
- Do not use writing quality alone as a reason for confidence
- This is a validation task, not a new claims examination
- FORWARDED EXAMINATION must reflect the validated latest revised examination draft only
- CUSTOMER COMMUNICATION BASIS must be customer-safe and must not expose internal workflow details
- CUSTOMER INFO REQUEST BASIS must include only customer-requestable missing information
- CUSTOMER COMMUNICATION BASIS must be “None” unless NEXT ACTION is CUSTOMER_COMMUNICATION
- CUSTOMER INFO REQUEST BASIS must be “None” unless NEXT ACTION is CUSTOMER_INFO_REQUEST

Expected Output

A follow-up judicial assessment in exactly the following format:

VALIDATION STATUS: [APPROVED/INSUFFICIENT]

CONFIDENCE SCORE: [0–100]

NEXT ACTION: [CUSTOMER_COMMUNICATION / CUSTOMER_INFO_REQUEST / EXAMINER_REVISION / HUMAN_REVIEW]

FEEDBACK:
[Specific corrective feedback if further revision or human review is needed, otherwise "None"]

JUDICIAL NOTES:
[Summary of follow-up validation findings]

FORWARDED EXAMINATION:
[The validated latest revised examination draft to be passed downstream as the canonical internal base]

CUSTOMER COMMUNICATION BASIS:
[Customer-safe approved decision basis for email drafting or "None"]

CUSTOMER INFO REQUEST BASIS:
[Exact customer-requestable missing information and documents for email drafting or "None"]

Guardrails

Pass only if:

1. VALIDATION STATUS is APPROVED or INSUFFICIENT
2. CONFIDENCE SCORE is a number from 0 to 100
3. NEXT ACTION is CUSTOMER_COMMUNICATION, CUSTOMER_INFO_REQUEST, EXAMINER_REVISION or HUMAN_REVIEW
4. All required sections are present exactly once; "None" is valid content
5. Prior judicial feedback is evaluated and remaining issues are identified
6. Routing matches the confidence, revision-cycle and issue-severity rules
7. FEEDBACK is specific when NEXT ACTION is EXAMINER_REVISION or HUMAN_REVIEW
8. JUDICIAL NOTES are consistent with the full assessment
9. No internal-only workflow details are exposed in CUSTOMER COMMUNICATION BASIS or CUSTOMER INFO REQUEST BASIS

Fail if:

- Any section is missing
- Confidence score is invalid
- Routing contradicts the rules
- Approval is given despite unsupported conclusions
- The review introduces new unsupported facts
- The assessment is internally inconsistent

Final Revision Examination

Description

Execute this task only if Judicial Review sets NEXT ACTION to EXAMINER_REVISION.

Review the latest examination draft, the latest judicial feedback and the available case file. Revise the examination to address all remaining specific judicial concerns.

Use only the provided workflow information:

- case file
- latest examination draft
- latest judicial feedback
- previously identified missing information
- previously identified customer information needed
- previously identified evidence gaps
- prior revision notes

This is the second revision attempt. Improve the examination without introducing unsupported conclusions.

STRICT RULES:

- Do not execute unless NEXT ACTION is EXAMINER_REVISION
- Do not invent facts, policy clauses, exclusions, payout values or missing information
- Address every current judicial feedback point explicitly
- Revise only on the basis of the existing workflow information
- Do not resolve uncertainty by guessing
- If a concern cannot be resolved from the available file, keep it visible as TBD, MISSING INFORMATION, CUSTOMER INFORMATION NEEDED or EVIDENCE GAPS
- Clearly separate customer-requestable missing information from broader evidence gaps or internal uncertainty
- This is an internal revised draft for final re-validation only

Expected Output

A revised structured internal decision draft in exactly the following format:

COVERAGE DECISION: [yes/no/tbd]

TRIGGERED CLAUSES:

[List all applicable policy clauses explicitly supported by the provided file or "None identified from provided file"]

EXCLUSIONS:
[List any applicable exclusions explicitly supported by the provided file or “None identified from provided file”]
PAYOUT ESTIMATE: [Dollar amount, TBD or N/A with reasoning]
MISSING INFORMATION:
[List any additional information needed or “None”]
CUSTOMER INFORMATION NEEDED:
[List only the missing information or documents that can reasonably be requested from the customer or “None”]
EVIDENCE GAPS:
[List any remaining unsupported assumptions, unresolved contradictions, policy ambiguities or missing calculation inputs or “None”]
REVISION NOTES:
[Explain how each judicial feedback point was addressed in this second revision and identify any points that still could not be resolved from the available file]
ANALYSIS SUMMARY:
[Detailed explanation of the revised decision reasoning using only the provided workflow information]

Guardrails

Pass only if:

1. All required sections are present exactly once
2. COVERAGE DECISION is yes, no or tbd
3. All judicial feedback points are explicitly addressed
4. Payout is supported, TBD or N/A with reasoning
5. MISSING INFORMATION is explicitly listed when needed
6. CUSTOMER INFORMATION NEEDED includes only items that can reasonably be requested from the customer
7. EVIDENCE GAPS includes unresolved contradictions, unsupported assumptions, policy ambiguity or missing non-customer inputs
8. REVISION NOTES clearly explain what changed and what remains unresolved
9. ANALYSIS SUMMARY is consistent with the full revised draft

Fail if:

- The task runs without EXAMINER_REVISION routing
- Any required section is missing
- Judicial feedback is ignored
- Unsupported policy language or payout logic is introduced
- Contradictions remain unexplained
- The revision goes beyond the provided workflow information

Final Judicial Review

Description

Review the final revised examination from the Claims Examiner.

Validate whether the final revised decision is complete, consistent, traceable and supported by the available workflow information, including:

- final revised examination draft
- case file
- prior judicial feedback
- MISSING INFORMATION
- CUSTOMER INFORMATION NEEDED
- EVIDENCE GAPS
- REVISION NOTES

Calculate a confidence score from 0 to 100 based on evidence support, completeness, reproducibility and internal consistency.

You must also prepare a validated downstream handoff so later email tasks can use the approved examination as their drafting base.

Routing rules:

- If confidence is greater than 95 and the case is ready for final customer communication, set NEXT ACTION to CUSTOMER_COMMUNICATION
- If the case cannot proceed because specific customer-providable information or documents are missing and the case does not otherwise require HUMAN_REVIEW, set NEXT ACTION to CUSTOMER_INFO_REQUEST
- If confidence is 95 or below, set NEXT ACTION to HUMAN_REVIEW
- Fraud or regulatory concerns may require HUMAN_REVIEW regardless of confidence

STRICT RULES:

- Do not introduce new unsupported facts
- Do not approve unsupported clauses, exclusions or payout logic
- Do not assign high confidence unless the final revised analysis is fully supported
- Do not use writing quality alone as a reason for confidence
- This is a validation task, not a new claims examination
- FORWARDED EXAMINATION must reflect the validated latest final revised examination draft only

- CUSTOMER COMMUNICATION BASIS must be customer-safe and must not expose internal workflow details
- CUSTOMER INFO REQUEST BASIS must include only customer-requestable missing information
- CUSTOMER COMMUNICATION BASIS must be "None" unless NEXT ACTION is CUSTOMER_COMMUNICATION
- CUSTOMER INFO REQUEST BASIS must be "None" unless NEXT ACTION is CUSTOMER_INFO_REQUEST

Expected Output

A final judicial assessment in exactly the following format:

VALIDATION STATUS: [APPROVED/INSUFFICIENT]

CONFIDENCE SCORE: [0–100]

NEXT ACTION: [CUSTOMER_COMMUNICATION / CUSTOMER_INFO_REQUEST / HUMAN_REVIEW]

FEEDBACK:
[Specific feedback for human review if needed, otherwise "None"]

JUDICIAL NOTES:
[Summary of final validation findings]

FORWARDED EXAMINATION:
[The validated latest final revised examination draft to be passed downstream as the canonical internal base]

CUSTOMER COMMUNICATION BASIS:
[Customer-safe approved decision basis for email drafting or "None"]

CUSTOMER INFO REQUEST BASIS:
[Exact customer-requestable missing information and documents for email drafting or "None"]

Guardrails

Pass only if:

1. VALIDATION STATUS is APPROVED or INSUFFICIENT
2. CONFIDENCE SCORE is a number from 0 to 100
3. NEXT ACTION is CUSTOMER_COMMUNICATION, CUSTOMER_INFO_REQUEST or HUMAN_REVIEW
4. All required sections are present exactly once; "None" is valid content
5. Prior judicial feedback is evaluated and remaining issues are identified
6. Routing matches the confidence and issue-severity rules
7. FEEDBACK is specific when NEXT ACTION is HUMAN_REVIEW
8. JUDICIAL NOTES are consistent with the full assessment

9. No internal-only workflow details are exposed in CUSTOMER COMMUNICATION BASIS or CUSTOMER INFO REQUEST BASIS

Fail if:

- Any section is missing
- Confidence score is invalid
- Routing contradicts the rules
- Approval is given despite unsupported conclusions
- The review introduces new unsupported facts
- The final assessment is internally inconsistent

Send Customer Decision Email

Description

Execute this task only if the final Judicial Review confirms all of the following:

1. NEXT ACTION is CUSTOMER_COMMUNICATION
2. CONFIDENCE SCORE is greater than 95
3. The case does not require CUSTOMER_INFO_REQUEST
4. CUSTOMER COMMUNICATION BASIS is provided
5. FORWARDED EXAMINATION is provided

If not all is confirmed: DO NOTHING!
Use the Judicial Review output as the only drafting base for this task:

- FORWARDED EXAMINATION = canonical validated internal claim examination
- CUSTOMER COMMUNICATION BASIS = customer-safe approved decision basis for email drafting

You must use google_gmail/send_email to actually send the email.
Actual recipient:

- n**********.com

Reference customer recipient:

- {email}

Email requirements:

- Write the entire email in {preferred_language}
- Use a translated subject equivalent to “Regarding your Claim Case {case_id}”

- Address the customer as {customer_name}
- Refer to policy/product {product}
- Base the message on CUSTOMER COMMUNICATION BASIS
- Use FORWARDED EXAMINATION only as supporting validated context
- Communicate the approved claim decision professionally
- Include the approved compensation amount prominently if applicable
- If no compensation applies, state that clearly and professionally
- Sign as {insurer} Customer Support
- Include a translated PoC note explaining that the intended recipient is {email}, but the email is being sent to the developer address for testing purposes

STRICT RULES:

- Do not send unless all routing preconditions are satisfied
- Do not use this task for CUSTOMER_INFO_REQUEST cases
- Do not use this task for HUMAN_REVIEW cases
- Do not draft from raw examiner content outside the Judge-approved handoff
- Do not invent claim details, compensation amounts or policy reasoning
- Do not expose internal workflow details such as confidence score, judicial notes, evidence gaps or feedback
- Do not use any recipient other than developer@example.com
- Do not compose without sending

EMAIL TEMPLATE:

> Dear {customer_name},
>
> We have examined your case and compared it to the terms of your policy {product}.
>
> [Professional content explaining the approved claim decision in {preferred_language}]
>
> ESTIMATED COMPENSATION:
>
> [Include the approved compensation amount prominently and clearly, if applicable. If no compensation applies, state that clearly in {preferred_language}.]
>
> [Additional professional content in {preferred_language}]
>
> Best Regards,
>
> {insurer} Customer Support
>
> P.S.: The intended recipient for this message is {email}, but because this is a proof of concept and customer email addresses are not active, this message is being sent to the developer email instead.

Expected Output

A delivery confirmation in exactly the following format:
DELIVERY STATUS: [SUCCESS or FAILED]
TOOL USED: [yes if Google Mail tool was used, no and why if not]
MESSAGE ID: [gmail message ID or delivery confirmation]
RECIPIENT: [developer@example.com]
SUBJECT: [translated subject line used]
LANGUAGE USED: [{preferred_language}]
EMAIL CONTENT:
[full email body exactly as sent]
COMMUNICATION BASIS USED:
[Copy or summary of the CUSTOMER COMMUNICATION BASIS used for drafting]
ESTIMATED COMPENSATION: [approved amount / no compensation / not applicable]
POC CONFIRMATION: [Confirm that the intended recipient was {email}, but the actual email was sent to developer@example.com for PoC purposes]

Guardrails

Pass automatically if NEXT ACTION is not CUSTOMER_COMMUNICATION.
Else:
Pass only if:

1. NEXT ACTION is CUSTOMER_COMMUNICATION
2. CONFIDENCE SCORE is greater than 95
3. CUSTOMER COMMUNICATION BASIS is provided
4. All required output sections are present exactly once
5. The email is sent using the tool
6. The email content is in the preferred language
7. The recipient is exactly developer@example.com
8. The email is based only on the approved communication basis and does not expose internal-only workflow details, except PoC information

Fail if:

- The gmail tool is not used
- The email is not sent
- The recipient is incorrect
- The content is not in the preferred language
- The email includes unsupported details, compensation amounts or policy reasoning
- Internal workflow details are exposed, except PoC information
- Any required output section is missing

Send Customer Information Request

Description

Execute this task only if the latest Judicial Review output confirms all of the following:

1. NEXT ACTION is CUSTOMER_INFO_REQUEST
2. CUSTOMER INFO REQUEST BASIS is provided
3. FORWARDED EXAMINATION is provided

If not all is confirmed: DO NOTHING!
Use the Judicial Review output as the only drafting base for this task:

- FORWARDED EXAMINATION = canonical validated internal claim examination
- CUSTOMER INFO REQUEST BASIS = exact customer-requestable missing information

Send an email to the customer explaining that claim processing is currently pending because additional information is required and specify exactly which documents or details are needed to continue.
You must use google_gmail/send_email to actually send the email.
Actual recipient:

- developer@example.com

Reference customer recipient:

- {email}

Email requirements:

- Write the entire email in {preferred_language}
- Use a translated subject equivalent to "Additional Information Needed for Claim Case {case_id}"
- Address the customer as {customer_name}
- Refer to policy/product {product} if relevant
- Base the message on CUSTOMER INFO REQUEST BASIS
- Use FORWARDED EXAMINATION only as supporting validated context
- Clearly explain that claim processing is currently pending because additional information is required
- List exactly what missing information or documents are needed
- Explain that processing can continue once the requested information is received
- Use professional, clear, customer-friendly language

- Sign as {insurer} Customer Support
- Include a translated PoC note explaining that the intended recipient is {email}, but the email is being sent to the developer address for testing purposes

STRICT RULES:

- Do not execute unless NEXT ACTION is CUSTOMER_INFO_REQUEST
- Do not use this task for final claim decision communication
- Do not use this task for HUMAN_REVIEW or CUSTOMER_COMMUNICATION cases
- Do not draft from raw examiner content outside the Judge-approved handoff
- Request only information explicitly contained in CUSTOMER INFO REQUEST BASIS
- Do not invent missing documents, facts, requirements or policy reasoning
- Do not expose internal workflow details such as confidence score, judicial notes, evidence gaps or feedback
- Do not use any recipient other than developer@example.com
- Do not compose without sending

Expected Output

A delivery confirmation in exactly the following format:

DELIVERY STATUS: [SUCCESS or FAILED]

MESSAGE ID: [gmail message ID or delivery confirmation]

TOOL USED: [yes if Google Mail tool was used, no and why if not]

RECIPIENT: [developer@example.com]

SUBJECT: [translated subject line used]

LANGUAGE USED: [{preferred_language}]

EMAIL CONTENT:
[full email body exactly as sent]

INFO REQUEST BASIS USED:
[Copy or summary of the CUSTOMER INFO REQUEST BASIS used for drafting]

MISSING INFORMATION REQUESTED:
[Exact list of missing information or documents requested from the customer]

POC CONFIRMATION: [Confirm that the intended recipient was {email}, but the actual email was sent to developer@example.com for PoC purposes]

Guardrails

Pass automatically if NEXT ACTION is not CUSTOMER_INFO_REQUEST.
Else:
Pass only if:

1. NEXT ACTION is CUSTOMER_INFO_REQUEST
2. CUSTOMER INFO REQUEST BASIS is provided
3. All required output sections are present exactly once
4. The email is sent using the tool
5. The email content is in the preferred language
6. The recipient is exactly developer@example.com
7. The email requests only the missing information or documents contained in the CUSTOMER INFO REQUEST BASIS
8. The email clearly states that processing is pending until the requested information is received
9. The email does not expose internal-only workflow details, except PoC information

Fail if:

- The gmail tool is not used
- The email is not sent
- The recipient is incorrect
- The content is not in the preferred language
- The email requests unsupported or invented information
- The email includes unsupported details, compensation amounts or policy reasoning
- Internal workflow details are exposed, except PoC information
- Any required output section is missing

Escalate to Human Review

Description

Execute this task only if at least one of the following conditions is true:

1. Judicial Review sets NEXT ACTION to HUMAN_REVIEW
2. After the examiner revision cycles, VALIDATION STATUS still remains INSUFFICIENT
3. The case involves complexity, ambiguity, repeated insufficiency, fraud concern, regulatory concern, compliance concern or other high-risk issues that cannot be safely resolved through CUSTOMER_INFO_REQUEST or EXAMINER_REVISION

Prepare a comprehensive Human-In-The-Loop escalation report for a human reviewer using the available workflow outputs.

The report must explain why automated processing could not safely complete the claim and why the case must be handled by a human instead of:

- CUSTOMER_COMMUNICATION
- CUSTOMER_INFO_REQUEST
- EXAMINER_REVISION

Use only available workflow information, including:

- case file
- examination attempts
- missing information
- evidence gaps
- judicial feedback
- revision history
- routing reason for escalation
- latest confidence score if available

STRICT RULES:

- Do not invent facts, policy clauses, exclusions, compensation amounts or reviewer conclusions
- Do not omit prior failed examination attempts or judicial feedback
- Clearly distinguish verified facts, unresolved issues and recommended reviewer actions
- Do not present the escalation report as a final claim decision
- Do not route to human review for simple missing-information cases that are better handled by CUSTOMER_INFO_REQUEST
- Include only the personal data necessary for human review
- Make clear exactly what requires human judgment or intervention

Expected Output

A comprehensive Human-In-The-Loop escalation report in exactly the following format:

CASE SUMMARY:
[Summary of the claim, policy context and current status]
EXAMINATION HISTORY:
[All examination attempts, outcomes, revisions and unresolved issues]
JUDICIAL FEEDBACK AND CONCERNS:
[Validation findings, contradictions, evidence gaps, risk factors and unresolved concerns]
CONFIDENCE AND ESCALATION REASON:

[Latest confidence score if available, plus explanation of why human review is required instead of CUSTOMER_COMMUNICATION, CUSTOMER_INFO_REQUEST or EXAMINER_REVISION]

RECOMMENDED HUMAN REVIEW ACTIONS:
[Specific actions the human reviewer should take]

PRIORITY LEVEL:
[LOW / MEDIUM / HIGH / URGENT]

Guardrails

Pass automatically if NEXT ACTION is not HUMAN_REVIEW.
Else:
Pass only if:

1. Escalation conditions are met
2. All required sections are present exactly once
3. CASE SUMMARY accurately reflects the available workflow information
4. EXAMINATION HISTORY includes all available examination attempts, revisions and outcomes
5. JUDICIAL FEEDBACK AND CONCERNS includes unresolved contradictions, evidence gaps and high-risk concerns
6. RECOMMENDED HUMAN REVIEW ACTIONS are specific and actionable
7. PRIORITY LEVEL is LOW, MEDIUM, HIGH or URGENT

Fail if:

- Escalation conditions are not met
- Any required section is missing
- The report presents unsupported conclusions as facts
- The case is escalated for simple missing-information issues that should be handled by CUSTOMER_INFO_REQUEST

Process Completion

Description

Coordinate the final outcome of the claims processing workflow.

This task is the single endpoint for all valid workflow paths. It must confirm the actual final processing result based on the completed path based only on outputs from prior workflow steps.

This task must summarise the actual completed workflow path and must not re-route the case.

Possible final paths:

1. CUSTOMER_COMMUNICATION

- Confirm that the customer decision email was successfully sent
- Confirm that the final routing approved customer communication

2. CUSTOMER_INFO_REQUEST
    - Confirm that the insufficient-information email was successfully sent
    - Confirm that the final routing required requesting additional customer information
3. HUMAN_REVIEW
    - Confirm that the case was escalated for human review
    - Confirm the reason automated processing could not safely continue

The final summary must include:

- processing path taken
- final confidence score if available
- key decision points across the workflow
- actions completed
- final case status

STRICT RULES:

- Do not re-evaluate the claim
- Do not introduce new facts, decisions or claim conclusions
- Do not claim a customer email was sent unless successful send confirmation exists
- Do not claim an insufficient-information email was sent unless successful send confirmation exists
- Do not claim human escalation unless escalation output exists
- Do not overwrite prior routing decisions
- Use only the final available workflow outputs
- This task is a final process summary only, not a new examination, judicial review or customer communication
- Do not claim human escalation without escalation output or final routing support

Expected Output

Expected Outcome:
A final processing summary in exactly the following format:

PROCESSING PATH:
[CUSTOMER_COMMUNICATION / CUSTOMER_INFO_REQUEST / HUMAN_REVIEW]

FINAL CONFIDENCE SCORE: [0–100 / Not available]

KEY DECISION POINTS:
[Chronological summary of major decisions, revisions, reviews, routing outcomes and final action taken]

ACTIONS COMPLETED:
[List of actions actually completed in the workflow]

CASE STATUS:
[Final workflow status]

Guardrails

Pass only if:

1. All required sections are present exactly once
2. PROCESSING PATH is CUSTOMER_COMMUNICATION, CUSTOMER_INFO_REQUEST or HUMAN_REVIEW
3. FINAL CONFIDENCE SCORE matches the latest judicial review score or states Not available
4. KEY DECISION POINTS are chronological and consistent with the workflow
5. ACTIONS COMPLETED lists only actions actually completed

Fail if:

- Any section is missing
- The final path is unsupported
- Customer decision email success is claimed without confirmation
- Insufficient-information email success is claimed without confirmation
- The summary introduces new unsupported conclusions
- The final summary contradicts prior workflow outputs

E Auditing Artifact Files, Figures and Examples

This appendix provides supporting material for the auditing artifact. While the main sections explain the mechanism and report the verification results, this appendix shows representative technical excerpts from the generated artifact files. Long text fields, internal outputs and repeated structures are shortened where necessary.

The auditing artifact transforms workflow events from the claim-processing PoC into tamper-evident audit evidence. It uses structured JSONL records, SHA-256 record hashes, Merkle tree commitments and a blockchain-style hash chain to link individual workflow events to a compact root commitment that can later be verified.

The examples show how workflow events are stored as hashed JSONL records, how these records are summarised in a Merkle commitment, how the Merkle root is anchored in a blockchain-style audit block, how a maliciously modified record looks and how the final verification report identifies tampering. These examples show the structure of the artifact files without reproducing every generated line in full. In addition, the full Merkle tree visualisations are included.

Full JSONL audit record example

The following example shows one representative hashed audit record from the generated JSONL audit log. In the actual file, each record is stored as one line. Here, the object is formatted across multiple lines for readability. Long prompt and task-output fields are shortened.


```
{
  "schema_version": "1.0",
  "record_id": "368e7d28-2618-4c17-b0c5-1893933b2840",
  "record_hash": "ecdefc4d2***2e6d775fa",
  "record_hash_algorithm": "sha256",
  "workflow_id": "wf-001",
  "step_id": 58,
  "event_type": "lite_agent_execution_completed",
  "parent_record_id": "496e060c-793a-4ccb-a606-eb8b2af6328a",
  "crew_id": "8a902fbbe7f6a257f51030cc2607f48e",
  "crew_name": "InsuranceClaimsProcessing
  CompleteDualRevisionSystemCrew",
  "agent_name": "Guardrail Agent",
  "agent_id": "df373e6***834dc048",
  "agent_identity": null,
  "agent_version": "1.0.0",

  "task_name": "Send Customer Decision Email",
  "task_id": "c9a6fccb-4979-47fe-b676-01945954ee48",
  "task_version": "1.0.0",
  "task_status": "completed",

  "input_messages": [
    "Ensure the following task result complies with the given guardrail.

     Task result:
     (DO NOTHING -- Routing and Guardrails strictly require explicit confirmation of:
     NEXT ACTION = CUSTOMER_COMMUNICATION, CONFIDENCE SCORE > 95, CUSTOMER COMMUNICATION BASIS
     provided, and FORWARDED EXAMINATION provided. These are NOT explicitly present in
     the Judicial Review output. As instructed, I must NOT proceed or compose/send
     any email in this case.) ... [shortened]"
  ],

  "output": {
    "valid": true,
    "feedback": null
  },
```

```
  "guardrail_version": "1.0.0",
  "guardrail_status": "success",
  "guardrail_failure_reason": null,
  "guardrail_retry_count": 1,

  "started_at": "2026-05-07T00:55:42.436Z",
  "ended_at": "2026-05-07T00:55:42.887Z",
  "tool_invocation": null
}
```


This record illustrates that the audit log captures both workflow metadata and validation information. It identifies the workflow, step, task, agent, parent record, execution status, Guardrail result and timestamps. The `record_hash` field is the SHA-256 commitment for the record content and later becomes one leaf in the Merkle tree.

Merkle commitment file excerpt

The generated Merkle commitment records the construction metadata and the Merkle root for the complete audit log. The full commitment contains 78 leaf entries. The excerpt below keeps the main metadata and selected leaves to show the structure without repeating all records.


```
{
  "workflow_id": "wf-001",
  "source_file": "audit_logs/audit_log_hashed.jsonl",
  "hash_algorithm": "sha256",
  "leaf_count": 78,
  "tree_depth": 8,
  "merkle_root": "3d0f30145bf11c2d89c9ad4d4a52f21dd0fe5faed324895e13eb4724d6d0c416",
  "created_at": "2026-05-19T15:17:47.835768Z",
  "leaf_order": "jsonl_line_order",
  "odd_leaf_strategy": "duplicate_last_leaf",

  "leaves": [
    {
      "line_number": 1,
      "step_id": 0,
      "record_id": "a6d0f4a5-3d4a-4d46-966b-49ba8ca6fe12",
      "record_hash": "5c037aa6e3b04ac223d9b848dbd952a538ff9e72387b74001492b39dfe0d18ce"
    },
    {
      "line_number": 2,
      "step_id": 1,
      "record_id": "113139c2-930a-4cb8-9a49-351a097931c2",
      "record_hash": "592ca7af8b6d53a22956e347ae95f60a9a4bb1336432c775cae808981b2f8d5d"
    },

    "... 74 further leaf entries omitted ...",

    {
      "line_number": 77,
      "step_id": 76,
      "record_id": "fb645fe6-95a7-4810-8d88-59c364aa2b4f",
      "record_hash": "02c29df7b8af7299189319caf173eb30df8ca48d7f54ca0cd5c474f930ef7fc2"
    },
    {
      "line_number": 78,
      "step_id": 77,
      "record_id": "02826366-b400-4a74-8289-e6d16cf71af8",
      "record_hash": "91e5b63ac853366b8b8959a77a70f6f2968e397ab5efc2d29bc0d0254f7c5d13"
    }
  ]
}
```


The commitment stores the hash algorithm, the number of leaves, the tree depth, the leaf ordering strategy and the strategy for uneven tree levels. The value `duplicate_last_leaf` means that, if a level has an uneven number of nodes, the last node is duplicated and paired with itself. This duplicate is only a structural copy and not an additional audit record.

Full blockchain JSONL example

The following example shows one block from the blockchain-style audit chain. In the generated file, each block is stored as one JSONL line. The block contains the committed Merkle root, construction metadata, the previous block hash and its own block hash.


```
{
  "block_schema_version": "1.0",
  "block_number": 9,
  "workflow_id": "wf-001",
  "created_at": "2026-05-12T01:21:39.046402Z",
  "previous_block_hash": "33d808a0a9218db1372dd1193782aaffb9fdeb472bffcb6225a354c56378d4cb",
  "merkle_root": "3d0f30145bf11c2d89c9ad4d4a52f21dd0fe5faed324895e13eb4724d6d0c416",
  "merkle_hash_algorithm": "sha256",
  "leaf_count": 78,
  "tree_depth": 8,
  "leaf_order": "jsonl_line_order",
  "odd_leaf_strategy": "duplicate_last_leaf",
  "source_commitment_file": "../audit_merkle_commitment.json",
  "source_jsonl_file": "../audit_logs/audit_log_hashed.jsonl",
  "block_hash": "5c43111afb79e7c2489e8360faff3b67d3153d5f53d608ae4adc1b148cde9852"
}
```


This block links the current Merkle commitment to the previous audit block. If an earlier block is modified, its recomputed hash changes and the following block no longer points to the correct predecessor.

Full malicious audit record example

The malicious audit records were created by modifying selected fields after the original hashes and Merkle commitment had already been generated. The example below shows the first manipulated record. The ID was changed to `MALICIOUSLY_CHANGED_RECORD_ID`, while the stored `record_hash` still corresponds to the original record. During verification, this causes a record-hash mismatch.


```
{
  "schema_version": "1.0",
  "record_id": "MALICIOUSLY_CHANGED_RECORD_ID",
  "record_hash": "5c037aa6e3b04ac223d9b848dbd952a538ff9e72387b74001492b39dfe0d18ce",
  "record_hash_algorithm": "sha256",
  "workflow_id": "wf-001",
  "step_id": 0,
  "event_type": "crew_kickoff_completed",
  "parent_record_id": null,

  "crew_id": "8a902fbbe7f6a257f51030cc2607f48e",
  "crew_name": "InsuranceClaimsProcessingCompleteDualRevisionSystemCrew",

  "agent_name": null,
  "agent_id": null,
  "agent_identity": null,
  "agent_version": null,
  "task_name": null,
  "task_id": null,
  "task_version": null,
  "task_status": null,

  "input_messages": [
    "Hallo zusammen,

     Am 03. Februar 2026 gegen 18:30 Uhr war ich mit meinem Hund in meiner
     Wohnung. Waehrend ich kurz in der Kueche war, sprang mein Labrador auf
     das Sofa im Wohnzimmer und beschaedigte mehrere Gegenstaende ...

     [claim text shortened]",

    {
```

```
      "inputs": {
        "case_id": "1fa12b0b7c2fc00b07c119757c9d7107",
        "customer_name": "Noah Meyer",
        "email": "noah.meyer@yahoo.com",
        "preferred_language": "de",
        "insurer": "Allianz",
        "product": "Hausrat Komfort",
        "file_path": "Insurance_Knowledge_Base/Allianz/allianz__hausrat-komfort__terms...",
        "sha256": "b110a5d24ae15576fe71459f75305af1c28082b923b0348305fe7a92b32449dc"
      }
    }
  ],

  "output": "PROCESSING PATH: CUSTOMER_COMMUNICATION

             FINAL CONFIDENCE SCORE: 100

             KEY DECISION POINTS:
             - Deckungspruefung and exclusion check were completed.
             - Internal validations confirmed the exclusion.
             - Customer communication was prepared, but no email was sent.

             [output shortened]",

  "guardrail_version": null,
  "guardrail_status": null,
  "guardrail_failure_reason": null,
  "guardrail_retry_count": null,
  "started_at": "2026-05-07T00:53:57.120Z",
  "ended_at": "2026-05-07T00:56:24.855Z",
  "tool_invocation": null
}
```


The malicious record demonstrates that even a small metadata change is detected when the record hash is recomputed. Since the changed record hash becomes a different Merkle leaf, the recomputed Merkle root also differs from the committed root.

Full verification report

The verification report compares the original audit log with the malicious audit log. The original chain verifies successfully. The malicious chain fails because three modified records no longer match their stored hashes. The leaf count still verifies because no record was added or removed.


```
{
  "report_schema_version": "1.0",
  "created_at": "2026-05-19T15:17:48.667350Z",
  "workflow_id": "wf-001",
  "tampered_node_indexes": [
    0,
    5,
    20
  ],
  "files": {
    "original_jsonl": "audit_logs/audit_log_hashed.jsonl",
    "malicious_jsonl": "audit_logs/audit_log_malicious.jsonl",
    "merkle_commitment": "audit_logs/audit_merkle_commitment.json",
    "blockchain": "audit_logs/audit_blockchain.jsonl",
    "report": "audit_logs/audit_verification_report.json"
  },
  "original_chain": {
    "verified": true,
    "record_count": 78,
    "record_hashes_verified": true,
    "merkle_root_verified": true,
    "leaf_count_verified": true,
    "blockchain_verified": true,
    "committed_merkle_root": "3d0f30145bf11c2d89c9ad4d4a52f21dd0fe5faed324895e13eb4724d6d0c416",
    "recomputed_merkle_root": "3d0f30145bf11c2d89c9ad4d4a52f21dd0fe5faed324895e13eb4724d6d0c416",
```

```
    "latest_block_hash": "0f29575560615c31e3806c32e0f10454572d0f543ad80e8aa19561a76a48b102",
    "errors": {
      "records": [],
      "blockchain": []
    }
  },
  "malicious_chain": {
    "verified": false,
    "tampering_detected": true,
    "record_count": 78,
    "record_hashes_verified": false,
    "merkle_root_verified": false,
    "leaf_count_verified": true,
    "blockchain_commitment_verified": false,
    "committed_merkle_root": "3d0f30145bf11c2d89c9ad4d4a52f21dd0fe5faed324895e13eb4724d6d0c416",
    "recomputed_malicious_merkle_root": "bd7bf191d1cfd36aa1f7c231f71b95c92516366b046156d962ba4691b58b1df4",
    "failed_records": [
      {
        "line_number": 1,
        "step_id": 0,
        "record_id": "MALICIOUSLY_CHANGED_RECORD_ID",
        "stored_hash": "5c037aa6e3b04ac223d9b848dbd952a538ff9e72387b74001492b39dfe0d18ce",
        "recomputed_hash": "16440f50af724fda106e73369ed29552827fa5a77ea1b7b8880d4859597b4e0b",
        "reason": "record_hash_mismatch"
      },
      {
        "line_number": 6,
        "step_id": 5,
        "record_id": "8030a732-a432-4fd1-b599-a8f32109fe94",
        "stored_hash": "c5ef12174a6e799dc3ed12823f21d15093fa77d63ffe118b3747b5cc088aafac",
        "recomputed_hash": "d218a7ec1f70e833f68ebeaf2aae68786c14f7149a649e8fbd3e7962dfd8c288",
        "reason": "record_hash_mismatch"
      },
      {
        "line_number": 21,
        "step_id": 20,
        "record_id": "305c9fb0-9b4a-40e7-b438-5027038a91a1",
        "stored_hash": "2f1ae9c93f2fd6d7bad64c8a15208b946f50592a441210b57681bcc1086a78d1",
        "recomputed_hash": "0ae5202e4c910395256b31bbadcf8ec2edf7e5e8c8b2d30dabb8197007a23191",
        "reason": "record_hash_mismatch"
      }
    ]
  }
}
```


The report confirms the expected tamper-detection behaviour. The original audit log still matches the committed Merkle root and the blockchain-style chain. The malicious log keeps the same number of records, but three records fail hash verification. These record-level changes propagate to a different recomputed Merkle root, so the malicious log no longer corresponds to the original commitment. Unlike a simple file comparison, the Merkle tree commitment can be stored and verified independently while still supporting record-level tamper localization. For larger audit logs, Merkle trees can also support more efficient partial verification when intermediate nodes or proofs are preserved, because not every record needs to be compared against a complete reference copy.

Full Merkle Tree Visualisation

This subsection contains the full Merkle tree visualisations generated for the auditing artifact as supporting material. They provide a complete view of how all audit record hashes are arranged and propagated toward the Merkle root. The nodes and hashes in level 0 represent the log entries.

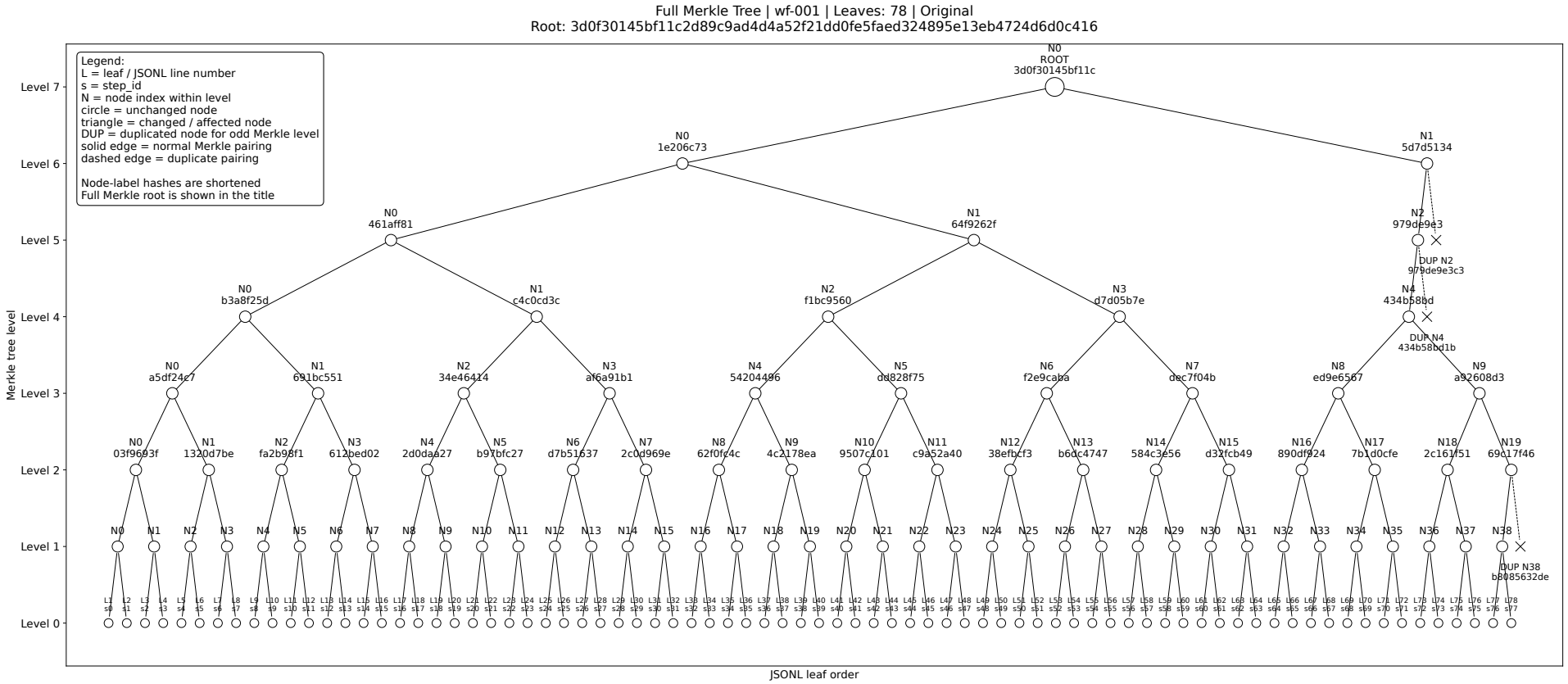


Figure E.17: Full Merkle tree for the original audit log

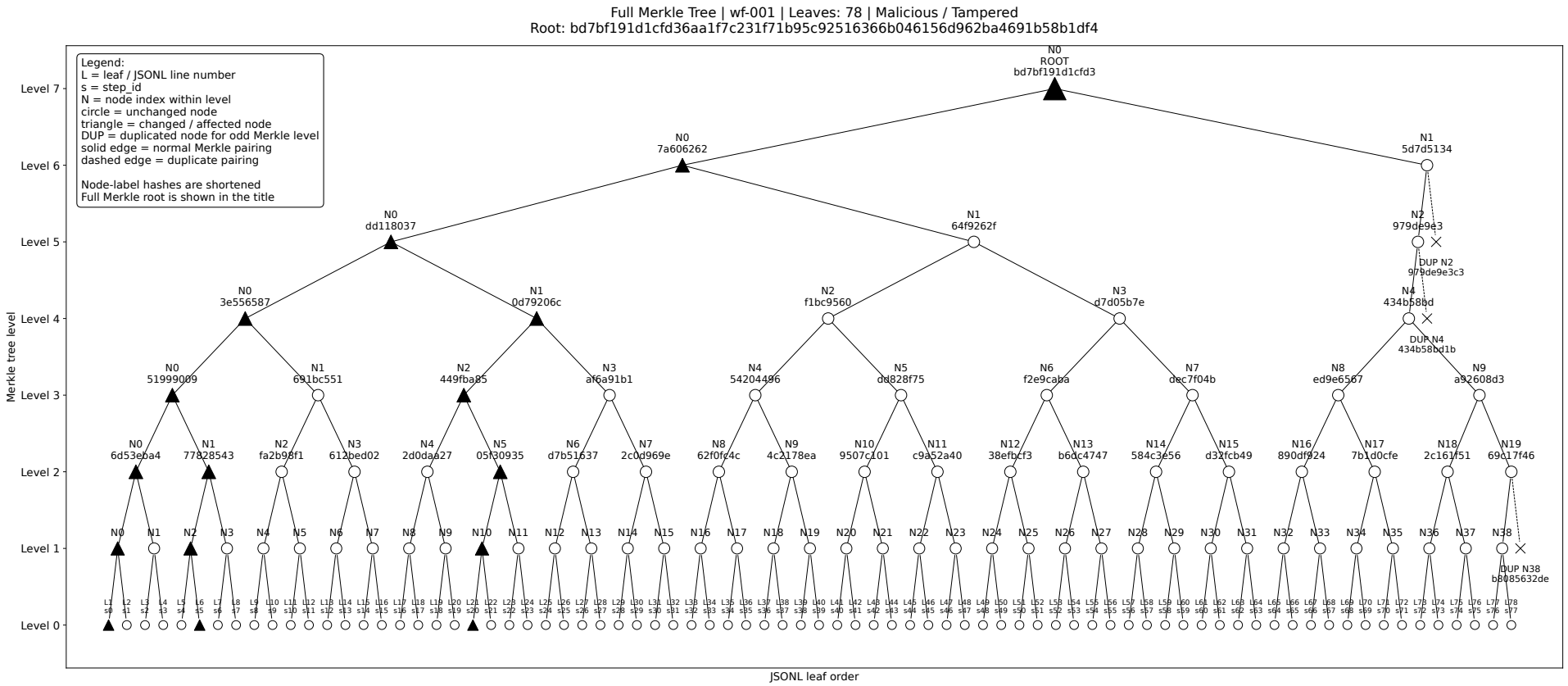


Figure E.18: Full Merkle tree for the malicious audit log

F GitHub Repositories

This appendix provides links to the GitHub repositories associated with this thesis. The repositories contain the Knowledge Source with scripts to prepare and transform the data and documents, as well as the WebApp and Auditing Merkle tree artifact. They are included to support transparency, reproducibility and further evaluation of the implemented workflow.

- **Master-Thesis-sMAS-Knowledge-Source**
  Knowledge source for the Claims Process Automation Workflow PoC.
  `https://github.com/1Zero64/Master-Thesis-sMAS-Knowledge-Source`
- **Master-Thesis-sMAS-Claim-App**
  Python WebApp to send claims to the CrewAI automation workflow.
  `https://github.com/1Zero64/Master-Thesis-sMAS-Claim-App`
- **Master-Thesis-sMAS-Merkle-Tree-Auditing**
  Demonstrator of a tamper-evident audit mechanism for structured multi-agent workflow logs via Merkle trees.
  `https://github.com/1Zero64/Master-Thesis-sMAS-Merkle-Tree-Auditing`